A BUILD-ALONG COURSE

Building an ORM on tokio-postgres

Derive macros, a model layer and a typed query builder on tokio-postgres, built from scratch as shelf — for readers of Async PostgreSQL in Rust.

Contents

0 Introduction and setup 1
Contents 1
0.1 What an ORM does, and what shelf will not do 2
0.2 Where shelf sits 3
0.3 The finished API in ten lines 3
0.4 Three crates, and why 4
0.5 How this book follows the build 5
0.6 Setup 6
0.7 The first derive, end to end 7
Summary 9
Exercises 9

1 Mapping rows by hand 12
1.1 A trait of shelf's own 12
1.2 By name or by position 14
1.3 Three ways to fail 17
1.4 Nested structs 19
1.5 Fetch helpers 20
1.6 Counting the boilerplate 22
In stockroom 24
When not to use a mapping trait 25
Summary 25
Exercises 25

2 Declarative macros 28
2.1 Matching tokens 28
2.2 A `model!` macro 29
2.3 Tuples, one line per arity 31
2.4 Hygiene, and `$crate` 32
2.5 Where declarative macros stop 34
In shelf 37
When not to use `macro_rules!` 37
Summary 38
Exercises 38

3 The first derive macro 42
3.1 Proc-macro crates 42
3.2 Three crates 44
3.3 Parsing with syn 3 44
3.4 Generating with quote 46
3.5 Hygiene 48
3.6 Seeing the expansion 50
3.7 Errors as values 51
In shelf 53

When not to write a derive 55
Summary 55
Exercises 56

4 Attributes, errors and generics 59
4.1 Helper attributes 59
4.2 Parsing with `parse_nested_meta` 60
4.3 The six attributes 64
4.4 Every error at once 69
4.5 UI tests 71
4.6 Generics 73
4.7 What darling would save 76
In shelf 81
When not to use an attribute 82
Summary 82
Exercises 83

5 Checking a statement once, not every row 86
5.1 What a lookup by name costs, again 86
5.2 A statement knows its columns 86
5.3 Types checked once 87
5.4 What the derive generates 87
5.5 Measured 87
5.6 Where it is used 87
The programs 87
Side by side 88
In shelf 88
Claims 88
When not to resolve once 88
Exercises 88

6 Tables: the model derive 89
6.1 What a table adds 89
6.2 `#[derive(Table)]` 89
6.3 SQL at compile time 90
6.4 Reading 90
6.5 Metadata as consts, and where they stop 90
The programs 90
Side by side 90
The data 91
Claims 91
When not to use `Table` 91
Exercises 91

7 The executor and shelf's errors 92
7.1 Why not `GenericClient` 92
7.2 The `Executor` trait 92
7.3 Three ways to send a statement 93
7.4 Transactions 93
7.5 `shelf::Error` 93
The programs 93
Side by side 94
Claims 94

When not to use the executor 94
Exercises 94

8 Writing rows: insert, update, delete, upsert 95
8.1 `#[derive(Insert)]` 95
8.2 Database defaults 96
8.3 Update and delete 96
8.4 Upsert 96
8.5 Constraint errors 96
The programs 96
Side by side 96
In stockroom 96
The data 97
Claims 97
When not to use shelf's writes 97
Exercises 97

9 Partial updates and optimistic locking 98
9.1 Absent or null 98
9.2 Three designs 98
9.3 Version columns 99
9.4 `updated_at` 99
9.5 ETags in axum 99
The programs 99
Side by side 100
In stockroom 100
Claims 100
When not to use `Patch` 100
Exercises 100

10 A typed query builder 101
10.1 What to type 101
10.2 Column constants 101
10.3 Expressions 102
10.4 Owned parameters 102
10.5 Order, limit and pages 102
10.6 What fails to compile 102
10.7 The escape hatch 102
The programs 102
Side by side 103
In stockroom 103
Claims 103
When not to use the builder 103
Exercises 103

11 Relations without N+1 104
11.1 Declaring relations 104
11.2 N+1, measured 104
11.3 Batched loading 105
11.4 Pipelining independent loads 105
11.5 Joins 105
11.6 Why no lazy loading 105
The programs 105

Side by side 106
The data 106
Claims 106
When not to use relation loaders 106
Exercises 106

12 Bulk writes and streams 107
12.1 `insert_many` 107
12.2 Binary COPY from the derive 108
12.3 Measured 108
12.4 Streams of models 108
12.5 A COPY rejected at its start 108
The programs 108
Side by side 108
In stockroom 109
Claims 109
When not to use shelf's bulk writes 109
Exercises 109

13 Trusting the mapping: schema checks and tests 110
13.1 What can drift 110
13.2 `verify` 110
13.3 Where to run it 111
13.4 Testing a derive 111
13.5 Compile-time checking 111
The programs 111
Side by side 111
Claims 112
When not to use `verify` 112
Exercises 112

14 Capstone: stockroom on shelf 113
14.1 The shape 113
14.2 Models for a real schema 113
14.3 Line for line 114
14.4 Where shelf steps aside 114
14.5 What a production ORM would add 114
The programs 114
Side by side 114
The data 114
Claims 114
When not to use shelf 115
Exercises 115

Appendix — attribute reference, generated code and phrasebook 116
A Attribute reference 116
B What each derive generates 117
C Phrasebook 117
D Glossary 118
E Further reading 118

CHAPTER 0

Introduction and setup

Async PostgreSQL in Rust taught tokio-postgres as a driver: you wrote every SQL statement yourself, and you turned every row into a struct with a `TryFrom<&Row>` that named each column again (main book §3.5). This book builds the layer that writes that code for you. **shelf** is a small ORM on tokio-postgres: derive macros that map rows to structs, a model layer that writes the everyday SQL, partial updates with optimistic locking, a typed query builder, relations without N+1 queries, bulk writes, streams and a schema check. You build it one chapter at a time, and every stage of it is in this repository, compiled and tested.

The goal is not to replace diesel or SeaORM. It is to know what an ORM does underneath, by building one on a driver you already understand: what a derive macro receives and returns, how it reports a mistake at the right line, what SQL a model method sends and how many round trips it costs, and where hand-written SQL is still the better tool.

Contents

Chapter	You will learn
0 · Introduction and setup (this page)	What an ORM does, where shelf sits, the three crates, how the book follows the build, and the setup
1 · Mapping rows by hand	A `FromRow` trait, reading by name or by position and what each costs, NULL, nested structs, a fetch helper
2 · Declarative macros	`macro_rules!`, a `model!` macro, `FromRow` for twelve tuple sizes from one macro, hygiene, and where it stops
3 · The first derive macro	Proc-macro crates, syn and quote, the three-crate split, hygiene, seeing the expansion, errors that point at your code
4 · Attributes, errors and generics	`#[shelf(…)]`: `rename`, `skip`, `flatten`, `json`, `try_from`; several errors at once, UI tests, generics
5 · Checking a statement once	Resolving columns and checking their types once per statement instead of on every row, measured
6 · Tables	`#[derive(Table)]`, primary keys, SQL written at compile time, `find` and `all`
7 · The executor and shelf's errors	One trait over `Client`, `Transaction` and deadpool, statement strategies measured, `shelf::Error`
8 · Writing rows	`#[derive(Insert)]`, database defaults, `RETURNING`, updates, deletes and upserts
9 · Partial updates and optimistic locking	`#[derive(Patch)]` three ways, `Option<Option<T>>`, version columns, `updated_at`
10 · A typed query builder	Typed columns, expressions, `$n` numbering, keyset pages, and what fails to compile
11 · Relations without N+1	`belongs_to` and `has_many`, the N+1 problem measured, `= ANY($1)`, pipelined loads, joins
12 · Bulk writes and streams	`insert_many` through `unnest`, binary COPY written by the derive, streams of models

Chapter	You will learn
13 · Trusting the mapping	Checking models against the live schema, testing macros, and what compile-time checks add
14 · Capstone	Part of stockroom rewritten on shelf behind the same routes, compared line for line
A · Appendix	An attribute reference, what each derive generates, a phrasebook, a glossary

The whole book is also a single PDF, shelf-book.pdf. A chapter that is not written yet appears there, and in this repository, as a **sketch**: a plan of the chapter in its place, which the chapter replaces when it is written. orm/README.md shows which is which.

Part I (Chapters 1–5) is the mapping: from a hand-written trait, through `macro_rules!`, to a derive that checks a whole statement before it reads the first row. Part II (6–9) is the model: tables, writing rows, partial updates. Part III (10–14) is queries, relations, bulk work, and the capstone. Read them in order: each chapter grows the crate the previous one left.

0.1 What an ORM does, and what shelf will not do

An object-relational mapper does three jobs, and the main book did all three by hand:

1. **It maps rows to structs**, and structs back to parameters. That is the `TryFrom<&Row>` stockroom has for every type it reads, and the `&[&(dyn ToSql + Sync)]` list it builds for every `INSERT`.
2. **It writes the everyday SQL**: `SELECT … WHERE id = $1`, `INSERT … RETURNING`, `UPDATE … SET`, `DELETE`. Their shape follows from the struct and the table, and nobody enjoys writing them for the twentieth type.
3. **It follows relations** between tables: an order's lines, a line's product. Done naively, that is one query per row, the N+1 problem. Done well, it is one query per relation.

shelf does all three, in that order: Part I maps rows, Part II writes the everyday SQL, and Part III follows relations. It also leaves things out, each for a reason:

Left out	Why
Migrations, and DDL generated from structs	A schema is better written as SQL, in migrations (main book Chapter 14)
Checking queries against a live database at compile time	sqlx's `query!` and Cornucopia do it; Chapter 13 says what it adds and what it costs
Lazy loading, an identity map, change tracking, a unit of work	Each hides round trips or state; Chapter 11 says why async Rust makes lazy loading a poor fit
Other databases, and a blocking API	shelf is an ORM for tokio-postgres; abstracting over drivers would hide what the driver does
Postgres enums, composites and domains	postgres-types' derives already map them (main book Chapter 5), and shelf passes them through
Hooks such as `before_insert`	A plain function call before the write does the same, and you can see it

One thread runs through the whole book: **every shelf method says what SQL it sends and how many round trips it costs**, and every chapter ends by saying when not to use what it built. An ORM you cannot predict is an ORM you end up fighting. Building one is the surest way never to be surprised by the one you use.

0.2 Where shelf sits

Rust already has libraries that do some of these jobs. They differ less in features than in two decisions: who writes the SQL, and when a mistake in the mapping is caught.

Library	Built on	You write	A mapping mistake is caught
diesel, with diesel-async	tokio-postgres (diesel-async's PostgreSQL connection)	Rust: a schema (`table!`, usually generated from the database) and queries in its builder	At compile time, against the `table!` schema
SeaORM	sqlx	Entities (the `DeriveEntityModel` derive), `ActiveModels` for writes, queries in its builder	At run time
sqlx	its own driver	SQL; `#[derive(FromRow)]` maps rows	At compile time with `query!` and `query_as!` (against a live database, or query data saved from one); at run time with `FromRow`, on every row
postgres-from-row	tokio-postgres	SQL; `#[derive(FromRow)]` maps rows	At run time, on every row
Cornucopia	tokio-postgres	SQL in `.sql` files, from which Rust functions are generated	When the code is generated, against a live database
shelf	tokio-postgres	SQL where it matters; derives write the everyday statements	At run time, once per statement, before the first row is read

A newer entrant, **toasty**, is the Tokio project's ORM: models are structs with `#[derive(Model)]`, behind one API for SQL databases and DynamoDB, and its PostgreSQL driver holds a `tokio_postgres::Client`. It is young (0.11, with breaking changes in each release). The main book's Appendix F has the versions, dates and details of every crate in this section that is built on tokio-postgres. The sqlx and SeaORM illustrations in this book follow sqlx 0.9 and SeaORM 2.0.

What shelf decides differently, and why it is worth building rather than adopting:

- **The SQL stays visible.** A derive writes statements you can predict, and each chapter prints the ones it generates. Anything else is your SQL, mapped by the same derive. There is no second query language to learn for the hard cases.
- **The mapping is checked at run time, but once.** shelf compares the struct with the statement's column list when the statement is prepared, so a renamed column or a wrong type is an error that names the struct, the field, the column and both types, even for a query that returns no rows (Chapter 5). Checking at compile time catches more, and costs a database at build time; Chapter 13 weighs the two.
- **tokio-postgres is underneath, with nothing hidden.** shelf takes the `Client` and `Transaction` you already have (and deadpool's connections, Chapter 7). Pipelining, COPY, `LISTEN` and everything else in the main book still work on the same connection.

0.3 The finished API in ten lines

By the end of Part III, a model and its everyday SQL look like this. It is the target design, and Chapters 6–13 build it, so here it is an illustration, not a program. §0.7 runs the part that exists today, and this section becomes a runnable program when the rest is built.

```rust
// shelf (target design: built in Chapters 6–13)
#[derive(Debug, FromRow, Table)]
#[shelf(table = "products")]
struct Product {
    #[shelf(primary_key, generated)]
    id: Uuid,
    sku: String,
    price_cents: i64,
    tags: Vec<String>,
    #[shelf(version)]
    version: i32,
    #[shelf(generated)]
    created_at: Timestamp,
}

#[derive(Insert)]
#[shelf(model = Product)]
struct NewProduct { sku: String, price_cents: i64, tags: Vec<String> }

let product = NewProduct { … }.insert(&db).await?;          // INSERT … RETURNING
let product = Product::find(&db, &id).await?;                 // Err(Error::NotFound)
let page = Product::query()
    .filter(Product::PRICE_CENTS.gt(1_000).and(Product::TAGS.contains("sale")))
    .order_by(Product::SKU.asc())
    .limit(20)
    .all(&db)
    .await?;
let lines = Order::load_lines(&db, &orders).await?;           // one query: = ANY($1)
NewProduct::insert_many(&db, &batch).await?;                  // unnest; copy_in for binary
COPY
shelf::verify::<Product>(&db).await?;                         // the struct against the
table
```

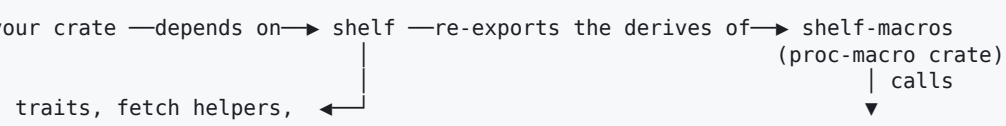
Each line is one statement, and one round trip once the statement is cached:

- `insert` sends an `INSERT … RETURNING` with the model's columns, and the database fills in `id` (with `uuidv7()`) and `created_at` (Chapter 8).
- `find` sends `SELECT <every column> FROM products WHERE id = $1` and turns "no row" into an error you can match (Chapter 6).
- The query builder renders one `SELECT` with `$1`, `$2` and `$3` as parameters. Its columns are typed, so comparing `PRICE_CENTS` with a string does not compile (Chapter 10).
- `load_lines` reads the lines of every order in the slice with one query, not one per order (Chapter 11), and `insert_many` writes the whole batch with one statement (Chapter 12).
- `verify` compares the struct with the table, so drift is caught at startup instead of at the first request that reads the column (Chapter 13).

0.4 Three crates, and why

A derive macro is a function that the compiler calls with the tokens of your struct, and that returns new tokens. Rust requires that function to live in a crate of its own kind, a *proc-macro crate*, which can export macros and nothing else. That is inconvenient: code that generates code is code you want to test and to show, and nothing outside the compiler can call a function in a proc-macro crate, neither a test in another crate nor an example in this book.

So shelf is three crates, and the code that writes code lives in the one that is an ordinary library:





Crate	Kind	What it holds
`shelf`	library	What you depend on: the traits and functions your code calls, the error type, and the derives, re-exported. A hidden module, `__private`, holds what generated code names, so your crate does not need its own dependency on tokio-postgres
`shelf-macros`	proc-macro	A shell: per derive, three lines that parse the input, call shelf-codegen, and return the result or the error
`shelf-codegen`	library	Everything the derives generate, as functions from a parsed struct to tokens. Its own tests, and this book's examples, call them directly

A fourth crate, `orm/playground`, holds this book's examples. Chapter 3 builds the split from the inside, and explains the one type that makes it possible, `proc_macro2::TokenStream`.

0.5 How this book follows the build

A build-along book has a problem: each chapter shows the crate at one stage, but the repository holds the finished crate. If the book quoted only the finished code, Chapter 3 would show attribute parsing that Chapter 4 explains, and checks that Chapter 5 motivates. This book handles it in four ways:

1. **Stage programs.** Each new mechanism appears first as a complete, runnable program in one file: a hand-written impl, a `macro_rules!` macro, or a derive's logic written as an ordinary function from a parsed struct to tokens. A proc macro cannot be defined in a single file, but its logic can, because syn and quote work in any program. So even the macro code in this book is compiled, run and checked.
2. **The finished crate, in excerpts.** Then the chapter shows how shelf does it, in excerpts whose first line names the file. A tool checks every excerpt against the file it quotes, and code that later chapters add is elided with `// …`.
3. **Behaviour, through the finished crate.** Claims about what shelf does are runnable programs that use `#[derive(shelf::FromRow)]` and friends against a real database.
4. **Nothing generated is typed by hand.** What a derive expands to is printed by running shelf-codegen, and compiler errors are pasted from a real build, as the main book pasted psql output.

Each chapter then follows the main book's pattern. **Side by side** sections solve one task three ways: **by hand** with tokio-postgres, as the main book would; with **shelf**; and with an established crate where it fits the topic (sqlx for row mapping, SeaORM for models and relations, diesel for the query builder). The first two are runnable programs; the third is an illustration, not compiled, and its first line names the crate. A table of what is the **same** and what is **different** follows. **In shelf** sections show the finished crate, an **In stockroom** section appears where stockroom's hand-written code is the comparison, and a section on **when not to** use what the chapter built comes before the summary and the exercises.

Five kinds of boxes appear where they are earned. The first four are the main book's:

> **Protocol lens:** what travels over the socket.

> **SQL lens:** what the server does with it.

> **Rust refresher:** a language feature you may be hazy on, where it first matters: `PhantomData`, async methods in traits, `LazyLock`.

> **Pitfall:** a mistake that compiles, passes a quick test, and fails in production.

The fifth is new:

> **Macro lens:** what the compiler hands a macro and what it takes back: tokens, spans, hygiene, and the order in which things expand.

Code blocks come in the main book's two kinds. A plain `rust` block is a complete program that was compiled, linted and run against a fresh database, and its `assert!`s say what it produces. A block whose first line names a file, a crate or "illustration" is an excerpt or a sketch, and is not compiled on its own.

0.6 Setup

What you need

The main book's setup, from its Chapter 0: PostgreSQL 18, a database for the examples named by `DATABASE_URL`, and Rust 1.85 or newer for the 2024 edition. This book was written with rustc 1.99.1.

shelf is not published on crates.io. It lives in this repository, next to the main book's workspace, and so do its examples. Work from a clone of the repository; every command below runs from its root.

The playground

The book's examples run in the crate `orm/playground`, whose package is `shelf-playground`. It depends on `shelf` and `shelf-codegen` by path, on the macro crates syn, quote, proc-macro2 and prettyplease, and on what the main book's playground has (tokio, tokio-postgres with its features, serde, jiff, uuid, deadpool-postgres). Unlike the main book's playground, it takes every version from the workspace's `Cargo.toml`, so there is nothing to copy.

Its library re-exports the main book's helpers, so an example connects with `shelf_playground::connect()`, which reads `DATABASE_URL` exactly as the main book's `connect()` does. It adds one helper of its own, `pretty`, which formats generated code; Chapter 3 prints it.

Each runnable block in a chapter becomes one file in `orm/playground/examples/`, named after the chapter and its position: the first program in §0.7 is `shelf_ch00_01`. To run one:

```
python3 tools/book.py --book orm extract                # write orm/playground/examples/
from the book
export DATABASE_URL=postgres://postgres:secret@localhost:5432/tpbook
cargo run -p shelf-playground --example shelf_ch00_01
```

> **Pitfall:** `orm/playground/examples/` is generated, and `extract` (like `check`) deletes every `.rs` file there that the book did not produce. Keep your own programs in `orm/playground/src/bin/` instead: `orm/playground/src/bin/mine.rs` runs with `cargo run -p shelf-playground --bin mine`.

To check a chapter the way it was checked when it was written, let `book.py` build it, lint it with clippy, and run every program against a database it creates for the purpose, `tpbook_shelf_check`. It needs `ADMIN_DATABASE_URL`, a connection with the right to create databases, which the repository's `.env.example` describes:

```
python3 tools/book.py --book orm check --chapter 0       # one chapter's examples
python3 tools/book.py --book orm check                   # the whole book
```

shelf's own tests

shelf is tested apart from the book: unit tests, tests that pin down what each derive expands to, tests that pin down the compiler errors it causes, and integration tests that give each test its own schema in a database named `shelf_test`. Chapter 13 explains each kind.

```
cargo test -p shelf -p shelf-codegen                     # needs ADMIN_DATABASE_URL
```

0.7 The first derive, end to end

Here is shelf as it stands at the end of Part I. `#[derive(FromRow)]` reads each named field from the column of the same name. This program derives it for a struct and reads two rows into it with `shelf::fetch_all`:

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
struct Product {
    id: i64,
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 price_cents bigint NOT NULL,
                 tags text[]
             );
             INSERT INTO products (sku, price_cents, tags)
             VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499, NULL);",
        )
        .await?;

    let products: Vec<Product> = shelf::fetch_all(
        &client,
        "SELECT id, sku, price_cents, tags FROM products ORDER BY id",
        &[],
    )
    .await?;

    println!("{products:#?}");
    assert_eq!(products.len(), 2);
    assert!(products[0].id < products[1].id);
    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[0].price_cents, 1299);
    assert_eq!(products[0].tags, Some(vec!["kitchen".to_string()]));
    assert_eq!(products[1].tags, None);
    Ok(())
}
```

```
[
    Product {
        id: 1,
        sku: "MUG-01",
        price_cents: 1299,
        tags: Some(
            [
                "kitchen",
            ],
        ),
    },
    Product {
        id: 2,
        sku: "TEE-02",
        price_cents: 2499,
        tags: None,
    },
]
```

`use shelf::FromRow` brings in two things with one name: the trait and the derive macro. That works because traits and macros live in different namespaces, and it is how serde's `Serialize` works too. The NULL in `tags` became `None` because the field is an `Option`, exactly as with `row.try_get` in the main book.

What did the derive write? Because the code generation lives in `shelf-codegen`, an ordinary library (§0.4), this program can call it on a struct and print the result:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            sku: String,
        }
    };
    let generated = shelf_codegen::from_row::expand(&input).expect("a struct with named
fields");
    let code = shelf_playground::pretty(generated);
    println!("{code}");
    assert!(code.contains(r#"column::<String>(columns, "Product", "sku", "sku")?"#));
}
```

It prints the `impl` that the compiler sees next to the struct in place of `#[derive(FromRow)]`:

```
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
        })
    }
}
```

Three things in it are worth noticing now, and each gets a chapter:

- The field names appear as strings, `"id"` and `"sku"`: the derive turned each field's name into the column it reads, the job you did by hand with `row.try_get("sku")`. Chapter 1 writes that by hand once more, and Chapters 2 and 3 make a macro write it.
- Every path is absolute: `::core::result::Result`, never `Result`. The `impl` lands in your module, where `Result` may mean something else. Chapter 3 calls this hygiene.
- There are two steps. `resolve` finds and checks each field's column once per statement, and `from_row_at` reads each row by position. Chapter 5 measures why.

Summary

- An ORM maps rows to structs, writes the everyday SQL, and follows relations. shelf does all three, and leaves out migrations, compile-time query checking, lazy loading, other databases and hooks.
- Libraries differ in who writes the SQL and when a mapping mistake is caught. shelf keeps the SQL visible and checks the mapping at run time, once per statement, before the first row.
- shelf is three crates: `shelf` (what you use), `shelf-macros` (a thin proc-macro shell) and `shelf-codegen` (the code generation, as a plain library that tests and the book call).
- Each mechanism appears first as a stage program, then as excerpts of the finished crate; behaviour is shown through the real derives, and expansions and compiler errors are captured, never typed.
- The book's examples run in `orm/playground`; `book.py --book orm extract` writes them and `check` runs them. Keep your own programs in `orm/playground/src/bin/`.

Exercises

1. **A missing column.** Add a field `weight_grams: i32` to the first program's `Product`, which the table does not have, and run it. What does the error name, and does the query run at all? *Hint:* it names the struct, the field and the column. Try it with `WHERE false` added to the query, which returns no rows.
2. **A renamed column.** Rename the table's `sku` column to `sku_code`, and keep the program working without changing the struct. *Hint:* the derive reads the column names the *query* produces, and `AS` renames a column in the result. (Chapter 4's `#[shelf(rename = "…")]` is the other way.)
3. **A keyword as a field name.** Add a field `r#type: String` to the struct in the second program and print the expansion again. Which column does it read? *Hint:* `type` is a keyword, so the field needs the raw-identifier prefix `r#`; the column does not.

Solutions

1. The error names all three: the struct, the field, and the column it looked for. It is reported before any row is read, because `fetch_all` checks the struct against the statement's column list when it prepares the statement, so it fails the same way when the query returns no rows. Chapter 5 shows how.

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
#[allow(dead_code)] // the program only shows the error
struct Product {
    id: i64,
    sku: String,
    weight_grams: i32,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
```

```
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    for sql in [
        "SELECT id, sku FROM products",
        "SELECT id, sku FROM products WHERE false",
    ] {
        let error = shelf::fetch_all::<Product>(&client, sql, &[])
            .await
            .unwrap_err();
        println!("{error}");
        assert_eq!(
            error.to_string(),
            r#"Product.weight_grams: the result has no column "weight_grams""#
        );
    }
    Ok(())
}
```

```
Product.weight_grams: the result has no column "weight_grams"
Product.weight_grams: the result has no column "weight_grams"
```

2. Rename the column back in the query, `SELECT id, sku_code AS sku, …`. The derive never sees the table, only the columns of the result:

```
use shelf::FromRow;

#[derive(Debug, FromRow)]
struct Product {
    id: i64,
    sku: String,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint PRIMARY KEY, sku_code text NOT NULL);
             INSERT INTO products VALUES (1, 'MUG-01');",
        )
        .await?;

    let products: Vec<Product> = shelf::fetch_all(
        &client,
        "SELECT id, sku_code AS sku FROM products ORDER BY id",
        &[],
    )
    .await?;
    assert_eq!(products[0].id, 1);
    assert_eq!(products[0].sku, "MUG-01");
    Ok(())
}
```

3. It reads the column `type`. The `r#` prefix is how Rust spells a keyword used as an identifier, and the derive strips it before it uses the name as a column (Chapter 3, §3.5):

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
```

```
            r#type: String,
        }
    };
    let code = shelf_playground::pretty(shelf_codegen::from_row::expand(&input).unwrap());
    assert!(code.contains(r#"column::<String>(columns, "Product", "type", "type")?"#));
    assert!(code.contains("r#type: ::shelf::__private::get::<String>"));
}
```

CHAPTER 1

Mapping rows by hand

You have written this function before. Every struct that stockroom reads from the database has a `TryFrom<&Row>` that names each field and reads each column: `id: row.try_get("id")?`, `code: row.try_get("code")?`, once per field, for every struct (main book §3.5). The SQL above it names the same columns again. Rename a column and you change three places, and nothing checks any of them against the others until a row arrives.

An ORM's first job is to write that function for you. Before writing the macro that does it, this chapter writes it by hand once more, carefully: as a trait of shelf's own, with the three ways it can fail, nested structs, and helpers that run a query and map its rows. It also measures what reading a column by name actually costs, because that number decides the shape of the derive in Part I.

In this chapter you will learn:

- why shelf has a `FromRow` trait of its own instead of using `TryFrom<&Row>`
- what `Row::try_get("sku")` does in tokio-postgres 0.7.18, and what it costs per row
- the three ways a mapping fails, and what each error says and does not say
- how to read a nested struct from the same row
- `fetch_all`, `fetch_one` and `fetch_opt`: a query and its mapping in one call, on a client or a transaction
- which parts of the hand-written code a macro should take over

1.1 A trait of shelf's own

The mapping needs a name. stockroom uses the standard library's `TryFrom<&Row>`; shelf defines a trait instead, with one method for now:

```
use tokio_postgres::Row;

/// A value built from one result row.
trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug, PartialEq)]
struct Product {
    id: i64,
    sku: String,
    name: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
```

```
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL UNIQUE,
                 name text NOT NULL,
                 price_cents bigint NOT NULL,
                 tags text[]
             );
             INSERT INTO products (sku, name, price_cents, tags)
             VALUES ('MUG-01', 'Mug', 1299, '{kitchen,sale}'),
                    ('TEE-02', 'T-shirt', 2499, NULL);",
        )
        .await?;

    let rows = client
        .query(
            "SELECT id, sku, name, price_cents, tags FROM products ORDER BY id",
            &[],
        )
        .await?;
    let products = rows
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products.len(), 2);
    assert_eq!(
        products[0].tags,
        Some(vec!["kitchen".into(), "sale".into()])
    );
    assert_eq!(
        products[1],
        Product {
            id: products[1].id,
            sku: "TEE-02".into(),
            name: "T-shirt".into(),
            price_cents: 2499,
            tags: None,
        }
    );
    Ok(())
}
```

The body is the same as stockroom's `try_from`. What a trait of shelf's own buys is room to grow, and three things `TryFrom<&Row>` cannot give:

- **Associated items.** Chapter 5 adds an associated type and a second method to `FromRow`, so that a statement's columns are found once instead of on every row. `TryFrom` is the standard library's trait, and nobody can add to it.
- **A short bound.** A generic helper over `TryFrom<&Row>` must say "for a row borrowed for any lifetime", as stockroom's `from_rows` does: `for<'r> T: TryFrom<&'r Row, Error = tokio_postgres::Error>`. Over `FromRow` it says `T: FromRow`.
- **Impls for types that are not yours.** Reading `SELECT sku, price_cents` into a tuple, `(String, i64)`, needs an impl for tuples. Rust's *orphan rule* allows an impl only if the trait or the type belongs to your

crate, and for `TryFrom<&Row> for (A, B)`, neither does. This fails to compile in any crate but tokio-postgres itself:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

impl<A: FromSqlOwned, B: FromSqlOwned> TryFrom<&Row> for (A, B) {
    type Error = tokio_postgres::Error;
    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}
```

```
error[E0117]: only traits defined in the current crate can be implemented for arbitrary
types
 --> src/bin/orphan.rs:4:1
  |
4 | impl<A: FromSqlOwned, B: FromSqlOwned> TryFrom<&Row> for (A, B) {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                                         |                 |
  |                                         |                 this is not defined in the
current crate because tuples are always foreign
  |                                         `tokio_postgres::Row` is not defined in the
current crate
  |
  = note: impl doesn't have any local type before any uncovered type parameters
  = note: for more information see https://doc.rust-lang.org/reference/items/implementations
.html#orphan-rules
  = note: define and implement a trait or new type instead
```

With `FromRow` defined in shelf, `impl<A, B> FromRow for (A, B)` is allowed, because the trait is local. Chapter 2 writes those impls for twelve tuple sizes at once.

> **Rust refresher:** `FromSqlOwned` is tokio-postgres's name for "a type that can be read from a column without borrowing from the row", a shorthand for `for<'a> FromSql<'a>` (main book §3.4). `String` is one; `&str` is not, because it points into the row's buffer. A mapping trait whose method returns `Self` with no lifetime, like `FromRow`, can only build owned values, and shelf keeps it that way: Chapter 4 refuses a struct with a lifetime parameter, with an error that says why.

1.2 By name or by position

`row.try_get("sku")` looks like a map lookup, but a `Row` has no map. In 0.7.18 it walks the row's columns and compares each name with `"sku"`. If none matches exactly, it walks them again, ignoring ASCII case (`row.rs`):

```
// tokio-postgres 0.7.18: src/row.rs, abridged
impl RowIndex for str {
    fn __idx<T>(&self, columns: &[T]) -> Option<usize>
    where
        T: AsName,
    {
        if let Some(idx) = columns.iter().position(|d| d.as_name() == self) {
            return Some(idx);
        };

        // FIXME ASCII-only case insensitivity isn't really the right thing to
        // do. Postgres itself uses a dubious wrapper around tolower and JDBC
        // uses the US locale.
        columns
            .iter()
            .position(|d| d.as_name().eq_ignore_ascii_case(self))
```

```
    }
}
```

Then, whether you asked by name or by position, `try_get` checks that the column's type can be read into the Rust type you asked for, and only then decodes the bytes:

```
// tokio-postgres 0.7.18: src/row.rs, abridged
fn get_inner<'a, I, T>(&'a self, idx: &I) -> Result<T, Error>
where
    I: RowIndex + fmt::Display,
    T: FromSql<'a>,
{
    let idx = match idx.__idx(self.columns()) {
        Some(idx) => idx,
        None => return Err(Error::column(idx.to_string())),
    };

    let ty = self.columns()[idx].type_();
    if !T::accepts(ty) {
        return Err(Error::from_sql(
            Box::new(WrongType::new::<T>(ty.clone())),
            idx,
        ));
    }

    FromSql::from_sql_nullable(ty, self.col_buffer(idx)).map_err(|e| Error::from_sql(e,
idx))
}
```

So a struct of twelve fields read by name does twelve searches per row. A search that finds its column at position p compares $p + 1$ names, so the twelve together compare $1 + 2 + \ldots + 12 = 78$ names on every row, whatever order the columns come in. A missing column costs two full walks, and the second walk means `"SKU"` finds a column named `sku`.

What does that cost? This program reads 100,000 rows of twelve columns (four `bigint`, four `text`, two `integer`, a `boolean` and a `double precision`, two of them sometimes NULL) into the same struct twice: by name, and by position. It fetches the rows once and times only the mapping, best of three rounds:

```
use std::time::{Duration, Instant};
use tokio_postgres::Row;

#[allow(dead_code)] // only the mapping is measured
struct Wide {
    c01: i64,
    c02: String,
    c03: i32,
    c04: Option<String>,
    c05: i64,
    c06: bool,
    c07: String,
    c08: i64,
    c09: Option<i32>,
    c10: String,
    c11: i64,
    c12: f64,
}

fn by_name(r: &Row) -> Result<Wide, tokio_postgres::Error> {
    Ok(Wide {
        c01: r.try_get("c01")?,
        c02: r.try_get("c02")?,
        c03: r.try_get("c03")?,
        c04: r.try_get("c04")?,
        c05: r.try_get("c05")?,
        c06: r.try_get("c06")?,
```

```
        c07: r.try_get("c07")?,
        c08: r.try_get("c08")?,
        c09: r.try_get("c09")?,
        c10: r.try_get("c10")?,
        c11: r.try_get("c11")?,
        c12: r.try_get("c12")?,
    })
}

// The SELECT lists the columns in reverse, so c01 is at position 11.
fn by_position(r: &Row) -> Result<Wide, tokio_postgres::Error> {
    Ok(Wide {
        c01: r.try_get(11)?,
        c02: r.try_get(10)?,
        c03: r.try_get(9)?,
        c04: r.try_get(8)?,
        c05: r.try_get(7)?,
        c06: r.try_get(6)?,
        c07: r.try_get(5)?,
        c08: r.try_get(4)?,
        c09: r.try_get(3)?,
        c10: r.try_get(2)?,
        c11: r.try_get(1)?,
        c12: r.try_get(0)?,
    })
}

fn best_of_three(
    rows: &[Row],
    map: fn(&Row) -> Result<Wide, tokio_postgres::Error>,
) -> Result<Duration, tokio_postgres::Error> {
    let mut best = Duration::MAX;
    for _ in 0..3 {
        let start = Instant::now();
        let mapped: Vec<Wide> = rows.iter().map(map).collect::<Result<_, _>>()?;
        best = best.min(start.elapsed());
        assert_eq!(mapped.len(), rows.len());
    }
    Ok(best)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let rows = client
        .query(
            "SELECT i::float8 AS c12, i AS c11, 'name ' || i AS c10,
                    CASE WHEN i % 3 = 0 THEN NULL ELSE i::int END AS c09,
                    i AS c08, 'tag ' || i AS c07, i % 2 = 0 AS c06, i AS c05,
                    CASE WHEN i % 5 = 0 THEN NULL ELSE 'note' END AS c04,
                    i::int AS c03, 'sku-' || i AS c02, i AS c01
             FROM generate_series(1, 100000::bigint) AS i",
            &[],
        )
        .await?;

    let name = best_of_three(&rows, by_name)?;
    let position = best_of_three(&rows, by_position)?;
    println!(
        "{} rows: by name {name:.1?}, by position {position:.1?} ({:.1}×)",
        rows.len(),
        name.as_secs_f64() / position.as_secs_f64()
    );
    assert!(position < name);
    Ok(())
}
```

```
100000 rows: by name 197.8ms, by position 123.2ms (1.6×)
```

That run was a debug build, which is how `book.py check` builds the examples (rustc 1.99.1, Apple Silicon, PostgreSQL 18.6 on localhost); three runs gave 1.5× to 1.7×. In a release build the same comparison is 38 ms by name against 12.5 ms by position, about **3×** (orm/PLAN.md §12, findings 8 and 13). The lookups, not the decoding, are most of the work, and they repeat on every row although the answer cannot change: every row of a statement has the same columns, in the same order.

Reading by position is not the fix, though. Positions break silently when someone adds a column to the `SELECT`, which is why the main book recommends names. The fix is to look each name up **once per statement** and then read every row by position. That needs a trait with somewhere to keep the positions, which is the first thing `FromRow` will grow (Chapter 5).

> **Pitfall:** the second, case-insensitive walk means a misspelt name can still find a column: `row.try_get("SKU")` reads a column named `sku`. PostgreSQL itself folds unquoted identifiers to lower case, and a quoted `"SKU"` is a different column from `sku`, so a mapping that matches names ignoring case can read the wrong one when both exist. shelf matches names exactly.

1.3 Three ways to fail

A row can fail to fit a struct in three ways: a column the struct needs is missing, a column has a type the field cannot hold, or a column is NULL and the field is not an `Option`. This program provokes each with a query that gets it wrong, and prints the error's `Display` and its `Debug`:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug)]
#[allow(dead_code)] // the program only shows the errors
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let mistakes = [
        // No `name` column.
        (
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents",
            "invalid column `name`",
        ),
        // `1299` is an int4, and the field is an i64.
        (
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            "error deserializing column 2",
        ),
```

```
        // A NULL name, and the field is a String.
        (
            "SELECT 'MUG-01' AS sku, NULL::text AS name, 1299::int8 AS price_cents",
            "error deserializing column 1",
        ),
    ];
    for (sql, expected) in mistakes {
        let row = client.query_one(sql, &[]).await?;
        let error = Product::from_row(&row).unwrap_err();
        println!("{error}\n    {error:?}");
        assert_eq!(error.to_string(), expected);
    }
    Ok(())
}
```

```
invalid column `name`
    Error { kind: Column("name"), cause: None }
error deserializing column 2
    Error { kind: FromSql(2), cause: Some(WrongType { postgres: Int4, rust: "i64" }) }
error deserializing column 1
    Error { kind: FromSql(1), cause: Some(WasNull) }
```

tokio-postgres reports what it knows: a column, by name or position, and, one level down in `source()`, the cause. What it cannot report is what you were doing: which struct, and which field. In a service with forty row types, "error deserializing column 2" in a log is the start of a search. Your `from_row` knows the struct and the field, but adding them by hand means a `map_err` on every line (Exercise 1).

This is what shelf reports for the same three queries, with the same struct and `#[derive(shelf::FromRow)]`:

```
#[derive(Debug, shelf::FromRow)]
#[allow(dead_code)] // the program only shows the errors
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    let mistakes = [
        (
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents",
            r#"Product.name: the result has no column "name""#,
        ),
        (
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            r#"Product.price_cents: column "price_cents" has type int4, which cannot be read
into i64"#,
        ),
        (
            "SELECT 'MUG-01' AS sku, NULL::text AS name, 1299::int8 AS price_cents",
            r#"Product.name: column "name" is NULL; read it into an Option"#,
        ),
    ];
    for (sql, expected) in mistakes {
        let error = shelf::fetch_one::<Product>(&client, sql, &[])
            .await
            .unwrap_err();
        println!("{error}");
        assert_eq!(error.to_string(), expected);
    }
    Ok(())
}
```

```
Product.name: the result has no column "name"
Product.price_cents: column "price_cents" has type int4, which cannot be read into i64
Product.name: column "name" is NULL; read it into an Option
```

Each message names the struct, the field, the column and, for a type mismatch, both types. The first two are not even row errors in shelf: they are found when the statement is prepared, before the query runs (Chapter 5). The third can only be found in a row, because a column's type says nothing about whether it holds NULL.

> **SQL lens:** why is `1299` an `int4` and `'Mug'` a `text`? An integer literal that fits in 32 bits is typed `integer`; a quoted literal in a `SELECT` list, with nothing else to go on, is resolved as `text`. The server decides each column's type when it prepares the statement, and the row description carries it to the client. That is the type `accepts` checks: the Rust side never guesses, and never widens an `int4` into an `i64` for you.

1.4 Nested structs

Some structs are better split. A price is an amount and a currency, and it has behaviour of its own (formatting, arithmetic), so it deserves a type. But in the table it is two columns of the `products` row, not a table of its own. Reading it by hand means calling the inner struct's `from_row` from the outer one, on the same row:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

#[derive(Debug, PartialEq)]
struct Price {
    cents: i64,
    currency: String,
}

impl FromRow for Price {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Price {
            cents: row.try_get("price_cents")?,
            currency: row.try_get("currency")?,
        })
    }
}

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price: Price,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price: Price::from_row(row)?, // the same row, read by the inner struct
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
```

```
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY,
                 price_cents bigint NOT NULL,
                 currency text NOT NULL
             );
             INSERT INTO products VALUES ('MUG-01', 1299, 'EUR');",
        )
        .await?;

    let row = client
        .query_one("SELECT sku, price_cents, currency FROM products", &[])
        .await?;
    let product = Product::from_row(&row)?;
    assert_eq!(
        product,
        Product {
            sku: "MUG-01".into(),
            price: Price {
                cents: 1299,
                currency: "EUR".into()
            },
        }
    );
    Ok(())
}
```

It works, and it shows the catch: the inner struct and the outer one take their names from the same row. `Price` reads `price_cents` and `currency`, so it fits this query and no other. A product with two prices, a list price and a sale price, cannot use `Price` twice: both would read the same two columns. You would alias them in SQL and give `Price` a second impl, or pass a prefix in at run time and build each name with `format!`, one allocation per column per row.

Chapter 4's `#[shelf(flatten)]` does the first half with one attribute: it reads a nested `FromRow` struct from the same row. The column names stay the inner struct's business, which you set with its own `#[shelf(rename = "…")]` attributes.

1.5 Fetch helpers

Every read so far has had the same three steps: run the query, map each row, collect the results into a `Result<Vec<_>, _>`. A helper does all three, and two siblings cover the other shapes a result can have, exactly one row and at most one row:

```
use tokio_postgres::types::ToSql;
use tokio_postgres::{GenericClient, Row};

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// Every row of `sql`, read into `T`.
async fn fetch_all<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Vec<T>, tokio_postgres::Error> {
    let rows = db.query(sql, params).await?;
    rows.iter().map(T::from_row).collect()
}

/// The one row of `sql`; an error if there are none, or several.
async fn fetch_one<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
```

```
    params: &[&(dyn ToSql + Sync)],
) -> Result<T, tokio_postgres::Error> {
    let row = db.query_one(sql, params).await?;
    T::from_row(&row)
}

/// The row of `sql`, if there is one; an error if there are several.
async fn fetch_opt<T: FromRow>(
    db: &impl GenericClient,
    sql: &str,
    params: &[&(dyn ToSql + Sync)],
) -> Result<Option<T>, tokio_postgres::Error> {
    let row = db.query_opt(sql, params).await?;
    row.as_ref().map(T::from_row).transpose()
}

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    // On the client.
    let cheap: Vec<Product> = fetch_all(
        &client,
        "SELECT sku, price_cents FROM products WHERE price_cents < $1 ORDER BY sku",
        &[&2000_i64],
    )
    .await?;
    assert_eq!(cheap.len(), 1);
    let one: Product = fetch_one(
        &client,
        "SELECT sku, price_cents FROM products WHERE sku = $1",
        &[&"TEE-02"],
    )
    .await?;
    assert_eq!(one.price_cents, 2499);
    let none: Option<Product> = fetch_opt(
        &client,
        "SELECT sku, price_cents FROM products WHERE sku = $1",
        &[&"VASE-03"],
    )
    .await?;
    assert_eq!(none, None);

    // The same helpers inside a transaction: it sees its own insert, and nothing outside
does.
    let tx = client.transaction().await?;
    tx.execute("INSERT INTO products VALUES ('VASE-03', 3999)", &[])
        .await?;
```

```
    let inside: Vec<Product> = fetch_all(&tx, "SELECT sku, price_cents FROM products",
&[]).await?;
    assert_eq!(inside.len(), 3);
    tx.rollback().await?;
    let after: Vec<Product> =
        fetch_all(&client, "SELECT sku, price_cents FROM products", &[]).await?;
    assert_eq!(after.len(), 2);
    Ok(())
}
```

The helpers take tokio-postgres's `GenericClient`, which `Client` and `Transaction` both implement (main book Chapter 8), so one repository function can run inside or outside a transaction. A deadpool connection derefs, in two steps, to a `Client`, so `&**object` passes too (main book Chapter 13). Chapter 7 replaces `GenericClient` with a trait of shelf's own, because tokio-postgres's is sealed and has no statement cache.

shelf's own `fetch_all` takes the same arguments, and does one thing more: it prepares the statement first and checks the struct against the statement's columns before it runs the query. That is why the first two mistakes in §1.3 were found before any row was read. It costs no extra round trip, because `query` with a SQL string prepares the statement anyway (main book §4.1). Chapter 5 shows how.

1.6 Counting the boilerplate

Look at what you typed for a struct of five fields, and where each name went:

Where the name is written	Written by	Checked against the table
The struct's field (`price_cents: i64`)	you	never: the compiler knows nothing of the table
The `SELECT` list (`SELECT …, price_cents, …`)	you	by the server, when the statement is prepared
The `try_get` call (`row.try_get("price_cents")`)	you	when a row is read, against the `SELECT` list

Each field is named three times, and the second and third are strings that nothing compares with the first. The struct's name is in none of them, which is why §1.3's errors could not name it. And every mapping has the same shape: one line per field, the field's name turned into a string. That is the part a macro can take over:

- **The `from_row` body**: every line follows from the field's name and type.
- **The error context**: the struct's and the field's names are known when the code is generated, so every error can carry them at no cost.
- **The column positions**: the code can find them once per statement and keep them (Chapter 5).

What a macro cannot take over yet is the SQL. A derive that only sees the struct does not know the table; Chapter 6's `#[derive(Table)]` is told it, and writes the column list too.

Side by side — *a row into a struct*

The task: read products, with a nullable array column, into a struct.

```
// sqlx (illustration)
#[derive(sqlx::FromRow)]
struct Product {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

let products: Vec<Product> =
    sqlx::query_as("SELECT sku, price_cents, tags FROM products ORDER BY sku")
        .fetch_all(&pool)
        .await?;
```

By hand and with shelf, in one program that checks that both read the same values:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
        })
    }
}

// shelf: the same fields, and one line instead of the impl.
#[derive(Debug, PartialEq, shelf::FromRow)]
struct ShelfProduct {
    sku: String,
    price_cents: i64,
    tags: Option<Vec<String>>,
}

const SQL: &str = "SELECT sku, price_cents, tags FROM products ORDER BY sku";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint, tags
text[]);
             INSERT INTO products VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499,
NULL);",
        )
        .await?;

    let by_hand = client
        .query(SQL, &[])
        .await?
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;
    let with_shelf: Vec<ShelfProduct> = shelf::fetch_all(&client, SQL, &[]).await?;

    assert_eq!(by_hand.len(), with_shelf.len());
    for (a, b) in by_hand.iter().zip(&with_shelf) {
        assert_eq!(
            (&a.sku, a.price_cents, &a.tags),
            (&b.sku, b.price_cents, &b.tags)
        );
    }
    assert_eq!(with_shelf[1].tags, None);
    Ok(())
}
```

	by hand	shelf	sqlx
Same	Fields are matched to columns by name; NULL is `Option`; types are checked at run time	The same	The same
Different	One line per field, in an impl you write and can change. Errors name a column, not the struct or field. Every name is looked up on every row	One line, `#[derive(FromRow)]`. The struct is checked against the statement once, before the first row; errors name the struct and field. Owned values only	The derive is generic over sqlx's databases (`R: Row`). Each name is looked up on every row, in a hash map the statement carries. `query_as!` can check the query at compile time

In stockroom

stockroom is the hand-written version at full size. Its `Store` has five fields, and the file that reads it names each of them four times: in the struct, in two `SELECT` lists (one in `list`, one in `create`'s `RETURNING`), and in `try_from`:

```
// stockroom/src/repo/stores.rs (excerpt)
pub struct Store {
    pub id: Uuid,
    pub code: String,
    pub name: String,
    pub address: Address,
    pub created_at: Timestamp,
}

impl TryFrom<&Row> for Store {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Store {
            id: row.try_get("id")?,
            code: row.try_get("code")?,
            name: row.try_get("name")?,
            address: row.try_get("address")?, // the composite, through the derive
            created_at: row.try_get("created_at")?,
        })
    }
}

// …
pub async fn list(db: &impl GenericClient) -> Result<Vec<Store>, AppError> {
    let stmt = db
        .prepare_cached("SELECT id, code, name, address, created_at FROM stores ORDER BY
code")
        .await?;
    from_rows(&db.query(&stmt, &[]).await?)
}
```

stockroom/src/repo/stores.rs

`address` is a PostgreSQL composite type, which `Address` maps through postgres-types' derive (main book Chapter 5). That is one column holding a whole struct, read by `FromSql`, and the opposite of §1.4's nesting, where one struct spans several columns. shelf keeps both: `FromSql` types such as `Address` are ordinary fields, and `flatten` handles the other kind.

Chapter 14 rewrites part of stockroom on shelf and counts the lines this saves.

When not to use a mapping trait

- **One value.** `query_one_scalar::<i64>("SELECT count(*) …")` reads it without a struct (main book §2.5).
- **A one-off query read in place.** In a migration check or a test, `row.get("total")` is clearer than a struct that exists for one line.
- **Hot loops over borrowed data.** Mapping into owned structs allocates a `String` per text column per row. When you only need to look at the values, borrow them from the rows with `row.get::<_, &str>(…)` and keep the rows alive (main book §3.4).

Summary

- shelf maps rows with a trait of its own, `FromRow`, rather than `TryFrom<&Row>`: it can grow associated items, gives generic code a short bound, and can be implemented for tuples, which the orphan rule forbids for `TryFrom<&Row>`.
- In 0.7.18, `try_get("name")` walks the columns comparing names (and walks them again, ignoring ASCII case, if none matches), then checks `accepts`, then decodes. Twelve fields by name compare 78 names per row in any order.
- By name costs about 3× by position in a release build, 1.6× in a debug build, on 100,000 rows of 12 columns. Positions break when the `SELECT` changes; the answer is to find the positions once per statement.
- A mapping fails on a missing column, a wrong type, or a NULL in a non-`Option` field. tokio-postgres's errors name the column but not the struct or field; shelf's name all of them.
- A nested struct can be read from the same row by calling its `from_row`, but its column names are then fixed.
- `fetch_all`, `fetch_one` and `fetch_opt` run a query and map it, on anything that implements `GenericClient`.
- Each field is named three times by hand, twice as strings nothing checks. A macro can write the mapping and its error context; the SQL needs a derive that knows the table.

Exercises

1. **Errors with context.** Change the trait so that `from_row` returns an error naming the struct and the field, as well as tokio-postgres's error and its cause, and implement it for a `Product` with three fields. *Hint:* a small helper, `get(row, "Product", "price_cents")`, keeps each line short. Count how often the struct's name now appears in the impl.
2. **Does the order matter?** Predict how §1.2's timing changes if the `SELECT` lists the columns in the struct's order instead of in reverse. Then measure it. *Hint:* in each order, at which position does each field's search stop?
3. **A tuple.** Implement `FromRow` for `(A, B)`, reading by position, and read `SELECT sku, price_cents FROM products` into a `Vec<(String, i64)>`. *Hint:* each element needs `FromSqlOwned`; §1.1 showed why this impl needs your own trait.

Solutions

1. The error type carries the two names as `&'static str`, so building it costs nothing:

```
use std::fmt;
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

#[derive(Debug)]
struct MappingError {
    model: &'static str,
    field: &'static str,
    source: tokio_postgres::Error,
}
```

```
impl fmt::Display for MappingError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}.{}: {}", self.model, self.field, self.source)?;
        // The useful part of a tokio-postgres error is often its cause.
        if let Some(cause) = std::error::Error::source(&self.source) {
            write!(f, ": {cause}")?;
        }
        Ok(())
    }
}

impl std::error::Error for MappingError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        Some(&self.source)
    }
}

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, MappingError>;
}

fn get<T: FromSqlOwned>(
    row: &Row,
    model: &'static str,
    field: &'static str,
) -> Result<T, MappingError> {
    row.try_get(field).map_err(|source| MappingError {
        model,
        field,
        source,
    })
}

#[derive(Debug)]
#[allow(dead_code)] // the program only shows the error
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, MappingError> {
        Ok(Product {
            sku: get(row, "Product", "sku")?,
            name: get(row, "Product", "name")?,
            price_cents: get(row, "Product", "price_cents")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'MUG-01' AS sku, 'Mug' AS name, 1299 AS price_cents",
            &[],
        )
        .await?;
    let error = Product::from_row(&row).unwrap_err();
    println!("{error}");
    assert_eq!(
        error.to_string(),
        "Product.price_cents: error deserializing column 2: \
         cannot convert between the Rust type `i64` and the Postgres type `int4`"
    );
```

```
    Ok(())
}
```

```
Product.price_cents: error deserializing column 2: cannot convert between the Rust type
`i64` and the Postgres type `int4`
```

The struct's name is now written once per field, and each field's name is written twice: as the field and as the string. Both are exactly what a macro knows when it generates the impl.

2. Nothing measurable changes. A search that finds its column at position p compares $p + 1$ names, and every column is looked up once per row, so the total is $1 + 2 + \dots + 12 = 78$ comparisons per row for *any* order of the `SELECT` list. Measured in a debug build, three rounds each: 202.7–206.9 ms in reverse order and 205.2–206.6 ms in the struct's order, against 124–130 ms by position in both. What does grow the cost is the number of columns: twice as many columns means four times as many comparisons per row.

3. The impl is allowed because `FromRow` is defined in this program:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

impl<A: FromSqlOwned, B: FromSqlOwned> FromRow for (A, B) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    let pairs = client
        .query("SELECT sku, price_cents FROM products ORDER BY sku", &[])
        .await?
        .iter()
        .map(<(String, i64)>::from_row)
        .collect::<Result<Vec<_>, _>>()?;
    assert_eq!(pairs, [("MUG-01".into(), 1299), ("TEE-02".into(), 2499)]);
    Ok(())
}
```

Writing the impl for `(A, B, C)`, then `(A, B, C, D)`, and so on to twelve, is the kind of repetition the next chapter's macros remove.

CHAPTER 2

Declarative macros

Chapter 1's `FromRow` impl is mechanical: the same line for every field, with the field's name in two places, once as a field and once as a string. Mechanical code is what macros are for, and Rust has two kinds. This chapter uses the simpler one, `macro_rules!`, which matches the tokens it is given against patterns and pastes the pieces it matched into a template.

It goes surprisingly far. A `model!` macro can declare a struct and write its `FromRow` impl in one go, and one invocation can write `FromRow` for tuples of every size from one to twelve, which shelf keeps to this day. Then it stops: as soon as a field needs an attribute (read it from another column, skip it, decode it as JSON), the patterns turn into a puzzle, and the errors point at the macro's rules instead of at your mistake. That is the case for the next chapter's procedural macros.

In this chapter you will learn:

- how `macro_rules!` matches tokens: rules, fragment specifiers and repetitions
- how to write a `model!` macro that declares a struct and its `FromRow` impl together
- how one macro invocation writes `FromRow` for tuples of one to twelve elements, and why the positions are matched as `tt`
- what hygiene protects in a macro, what it does not, and what `$crate` is for
- where declarative macros stop, and why shelf's derives are procedural

2.1 Matching tokens

A `macro_rules!` macro is a list of rules. Each rule is a *matcher*, a pattern over tokens, and a *transcriber*, the code to produce when the pattern matches. The compiler tries the rules in order and expands the first one that matches. Here is a macro with one rule, which turns a list of column names into the text of a `SELECT` list, at compile time:

```
/// Joins column names into a `SELECT` list: `columns!(id, sku)` is `"id, sku"`.
macro_rules! columns {
    ($first:ident $(, $rest:ident)* $(,)?) => {
        concat!(stringify!($first) $(, ", ", stringify!($rest))*)
    };
}

const PRODUCT_COLUMNS: &str = columns!(id, sku, price_cents);

fn main() {
    assert_eq!(PRODUCT_COLUMNS, "id, sku, price_cents");
    assert_eq!(columns!(id), "id");
    assert_eq!(columns!(id, sku,), "id, sku"); // a trailing comma is allowed
}
```

The matcher reads like this:

- `$first:ident` matches one identifier and calls it `$first`. The part after the colon is a *fragment specifier*: it says what kind of syntax the metavariable accepts.

- `$(, $rest:ident)*` is a *repetition*: a comma followed by an identifier, zero or more times (`*`). Each identifier it matches is one `$rest`.
- `$(,)?` matches an optional trailing comma (`?` is zero or one).

The transcriber uses the same notation. `$(, ", ", stringify!($rest))*` repeats its contents once for every `$rest` that was matched. `stringify!` turns tokens into a string literal, and `concat!` joins literals into one: both run during expansion, so `PRODUCT_COLUMNS` is a plain `&'static str` literal and the program does no work to build it. The first element is matched on its own because the separator goes *between* elements: there is no way to say "a comma before every element but the first" in one repetition.

The fragment specifiers this book uses:

Specifier	Matches	Example
`ident`	an identifier	`price_cents`, `Product`
`ty`	a type	`Option<Vec<String>>`
`expr`	an expression	`row.len() + 1`
`literal`	a literal	`"sku"`, `0`
`vis`	a visibility, possibly empty	`pub`, `pub(crate)`, or nothing
`meta`	the inside of an attribute	`derive(Debug, PartialEq)`
`tt`	one token tree: a single token, or a group in `()`, `[]` or `{}`	`0`, `(a, b)`

The Rust Reference's Macros By Example lists the others (`path`, `pat`, `block`, `item`, `lifetime`, `stmt`) and the rules about which tokens may follow each one.

> **Rust refresher:** macros expand before the compiler checks types, or even knows what most names refer to. A macro sees `Option<Vec<String>>` as a handful of tokens; the `ty` specifier checks that they *form* a type, but the macro cannot ask whether that type is an `Option`, or whether it implements `FromSql`. This is true of every macro in Rust, procedural ones included. shelf's derive cannot know that your `type Tags = Option<Vec<String>>` is an `Option` either. It does not need to: it generates code that only compiles if the types fit, and lets the compiler check.

2.2 A `model!` macro

A struct definition is tokens too. A macro that accepts one can paste it back unchanged and add the `FromRow` impl after it, writing one `try_get` per field with `stringify!` for the column name:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// Declares a struct, and implements `FromRow` for it: each field is read from the column
of
/// its name.
macro_rules! model {
    (
        $(#[$meta:meta])*
        $vis:vis struct $name:ident {
            $($field_vis:vis $field:ident : $ty:ty),* $(,)?
        }
    ) => {
        $(#[$meta])*
        $vis struct $name {
```

```
            $($field_vis $field: $ty),*
        }

        impl FromRow for $name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self {
                    $($field: row.try_get(stringify!($field))?),*
                })
            }
        }
    };
}

model! {
    #[derive(Debug, PartialEq)]
    pub struct Product {
        pub id: i64,
        pub sku: String,
        pub price_cents: i64,
        pub tags: Option<Vec<String>>,
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                 sku text NOT NULL,
                 price_cents bigint NOT NULL,
                 tags text[]
             );
             INSERT INTO products (sku, price_cents, tags)
             VALUES ('MUG-01', 1299, '{kitchen}'), ('TEE-02', 2499, NULL);",
        )
        .await?;

    let products = client
        .query(
            "SELECT id, sku, price_cents, tags FROM products ORDER BY id",
            &[],
        )
        .await?
        .iter()
        .map(Product::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products[0].sku, "MUG-01");
    assert_eq!(products[0].tags, Some(vec!["kitchen".to_string()]));
    assert_eq!(products[1].price_cents, 2499);
    assert_eq!(products[1].tags, None);
    Ok(())
}
```

Three pieces of the matcher do the work:

- `$(#[$meta:meta])*` accepts any number of attributes on the struct and pastes them back, so `#[derive(Debug, PartialEq)]` still applies. Without it, `model!` would refuse every attribute.
- `$vis:vis` and `$field_vis:vis` accept `pub`, `pub(crate)` or nothing at all, so the macro does not decide the struct's visibility for you.
- The field repetition, `$($field_vis:vis $field:ident : $ty:ty),*`, gives the transcriber two lists that line up, `$field` and `$ty`, and it uses them twice: once to declare the fields and once, `$field` alone, to read them. The field names are written once, by you, and the column names come from them.

The struct's fields are now named once instead of twice. The cost is that the struct lives inside a macro call, which is no longer ordinary Rust to your tools. rustfmt leaves the inside of `model! { … }` alone, so its formatting is yours to keep, and a mistake inside it is reported against the macro's rules, as §2.5 shows. That cost is the reason shelf's mapping is a derive: a derive leaves your struct an ordinary struct, and adds the impl next to it.

2.3 Tuples, one line per arity

A query that returns two columns does not always deserve a struct. Chapter 1 implemented `FromRow` for `(A, B)` by hand, reading by position; shelf has the same for tuples of one to twelve elements, and a single macro writes all twelve impls. Each line of the invocation is one tuple size, with each element's type parameter next to its position:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

/// `FromRow` for tuples, read by position: one impl per line of the invocation.
macro_rules! tuple_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: FromSqlOwned),+> FromRow for ($($T,)+) {
                fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                    Ok(($(row.try_get::<_, $T>($i)?,)+))
                }
            }
        )+
    };
}

tuple_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
    (A 0, B 1, C 2, D 3)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents, true AS active, 3 AS stock",
            &[],
        )
        .await?;

    let (sku,) = <(String,)>::from_row(&row)?;
    let pair = <(String, i64)>::from_row(&row)?;
    let all = <(String, i64, bool, i32)>::from_row(&row)?;
    assert_eq!(sku, "MUG-01");
    assert_eq!(pair, ("MUG-01".to_string(), 1299));
    assert_eq!(all, ("MUG-01".to_string(), 1299, true, 3));
    Ok(())
}
```

The matcher has a repetition inside a repetition. The outer one, `$( (…) )+`, matches each parenthesized line; the inner one, `$($T:ident $i:tt),+`, matches the elements of a line. In the transcriber, `$( impl … )+` repeats per line, and inside it `$($T: FromSqlOwned),+` and `($($T,)+)` repeat per element. The second one writes a

comma after *every* element, including the last, which is not just tidy: a one-element tuple is spelled `(A,)`, and `(A)` would be `A` in parentheses.

Why is the position matched as a `tt`, a single token tree, and not as a `literal`? Here either would do, because `$i` is pasted into expression positions: `row.try_get::<_, $T>($i)`. But the two are pasted differently. A `tt` goes in as the token it was. A `literal` or an `expr` goes in as one sealed piece of syntax, which some positions refuse. A tuple field is one of them:

```
macro_rules! field_tt { ($t:expr, $i:tt) => { $t.$i }; }
macro_rules! field_lit { ($t:expr, $i:literal) => { $t.$i }; }
fn main() {
    let t = (10, 20);
    println!("{}", field_tt!(t, 1));
    println!("{}", field_lit!(t, 1));
}
```

`field_tt!(t, 1)` becomes `t.1`. `field_lit!(t, 1)` does not compile:

```
error: unexpected token: `literal` metavariable
 --> literal.rs:2:56
  |
2 | macro_rules! field_lit { ($t:expr, $i:literal) => { $t.$i }; }
  |                                                        ^^
...
6 |     println!("{}", field_lit!(t, 1));
  |                    ---------------- in this macro invocation
```

So `tt` is the habit for a position: it works everywhere a number does. Exercise 3 needs that.

Writing the positions out by hand looks like something the macro should count for itself. Nightly Rust has *metavariable expressions*, whose `${index()}` gives the position inside a repetition, but in rustc 1.99.1 they are still unstable (error E0658, tracking issue #83527). A recursive macro can avoid repeating the lines instead (Exercise 1), but the flat list is the easiest to read and to check, so shelf keeps it.

2.4 Hygiene, and `$crate`

A macro's output lands in the caller's code, among the caller's names. Two questions follow: can the macro's own variables collide with the caller's, and do the names the macro writes mean what its author intended? `macro_rules!` answers them differently, and this program shows both answers:

```
mod text {
    /// Pads a cell of a plain-text table to eight characters.
    pub fn cell(value: &str) -> String {
        format!("{value:<8}|")
    }

    /// One line of a plain-text table, from any number of cells.
    #[macro_export]
    macro_rules! table_line {
        ($($value:expr),+ $(,)?) => {{
            let mut line = ::std::string::String::new();
            $( line.push_str(&$crate::text::cell($value)); )+
            line
        }};
    }
}

// A name the macro must not depend on: this crate's `String` is not std's.
#[allow(dead_code)]
struct String;

fn main() {
    let line = "MUG-01";
```

```
    let header = table_line!("sku", "price");
    let row = table_line!(line, "1299"); // the caller's `line`, not the macro's
    assert_eq!(header, "sku     |price   |");
    assert_eq!(row, "MUG-01  |1299    |");
    assert_eq!(line, "MUG-01");
}
```

Local variables are hygienic. The macro declares `let mut line`, and the caller passes in its own `line`. Inside the expansion both are in scope, with the same name, yet `$value` still means the caller's `&str` and the macro's `push_str` still targets its own `String`. The compiler remembers where each identifier was written: a variable declared by the macro is visible only to the macro's tokens, and the caller's tokens see only the caller's variables.

Paths are not. Every other name in the expansion, types, functions, traits, constants, is looked up where the macro is *called*. That is why `table_line!` writes `::std::string::String::new()`: in `main`'s module, `String` is the caller's unit struct, and `String::new()` would not compile. A leading `::` means that the first segment names a crate (`std`, `core`, `shelf`), never an item of the caller's module, so nothing the caller declares can shadow it. The Reference calls this mix *mixed-site hygiene*.

`model!` from §2.2 is not that careful. It writes `Result<Self, tokio_postgres::Error>`, and an application with the common alias `type Result<T> = std::result::Result<T, AppError>` in scope breaks it:

```
mod app {
    use super::{FromRow, Row};

    type Result<T> = std::result::Result<T, String>;

    model! {
        pub struct Product {
            pub sku: String,
        }
    }
}
```

```
error[E0107]: type alias takes 1 generic argument but 2 generic arguments were supplied
  --> src/bin/model_alias.rs:20:39
   |
20 |               fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
   |                                         ^^^^^^     ---------------------- help: remove
the unnecessary generic argument
   |                                         |
   |                                         expected 1 generic argument
...
34 | /     model! {
35 | |         pub struct Product {
36 | |             pub sku: String,
37 | |         }
38 | |     }
   | |_____- in this macro invocation
   |
note: type alias defined here, with 1 generic parameter: `T`
  --> src/bin/model_alias.rs:32:10
   |
32 |     type Result<T> = std::result::Result<T, String>;
   |          ^^^^^^ -
   = note: this error originates in the macro `model` (in Nightly builds, run with -Z macro-
backtrace for more info)
```

The fix is the same as in `table_line!`: absolute paths for everything the macro names, `::core::result::Result` and `::core::result::Result::Ok`, never `Result` and `Ok`. Notice also that `app` had to import `FromRow` and `Row` for `model!` to work at all, because the expansion names them as bare identifiers. A careful `model!` would not ask that of its caller.

`$crate` names the macro's own crate. `cell` is a function in this program, so `table_line!` could have written `crate::text::cell`. But a macro exported from a library (`#[macro_export]`) runs in other crates, where `crate` means the *caller's* crate. `$crate` expands to the crate that defined the macro, wherever it is called, and it keeps working when the caller renames the dependency in `Cargo.toml`. It is how a library's macro reaches the library's helpers.

> **Macro lens:** `$crate` exists only in `macro_rules!`. A procedural macro is a crate of its own, and the tokens it returns have no way to say "the crate that re-exports me", so shelf's generated code writes `::shelf::…` and assumes the dependency is called `shelf`. serde does the same, and offers `#[serde(crate = "…")]` for the rare caller who renamed it. Chapter 3 returns to the rest of a derive's hygiene: absolute paths everywhere, and the hidden module `::shelf::__private` that spares your crate a direct dependency on tokio-postgres.

2.5 Where declarative macros stop

`model!` maps each field to the column of its name. Real models need more, and the first thing they need is an attribute: this field reads the column `sku_code`. The natural syntax is an attribute on the field. `model!` does not accept it:

```
model! {
    pub struct Product {
        pub id: i64,
        #[col = "sku_code"]
        pub sku: String,
    }
}
```

```
error: no rules expected `#`
  --> src/bin/model_attr.rs:32:9
   |
 7 | macro_rules! model {
   | ------------------ when calling this macro
...
32 |         #[col = "sku_code"]
   |         ^ no rules expected this token in macro call
   |
note: while trying to match meta-variable `$field_vis:vis`
  --> src/bin/model_attr.rs:11:15
   |
11 |             $($field_vis:vis $field:ident : $ty:ty),* $(,)?
   |               ^^^^^^^^^^^^^^
```

Accepting the attribute is easy: add `$(#[$fattr:meta])*` before each field, as for the struct. *Acting* on it is not. A matcher cannot say "a `col = "…"` attribute if there is one, otherwise the field's name", because the transcriber has no `if`. The way out is a rule per kind of field, and a macro that walks the fields one at a time, calling itself on the rest with what it has built so far: a *tt-muncher*. Its shape, not compiled:

```
// A tt-muncher for model!'s fields (illustration): one field consumed per step.
(@fields [$($done:tt)*] #[col = $col:literal] $field:ident : $ty:ty, $($rest:tt)*) => {
    model!(@fields [$($done)* $field: row.try_get($col)?,] $($rest)*)
};
(@fields [$($done:tt)*] $field:ident : $ty:ty, $($rest:tt)*) => {
    model!(@fields [$($done)* $field: row.try_get(stringify!($field))?,] $($rest)*)
};
(@fields [$($done:tt)*]) => { /* write the impl from everything in $done */ };
```

That handles one attribute. Each new one (`skip`, `json`, `flatten`, `try_from`) adds rules, combinations of them multiply the rules, and an attribute that is misspelt does not match any rule, so the error is "no rules expected" at the `#`, with no word about what was wrong.

Generics go the same way:

```
model! {
    pub struct Page<T> {
        pub items: Vec<T>,
    }
}
```

```
error: no rules expected `<`
  --> src/bin/model_attr.rs:38:20
   |
 7 | macro_rules! model {
   | ------------------ when calling this macro
...
38 |     pub struct Page<T> {
   |                     ^ no rules expected this token in macro call
   |
note: while trying to match `{`
  --> src/bin/model_attr.rs:10:37
   |
10 |         $vis:vis struct $name:ident {
   |                                     ^
```

Matching `<T>` is possible. Matching `<'a, T: Clone, const N: usize>` and a `where` clause, and then adding the bound each field needs, is a parser for Rust's generics written in `macro_rules!`.

Each of these problems has the same root: `macro_rules!` matches tokens against fixed patterns, and a struct is not a fixed pattern. What the job needs is a macro that receives the struct as *data*, already parsed, with a list of fields it can loop over, attributes it can inspect with ordinary `if`s, generics it can modify, and a way to report a mistake at the exact token that caused it, in its own words. That is a procedural macro, and syn is the parser. Chapter 3 writes one.

Side by side — *generating impls for many types*

The task: `FromRow` for tuples, read by position. sqlx has the same impls, written by a macro with one invocation per tuple size:

```
// sqlx (illustration), from sqlx-core 0.9.0's src/from_row.rs, abridged
macro_rules! impl_from_row_for_tuple {
    ($( ($idx:tt) -> $T:ident );+;) => {
        impl<'r, R, $($T,)+> FromRow<'r, R> for ($($T,)+)
        where
            R: Row,
            usize: crate::column::ColumnIndex<R>,
            $($T: crate::decode::Decode<'r, R::Database> +
crate::types::Type<R::Database>,)+
        {
            #[inline]
            fn from_row(row: &'r R) -> Result<Self, Error> {
                Ok(($(row.try_get($idx as usize)?,)+))
            }
        }
    };
}

impl_from_row_for_tuple!(
    (0) -> T1;
    (1) -> T2;
);
// … one invocation per size, up to sixteen elements
```

```
let pairs: Vec<(String, i64)> = sqlx::query_as("SELECT sku, price_cents FROM products")
    .fetch_all(&pool)
    .await?;
```

By hand, two sizes written out, and shelf's own tuples, in one program:

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand: one impl per size, each written out.
impl<A: FromSqlOwned> FromRow for (A,) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?,))
    }
}

impl<A: FromSqlOwned, B: FromSqlOwned> FromRow for (A, B) {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok((row.try_get(0)?, row.try_get(1)?))
    }
}

const SQL: &str = "SELECT sku, price_cents FROM products ORDER BY sku";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2499);",
        )
        .await?;

    let by_hand = client
        .query(SQL, &[])
        .await?
        .iter()
        .map(<(String, i64)>::from_row)
        .collect::<Result<Vec<_>, _>>()?;

    // shelf: the impls come from one macro invocation in shelf/src/tuples.rs.
    let with_shelf: Vec<(String, i64)> = shelf::fetch_all(&client, SQL, &[]).await?;
    assert_eq!(by_hand, with_shelf);

    // A wrong element type: the error names the position and the column.
    let error = shelf::fetch_all::<(String, i32)>(&client, SQL, &[])
        .await
        .unwrap_err();
    println!("{error}");
    assert_eq!(
        error.to_string(),
        r#"tuple.1: column "price_cents" has type int8, which cannot be read into i32"#
    );
    Ok(())
}
```

```
tuple.1: column "price_cents" has type int8, which cannot be read into i32
```

	by hand	shelf	sqlx
Same	One impl per tuple size, reading by position	The same impls, written by a macro	The same impls, written by a macro
Different	Every impl typed out; tokio-postgres's error names only the column	Twelve sizes from one invocation, a line each; errors name the position (`tuple.1`) and the column, and are found once per statement	Sixteen sizes, one invocation each; generic over sqlx's databases

In shelf

shelf's tuple impls are §2.3's macro, with two differences. `FromRow` has the two methods Chapter 5 introduces, `resolve` (check the statement's columns once) and `from_row_at` (read a row), and the positions are pasted a second time through `stringify!`, so that an error can name the element (`"1"` in `tuple.1` above):

```
// orm/shelf/src/tuples.rs (excerpt)
macro_rules! tuple_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: FromColumn),+> FromRow for ($($T,)+) {
                type Columns = ();

                fn resolve(columns: &[Column]) -> Result<(), Error> {
                    $( column_at::<$T>(columns, "tuple", stringify!($i), $i)?; )+
                    Ok(())
                }

                fn from_row_at(row: &Row, _: &()) -> Result<Self, Error> {
                    Ok(($( get::<$T>(row, $i, "tuple", stringify!($i))?, )+))
                }
            }
        )+
    };
}

tuple_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
    // … one line per size, up to twelve elements
}
```

orm/shelf/src/tuples.rs

The bound is `FromColumn`, shelf's own name for tokio-postgres's `FromSqlOwned`, which exists to make one compiler error read better (Chapter 4). A tuple's `Columns` is `()`: its positions are fixed, so there is nothing to look up, and `resolve` only checks each column's type. This macro lives inside shelf and is not exported, so it can name `FromRow`, `Column` and `get` directly: its expansion is always in `tuples.rs`, where those names are imported. A macro for other crates would need the absolute paths of §2.4.

When not to use `macro_rules!`

- **When a generic impl does the job.** shelf's `FromColumn` has a single impl, `impl<T: FromSqlOwned> FromColumn for T`, which covers every type at once with no macro. A macro is for impls that differ in *shape*, like tuples of different sizes.

- **When a function does the job.** A function is type-checked where it is written, and its errors point at it. Reach for a macro when you need code that a function cannot express: new items, new fields, a variable number of arguments.
- **When the input needs attributes, generics or good error messages.** That is a procedural macro's job (Chapter 3).

Summary

- `macro_rules!` matches tokens against rules, in order, and pastes what it matched into a template. Fragment specifiers (`ident`, `ty`, `expr`, `tt`, `vis`, `meta`, `literal`) say what each metavariable accepts; `*`, `+` and `?` repeat.
- Macros run before type checking: they see tokens, never types, and that holds for procedural macros too.
- A `model!` macro can declare a struct and write its `FromRow` impl, so field names are written once. The struct then lives inside a macro call, which tools handle less well.
- One invocation writes `FromRow` for tuples of one to twelve elements, a line per size. Match positions as `tt`: a captured `literal` cannot be pasted as a tuple index. `${index()}` is still unstable in rustc 1.99.1.
- Local variables in a macro are hygienic; paths are resolved at the call site, so a macro for other people's code writes `::core::result::Result`. `$crate` names the macro's own crate.
- Field attributes, generics and readable errors are beyond what `macro_rules!` does well. A procedural macro gets the struct as parsed data.

Exercises

1. **One line instead of twelve.** Rewrite §2.3's `tuple_impls!` so that a single invocation, `tuple_impls!([] A 0, B 1, C 2, D 3)`, writes all four impls. *Hint:* keep the elements already handled in the brackets, an *accumulator*. Each step moves one element from the list into the accumulator, writes the impl for the accumulator, and calls the macro again with the rest. Peeling elements off the front without an accumulator gets the positions wrong: the rest would start at 1.
2. **The column list too.** Make `model!` also generate a constant `Product::COLUMNS` holding `"id, sku, price_cents, tags"`, and use it to write the query. *Hint:* a macro can call another macro: §2.1's `columns!` is half the work.
3. **Tuples as parameters.** Write a trait `Params` with `fn params(&self) -> Vec<&(dyn ToSql + Sync)>`, implement it for tuples of one to three elements with a macro, and insert a row with `("VASE-03", 3999_i64).params()`. *Hint:* a tuple's fields are `self.0`, `self.1`, and so on, which is where §2.3's `tt` earns its keep.

Solutions

1. Each step writes one impl and recurses, so the invocation is one line. The cost is a macro that is harder to read than the flat list, and a recursion depth, one level per element, that the compiler limits (128 levels by default):

```
use tokio_postgres::Row;
use tokio_postgres::types::FromSqlOwned;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

macro_rules! tuple_impls {
    // Nothing left to move: done.
    ([$($T:ident $i:tt),*]) => {};
    // Move the next element into the accumulator, write its impl, and go on with the rest.
    ([$($T:ident $i:tt),*] $U:ident $j:tt $(, $rest_t:ident $rest_i:tt)*) => {
```

```
        impl<$($T: FromSqlOwned,)* $U: FromSqlOwned> FromRow for ($($T,)* $U,) {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(($(row.try_get::<_, $T>($i)?,)* row.try_get::<_, $U>($j)?,))
            }
        }
        tuple_impls!([$($T $i,)* $U $j] $($rest_t $rest_i),*);
    };
}

tuple_impls!([] A 0, B 1, C 2, D 3);

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    let row = client
        .query_one("SELECT 'MUG-01' AS sku, 1299::int8, true, 3", &[])
        .await?;
    assert_eq!(<(String,)>::from_row(&row)?, ("MUG-01".to_string(),));
    assert_eq!(
        <(String, i64, bool, i32)>::from_row(&row)?,
        ("MUG-01".to_string(), 1299, true, 3)
    );
    Ok(())
}
```

2. `model!` hands its field names to `columns!`, which joins them at compile time:

```
use tokio_postgres::Row;

trait FromRow: Sized {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

macro_rules! columns {
    ($first:ident $(, $rest:ident)* $(,)?) => {
        concat!(stringify!($first) $(, ", ", stringify!($rest))*)
    };
}

macro_rules! model {
    (
        $(#[$meta:meta])*
        $vis:vis struct $name:ident {
            $($field_vis:vis $field:ident : $ty:ty),* $(,)?
        }
    ) => {
        $(#[$meta])*
        $vis struct $name {
            $($field_vis $field: $ty),*
        }

        impl $name {
            /// The columns `from_row` reads, as a `SELECT` list.
            const COLUMNS: &'static str = columns!($($field),*);
        }

        impl FromRow for $name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self {
                    $($field: row.try_get(stringify!($field))?),*
                })
            }
        }
    };
}

model! {
    #[derive(Debug)]
```

```
    struct Product {
        id: i64,
        sku: String,
        price_cents: i64,
        tags: Option<Vec<String>>,
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    assert_eq!(Product::COLUMNS, "id, sku, price_cents, tags");

    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (id bigint, sku text, price_cents bigint, tags
text[]);
             INSERT INTO products VALUES (1, 'MUG-01', 1299, NULL);",
        )
        .await?;
    // COLUMNS holds identifiers the macro wrote from the struct, never a value, so it can
    // go into the SQL text; values still travel as $n parameters.
    let sql = format!("SELECT {} FROM products WHERE id = $1", Product::COLUMNS);
    let product = Product::from_row(&client.query_one(&sql, &[&1_i64]).await?)?;
    assert_eq!(product.sku, "MUG-01");
    assert_eq!(
        (product.id, product.price_cents, product.tags),
        (1, 1299, None)
    );
    Ok(())
}
```

The `format!` runs on every call. Chapter 6 builds such statements once, at compile time, from constants the derive writes, and quotes each identifier on the way.

3. The impls use `self.$i`, which compiles only because `$i` is a `tt`:

```
use tokio_postgres::types::ToSql;

/// A tuple's values, as the parameter list `query` and `execute` take.
trait Params {
    fn params(&self) -> Vec<&(dyn ToSql + Sync)>;
}

macro_rules! params_impls {
    ($( ($($T:ident $i:tt),+) )+) => {
        $(
            impl<$($T: ToSql + Sync),+> Params for ($($T,)+) {
                fn params(&self) -> Vec<&(dyn ToSql + Sync)> {
                    vec![$(&self.$i as &(dyn ToSql + Sync)),+]
                }
            }
        )+
    };
}

params_impls! {
    (A 0)
    (A 0, B 1)
    (A 0, B 1, C 2)
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE products (sku text, price_cents bigint)")
        .await?;
```

```
    let inserted = client
        .execute(
            "INSERT INTO products VALUES ($1, $2)",
            &("VASE-03", 3999_i64).params(),
        )
        .await?;
    assert_eq!(inserted, 1);
    let price: i64 = client
        .query_one_scalar(
            "SELECT price_cents FROM products WHERE sku = $1",
            &("VASE-03",).params(),
        )
        .await?;
    assert_eq!(price, 3999);
    Ok(())
}
```

With `$i:literal` instead, `self.$i` is refused with the error shown in §2.3.

CHAPTER 3

The first derive macro

A procedural macro is a Rust function that runs inside the compiler. `#[derive(FromRow)]` hands it the tokens of your struct; it returns more tokens, an `impl` block, which the compiler compiles as if you had written it. There is no pattern language this time: the function parses the struct into a syntax tree, loops over its fields, looks at whatever it likes, and builds the code it needs with ordinary Rust.

This chapter writes the first version of shelf's derive: every named field read from the column of its name, which is Chapter 1's impl written by a program. Most of the work is not the generator, which is twenty lines of syn and quote, but the setting around it: the kind of crate it must live in, the crate it should live in instead, the paths that keep the generated code working in any module, the ways to see what was generated, and how a derive reports a mistake at the right token.

In this chapter you will learn:

- what a proc-macro crate is, what the compiler hands a derive, and what it takes back
- why shelf splits its macros into a code generator and a thin proc-macro shell, and what `proc_macro2` has to do with it
- how syn 3 parses a struct, and how quote builds tokens with interpolation and repetition
- what hygiene means for a derive: absolute paths, `__private`, raw identifiers
- three ways to see an expansion: printing it, `cargo expand`, and rust-analyzer
- how a derive reports an error as a value, pointing at the token that caused it

3.1 Proc-macro crates

A derive macro lives in a crate whose `Cargo.toml` says so:

```
# orm/shelf-macros/Cargo.toml (excerpt)
[lib]
proc-macro = true

[dependencies]
shelf-codegen = { workspace = true }
syn = { workspace = true }
```

`proc-macro = true` changes what the crate is. It is compiled for the machine running the compiler (not the one your program will run on), loaded into the compiler as a plugin, and its public functions are called during compilation. A derive is one such function, marked with the attribute that names it:

```
// orm/shelf-macros/src/lib.rs (excerpt)
#[proc_macro_derive(FromRow, attributes(shelf))]
pub fn derive_from_row(input: TokenStream) -> TokenStream {
    // …
}
```

`FromRow` is the name you write in `#[derive(…)]`. `attributes(shelf)` declares `#[shelf(…)]` as a *helper attribute* that may appear on the struct and its fields; Chapter 4 uses it. The function takes and returns a

`proc_macro::TokenStream`: a sequence of tokens (identifiers, punctuation, literals, and groups in brackets), not text and not a syntax tree.

What exactly does a derive receive? This derive, a probe written for this chapter and not part of shelf, turns its input into a string and adds it to the type as a constant:

```
#[proc_macro_derive(ShowInput, attributes(shelf))]
pub fn show_input(input: TokenStream) -> TokenStream {
    let text = input.to_string();
    let parsed: syn::DeriveInput = syn::parse(input).unwrap();
    let name = parsed.ident;
    quote! { impl #name { pub const INPUT: &'static str = #text; } }.into()
}
```

Applied to this struct, between two other derives:

```
/// A product in the catalog.
#[derive(Debug, show::ShowInput, Clone)]
#[allow(dead_code)]
struct Product {
    /// The stock-keeping unit.
    #[shelf(rename = "sku_code")]
    sku: String,
    #[cfg(any())]
    gone: i32,
    price_cents: i64,
}
```

`Product::INPUT` holds:

```
/// A product in the catalog.
#[allow(dead_code)] struct Product
{
    /// The stock-keeping unit.
    #[shelf(rename = "sku_code")] sku: String, price_cents: i64,
}
```

> **Macro lens:** a derive gets the whole item: doc comments, other attributes, and the `#[shelf(…)]` helper attributes it will read. It does not get the `#[derive(…)]` line itself, nor the other derives listed in it, and the field under `#[cfg(any())]`, a condition that is never true, is already gone: `cfg` is evaluated before derives run. What the derive returns is *added after* the struct; it does not replace it. That is why `Product::INPUT` compiles: the struct is still there for the `impl` to attach to.

A proc-macro crate has two restrictions, and both shape shelf. It **can export nothing but macros**. A public function next to the derive, say the code generator itself for a test to call, is refused:

```
error: `proc-macro` crate types currently cannot export any items other than functions
tagged with `#[proc_macro]`, `#[proc_macro_derive]`, or `#[proc_macro_attribute]`
  --> show/src/lib.rs:14:1
   |
14 | pub fn expand(input: &syn::DeriveInput) -> proc_macro2::TokenStream {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

And **its types work only inside the compiler**. `proc_macro::TokenStream` is a handle to data the compiler owns. In an ordinary program, `TokenStream::new()` happens to work, but parsing code into a token stream panics:

```
extern crate proc_macro;
fn main() {
```

```
    let parsed: proc_macro::TokenStream = "struct Product;".parse().unwrap();
}
```

```
thread 'main' (23461715) panicked at /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/
library/proc_macro/src/bridge/client.rs:200:32:
procedural macro API is used outside of a procedural macro
```

So the function at the heart of the derive can be called by the compiler and by nothing else: not by a unit test in another crate, not by an example in this book.

3.2 Three crates

That is why shelf is three crates (§0.4). `shelf-codegen` is an ordinary library that holds the code generation, as functions from a parsed `syn::DeriveInput` to a `proc_macro2::TokenStream`. `shelf-macros` is the proc-macro crate, and stays a shell of three lines per derive: parse the input, call the generator, and turn an error into a compile error:

```
// orm/shelf-macros/src/lib.rs (excerpt)
use proc_macro::TokenStream;
use syn::{DeriveInput, parse_macro_input};

#[proc_macro_derive(FromRow, attributes(shelf))]
pub fn derive_from_row(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    shelf_codegen::from_row::expand(&input)
        .unwrap_or_else(syn::Error::into_compile_error)
        .into()
}
```

orm/shelf-macros/src/lib.rs

`proc_macro2` makes the split possible. It is a crate with the same types as the compiler's `proc_macro` (`TokenStream`, `Ident`, `Span`, `Literal`, `Group`), which work in two modes. Inside a proc macro, they wrap the compiler's own types; anywhere else, they fall back to an implementation of their own. syn parses `proc_macro2` tokens and quote produces them, so `shelf-codegen` never touches `proc_macro`, and runs equally well in the compiler, in its tests, and in this book's examples. The shell converts at the edges: `parse_macro_input!` parses the compiler's tokens into a `DeriveInput` (or returns the parse error as a compile error), and `.into()` turns the generator's `proc_macro2::TokenStream` back into the compiler's.

`shelf` re-exports the derive (`pub use shelf_macros::FromRow;`), so your crate depends on `shelf` alone and writes `#[derive(shelf::FromRow)]`, or imports it with `use shelf::FromRow`, which brings in the trait and the derive at once (§0.7).

3.3 Parsing with syn 3

syn parses Rust source, as tokens, into a syntax tree. A derive needs one entry point, `DeriveInput`, which is a struct, an enum or a union with its attributes, visibility, name and generics. This program parses a struct the way a derive receives it and walks its fields:

```
use quote::ToTokens;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        /// A product in the catalog.
        pub struct Product {
            id: i64,
```

```
            pub sku: String,
            tags: Option<Vec<String>>,
        }
    };

    println!("name: {}", input.ident);
    println!("attributes: {}", input.attrs.len());
    let Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    for field in &fields.named {
        let name = field.ident.as_ref().expect("a named field");
        println!("  field {name}: {}", field.ty.to_token_stream());
    }

    assert_eq!(input.ident, "Product");
    assert_eq!(fields.named.len(), 3);
    assert_eq!(
        fields.named[2].ty.to_token_stream().to_string(),
        "Option < Vec < String > >"
    );
}
```

```
name: Product
attributes: 1
  field id: i64
  field sku: String
  field tags: Option < Vec < String > >
```

`parse_quote!` builds a syntax tree from Rust code written in place; the book's stage programs use it to stand in for the compiler. The doc comment is an attribute (`#[doc = "…"]`), hence `attributes: 1`. And the type of `tags` prints with spaces between its tokens, because it *is* tokens: `Option`, `<`, `Vec`, `<`, `String`, `>`, `>`. syn knows it is a path type with generic arguments, but it does not know what `Option` refers to (Chapter 2's refresher).

The pieces a derive walks through:

Type	What it holds
`DeriveInput`	`attrs`, `vis`, `ident` (the name), `generics`, and `data`
`Data`	`Struct(DataStruct)`, `Enum(DataEnum)` or `Union(DataUnion)`
`Fields`	`Named` (`{ id: i64 }`), `Unnamed` (`(i64, String)`) or `Unit`
`Field`	`attrs`, `vis`, `ident` (`None` in a tuple struct), `ty`, and two items new in syn 3

syn compiles only what you ask for. Its default features (`derive`, `parsing`, `printing`, `clone-impls` and `proc-macro`) are all a derive needs. The workspace also enables `full`, the whole of Rust's syntax, because `pretty` (§3.4) parses generated code into a `syn::File`, which `full` provides.

syn 3, not syn 2. shelf uses syn 3.0.9. Most material online, and most derives on crates.io, are written against syn 2, and for a derive the difference is small. `DeriveInput`, `Data`, `Fields` and `Attribute` are unchanged, as are `parse_nested_meta` (Chapter 4) and `split_for_impl`. `Field` changed in two places: syn 2's `mutability` became `modifiers: FieldModifiers`, and a new `default` holds a field's default value (`x: i32 = 5`, a language feature still unstable in Rust 1.99). Code that *reads* a field's `attrs`, `ident` and `ty`, as shelf does, compiles against either version; code that builds or destructures a whole `Field` does not. The rest of 3.0's breaking changes are in types, expressions, statements, patterns, items, generics and literals; the 3.0.0 release notes list them.

3.4 Generating with quote

quote's `quote!` macro is the other half: it turns Rust code written in place into a `proc_macro2::TokenStream`, and `#name` inside it pastes in any value that implements `ToTokens`. A repetition, `#(…)*`, pastes an iterator, one element at a time, with whatever separators the repetition contains. That is enough for a first derive. Here is its logic as a stage program: the generator as an ordinary function, run on a struct built with `parse_quote!`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

/// The first `#[derive(FromRow)]`: every named field read from the column of its name, as
/// Chapter 1 wrote it by hand.
fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.to_string();
        quote! { #ident: row.try_get(#column)? }
    });

    Ok(quote! {
        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#reads,)* })
            }
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            id: i64,
            sku: String,
            tags: Option<Vec<String>>,
        }
    };
    let code = shelf_playground::pretty(expand(&input).expect("a struct with named
fields"));
    println!("{code}");
    assert!(code.contains(r#"sku: row.try_get("sku")?"#));
    assert!(code.contains(r#"tags: row.try_get("tags")?"#));
}
```

```
impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Self {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            tags: row.try_get("tags")?,
        })
```

```
    }
}
```

That is Chapter 1's impl, letter for letter. Each value pasted into `quote!` becomes the tokens its type stands for:

- `#name` and `#ident` are `syn::Ident`s, pasted as identifiers.
- `#column` is a `String`, and a string is pasted as a string *literal*: `"sku"`, with the quotes. This is how the field's name becomes the column's name.
- `#(#reads,)*` pastes every element of the iterator `reads`, each followed by a comma.

Two helpers come up constantly. `quote::format_ident!` builds a new identifier from parts, as `format!` builds a string (`format_ident!("{}Columns", name)` is `ProductColumns`). And `syn::Index` pastes a number as a tuple index, which the next pitfall is about.

> **Pitfall:** a Rust integer pasted into `quote!` keeps its type as a suffix: `1usize`. That is right in an expression and wrong as a tuple field, which shelf's finished generator writes (`columns.0`, `columns.1`). The derive compiles, and its output fails to parse wherever it is used. `syn::Index::from(position)` pastes an unsuffixed `1`:

```
use quote::quote;
use syn::Index;

fn main() {
    let position: usize = 1;
    let suffixed = quote!(columns.#position);
    assert_eq!(suffixed.to_string(), "columns . 1usize");
    let error = syn::parse2::<syn::Expr>(suffixed)
        .err()
        .expect("a suffixed tuple index does not parse");
    assert_eq!(error.to_string(), "expected unsuffixed integer");

    let at = Index::from(position);
    assert_eq!(quote!(columns.#at).to_string(), "columns . 1");
}
```

The stage program printed its output with `pretty`, a helper in the shelf playground. It parses the tokens as a file of items and formats them with prettyplease, which formats as rustfmt would but works on a syntax tree instead of a file:

```
use proc_macro2::TokenStream;

/// Formats generated code as rustfmt would, so that an example can print what a derive
writes
/// and assert on it. A derive generates items (`impl` blocks, functions, constants), and
that
/// is what `tokens` must hold.
pub fn pretty(tokens: TokenStream) -> String {
    let file: syn::File = syn::parse2(tokens).expect("a derive generates items");
    prettyplease::unparse(&file)
}
```

`pretty` also checks the output: tokens that do not parse as items, such as the `1usize` above, make it panic in the stage program instead of failing later in a user's build.

3.5 Hygiene

The stage generator's output works in Chapter 1's program, and in few others. It names `FromRow`, `Row`, `Result`, `Ok` and `tokio_postgres` as bare paths, and the `impl` lands in the user's module, next to their struct. If that module has no `use tokio_postgres::Row`, or its own `type Result<T>`, or a different `FromRow`, the generated code breaks, or quietly means something else.

A procedural macro has even less protection than Chapter 2's `macro_rules!`. Every token `quote!` writes carries `Span::call_site()`, and the compiler resolves such tokens *as if you had typed them next to the struct*: paths, and local variables too. (`Span::mixed_site()` gives `macro_rules!`'s rules, where a macro's local variables are its own; shelf's generated code has no locals that could collide, so it does not need it.) So the generated code must not depend on any name the user's module defines:

- **Every path is absolute.** `::core::result::Result`, `::core::result::Result::Ok`, `::shelf::FromRow`, `::shelf::Error`. A leading `::` starts at a crate's name, which nothing in the user's module can shadow.
- **tokio-postgres is reached through shelf.** The generated code needs `Row` and `Column`, and the user's crate may not depend on tokio-postgres, or may depend on another version. So shelf re-exports what generated code names in a hidden module:

```
// orm/shelf/src/lib.rs (excerpt)
/// What generated code refers to. Not part of the API: it may change in any release.
#[doc(hidden)]
pub mod __private {
    pub use crate::row::{column, column_at, get, get_try_from};
    pub use tokio_postgres;
    pub use tokio_postgres::types::Json;
}
```

orm/shelf/src/lib.rs

`::shelf::__private::tokio_postgres::Row` is then the `Row` that shelf itself was compiled against, whatever the user's crate has. The double underscore and `#[doc(hidden)]` are the usual signal that a module is for macros only: it does not appear in the documentation, and it may change in any release. Its helpers, `column` and `get`, keep the generated code short; Chapter 5 explains them.

Raw identifiers. A field may be named after a keyword, with Rust's raw-identifier prefix: `r#type`. Its identifier's text includes the prefix, so the stage generator, which uses `ident.to_string()`, would read a column called `r#type`. syn's `IdentExt::unraw` strips it:

```
use syn::ext::IdentExt;
use syn::{DeriveInput, Fields, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { r#type: String }
    };
    let syn::Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    let ident = fields.named[0].ident.as_ref().expect("a named field");

    assert_eq!(ident.to_string(), "r#type"); // what the stage generator would read
    assert_eq!(ident.unraw().to_string(), "type"); // the column's name
}
```

The field itself is still written `r#type` in the generated initializer (`r#type: …`), where the prefix is needed; only the column's name drops it.

`#[automatically_derived]` goes on the generated `impl`. The Rust Reference says it "has no direct effect, but it may be used by tools and diagnostic lints to detect these automatically generated implementations". The standard library's derives add it, and so does every careful derive.

This program puts the finished derive in a hostile module: one with its own `Result`, and a constant named `Ok`. It also reads a column named after a keyword:

```
mod catalog {
    // Names that would capture a careless expansion.
    #[allow(dead_code)]
    pub type Result<T> = std::result::Result<T, String>;
    #[allow(dead_code, non_upper_case_globals)]
    const Ok: () = ();

    #[derive(Debug, shelf::FromRow)]
    pub struct Product {
        pub sku: String,
        pub r#type: String,
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, type text NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 'mug'), ('TEE-02', 'shirt');",
        )
        .await?;

    let products: Vec<catalog::Product> =
        shelf::fetch_all(&client, "SELECT sku, type FROM products ORDER BY sku",
&[]).await?;
    assert_eq!(products[0].r#type, "mug");
    assert_eq!(products[1].sku, "TEE-02");
    Ok(())
}
```

shelf's own tests do the same with a `type Result = ();`, a `const Ok`, and a trait called `Default`, which `#[shelf(skip)]` must not pick up (Chapter 4):

```
// orm/shelf/tests/derive.rs (excerpt)
// The generated code must not depend on what the user's module calls `Result`, `Ok` or
// `Default`.
#[allow(dead_code)]
type Result = ();
#[allow(dead_code, non_upper_case_globals)]
const Ok: () = ();
#[allow(dead_code)]
trait Default {}

#[allow(dead_code)] // only the impls matter here
#[derive(shelf::FromRow)]
struct Product {
    id: i64,
    r#type: String,
    tags: Option<Vec<String>>,
    #[shelf(skip)]
    cached: Vec<u8>,
}
```

orm/shelf/tests/derive.rs

3.6 Seeing the expansion

You will want to see what a derive wrote: to learn, to debug, and to review a change to the generator. There are three ways.

Print it. Because the generator lives in an ordinary library, any program can call it and format the result with `pretty`, as §0.7 and §3.4 did. This is the way this book uses throughout, because it works on stable Rust with nothing to install, and because the output can be asserted on. shelf's expansion tests do exactly this: each file in `orm/shelf-codegen/tests/expand/` holds structs with a derive, and a test compares the pretty-printed expansion of each with a file next to it, so that any change to the generated code shows up as a diff in review (Chapter 13).

Ask the compiler. rustc can print a crate after expanding every macro in it, with the unstable flag `-Zunpretty=expanded`. The tool cargo-expand wraps it (`cargo install cargo-expand`, then `cargo expand`), and formats the result with rustfmt. An unstable flag is normally refused by a stable compiler:

```
error: the option `Z` is only accepted on the nightly compiler
```

cargo-expand gets around that by setting `RUSTC_BOOTSTRAP=1` for the compiler it runs (its `src/main.rs`), so it works on stable Rust. This is what the compiler itself compiled for §3.5's `catalog::Product` (the raw command cargo-expand wraps, run the same way, before formatting):

```
    pub struct Product {
        pub sku: String,
        pub r#type: String,
    }
    #[automatically_derived]
    impl ::core::fmt::Debug for Product {
        #[inline]
        fn fmt(&self, f: &mut ::core::fmt::Formatter) -> ::core::fmt::Result {
            ::core::fmt::Formatter::debug_struct_field2_finish(f, "Product",
                "sku", &self.sku, "type", &&self.r#type)
        }
    }
    #[automatically_derived]
    impl ::shelf::FromRow for Product {
        type Columns = (usize, usize);
        fn resolve(columns: &[::shelf::__private::tokio_postgres::Column])
            -> ::core::result::Result<Self::Columns, ::shelf::Error> {
            ::core::result::Result::Ok((::shelf::__private::column::<String>(columns,
                            "Product", "sku", "sku")?,
                    ::shelf::__private::column::<String>(columns, "Product",
                            "type", "type")?))
        }
        fn from_row_at(row: &::shelf::__private::tokio_postgres::Row,
            columns: &Self::Columns)
            -> ::core::result::Result<Self, ::shelf::Error> {
            ::core::result::Result::Ok(Self {
                    sku: ::shelf::__private::get::<String>(row, columns.0,
                            "Product", "sku")?,
                    r#type: ::shelf::__private::get::<String>(row, columns.1,
                            "Product", "type")?,
                })
        }
    }
```

The compiler's view shows what `pretty` cannot: the expansion in place, next to every other derive. `#[derive(Debug)]`, from the standard library, follows the same rules as shelf: an `#[automatically_derived]` impl, every path starting at `::core`, and the field `r#type` named `"type"`, without its prefix. Expansion to text is lossy, as cargo-expand's README warns (hygiene, for one, does not survive printing), so treat it as a debugging aid.

Ask the editor. rust-analyzer's command "Expand macro recursively at caret" (in VS Code: *rust-analyzer: Expand macro recursively at caret*) shows the expansion of the macro under the cursor.

3.7 Errors as values

A derive that cannot do its job could panic, and the compiler would report it. This is a probe derive that panics on an enum:

```
error: proc-macro derive panicked
 --> user/src/bin/panics.rs:1:10
  |
1 | #[derive(show::ShowInput)]
  |          ^^^^^^^^^^^^^^^
  |
  = help: message: ShowInput needs a struct
```

The message survives, but the error points at the derive's name instead of the code that is wrong, and a panic can report only one problem. So shelf's generator never panics on user input. It returns `syn::Result<TokenStream>`, whose error is a value with a message and a *span*, the place in the user's source it is about; several errors can be combined into one (Chapter 4). `syn::Error::new_spanned(tokens, message)` takes the span from any tokens of the input. The shell turns the error into tokens with `into_compile_error`, and the compiler reports them as an error at that span.

This program runs shelf's generator on two inputs it refuses, and prints the message and the tokens the error becomes:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let status: DeriveInput = parse_quote! {
        enum Status { Pending, Paid }
    };
    let error = shelf_codegen::from_row::expand(&status).unwrap_err();
    println!("{error}");
    println!("{}", error.to_compile_error());
    assert!(error.to_string().contains("derive postgres_types::FromSql"));

    let pair: DeriveInput = parse_quote! {
        struct Pair(i32, i32);
    };
    let error = shelf_codegen::from_row::expand(&pair).unwrap_err();
    println!("{error}");
    assert!(error.to_string().starts_with("FromRow needs named fields"));
}
```

```
FromRow maps a row to a struct; for a Postgres enum, derive postgres_types::FromSql instead
:: core :: compile_error ! { "FromRow maps a row to a struct; for a Postgres enum, derive
postgres_types::FromSql instead" }
FromRow needs named fields: each one is read from the column of that name
```

An error becomes an invocation of the standard `compile_error!` macro, whose tokens carry the error's span. Here, outside the compiler, the span is a placeholder. In a real build it points at the user's code, and shelf's UI tests (Chapter 4) capture what the compiler then prints. For an enum, a tuple struct and a union:

```
// orm/shelf/tests/ui/shapes.rs (excerpt)
#[derive(FromRow)]
enum Status {
    Pending,
    Paid,
}
```

```
#[derive(FromRow)]
struct Pair(i32, i32);

#[derive(FromRow)]
union Bits {
    int: i32,
    float: f32,
}
```

```
error: FromRow maps a row to a struct; for a Postgres enum, derive postgres_types::FromSql
instead
 --> tests/ui/shapes.rs:5:1
  |
5 | enum Status {
  | ^^^^

error: FromRow needs named fields: each one is read from the column of that name
  --> tests/ui/shapes.rs:11:8
   |
11 | struct Pair(i32, i32);
   |        ^^^^

error: FromRow cannot be derived for a union
  --> tests/ui/shapes.rs:14:1
   |
14 | union Bits {
   | ^^^^^
```

Each error points at the token it was built from: the `enum` keyword (`data.enum_token`), the tuple struct's name (`input.ident`), the `union` keyword. The message says what to do instead, where there is something to do: a Postgres enum is one column, which postgres-types' derive maps (main book Chapter 5).

Spans also decide where errors in the *generated* code land, and the rule is simple: tokens the derive writes itself carry the span of the `#[derive]`, and tokens copied from the input keep their own. A field's type is copied from the struct into `get::<#ty>`, so if that type cannot be read from a column, the compiler's error points at the field's type in your struct, not at the derive. Chapter 4 uses that, and makes the message read well.

Side by side — *attaching generated behaviour to a type*

The task: give a struct a row-mapping impl. sqlx's derive generates one that is generic over every database sqlx supports; this is its shape for a two-field struct, from sqlx 0.9's code generator:

```
// sqlx (illustration): the shape of what #[derive(sqlx::FromRow)] generates in sqlx 0.9
#[automatically_derived]
impl<'a, R: ::sqlx::Row> ::sqlx::FromRow<'a, R> for Product
where
    &'a ::std::primitive::str: ::sqlx::ColumnIndex<R>,
    String: ::sqlx::decode::Decode<'a, R::Database>,
    String: ::sqlx::types::Type<R::Database>,
    i64: ::sqlx::decode::Decode<'a, R::Database>,
    i64: ::sqlx::types::Type<R::Database>,
{
    fn from_row(__row: &'a R) -> ::sqlx::Result<Self> {
        let sku: String = __row.try_get("sku")?;
        let price_cents: i64 = __row.try_get("price_cents")?;
        ::std::result::Result::Ok(Product { sku, price_cents })
    }
}
```

By hand (Chapter 1's trait) and with shelf's derive, on the same struct, in one program:

```
use tokio_postgres::Row;

trait FromRow: Sized {
```

```
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error>;
}

// By hand: the impl is yours to write, read and change.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl FromRow for Product {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

// shelf: the impl is generated next to the struct, at compile time.
#[derive(Debug, PartialEq, shelf::FromRow)]
struct ShelfProduct {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = shelf_playground::connect().await?;
    let sql = "SELECT 'MUG-01' AS sku, 1299::int8 AS price_cents";

    let by_hand = Product::from_row(&client.query_one(sql, &[]).await?)?;
    let with_shelf: ShelfProduct = shelf::fetch_one(&client, sql, &[]).await?;
    assert_eq!(
        (by_hand.sku, by_hand.price_cents),
        (with_shelf.sku, with_shelf.price_cents)
    );
    Ok(())
}
```

	by hand	shelf	sqlx
Same	An impl of a row-mapping trait, next to the struct	The same impl, written by a derive at compile time	The same impl, written by a derive at compile time
Different	You write and maintain it, and it can do anything	For tokio-postgres's `Row` only; paths are absolute and reach tokio-postgres through `::shelf::__private`; columns are found once per statement (Chapter 5)	Generic over every sqlx database (`R: Row`), with a `Decode` and a `Type` bound per field; each field is looked up by name on every row

In shelf

The finished generator is §3.4's stage program, grown. The skeleton is the same: refuse what is not a struct with named fields, loop over the fields, collect a piece of code per field, and paste the pieces into one `impl` with absolute paths. What the field loop collects is more, because `FromRow` has the two-step shape Chapter 5 explains, and the attributes of Chapter 4 are elided here:

```
// orm/shelf-codegen/src/from_row.rs (excerpt)
/// The `impl shelf::FromRow` for `input`, or every error found in it.
```

```
pub fn expand(input: &DeriveInput) -> Result<TokenStream> {
    let fields = named_fields(input)?;
    // … the attributes are parsed here, and every error is collected (Chapter 4)

    let model = input.ident.to_string();
    let mut bounds = Bounds::new(&input.generics);
    let mut column_types = Vec::new(); // the tuple `Columns`, one element per field
    let mut resolves = Vec::new(); // `resolve`'s tuple, element by element
    let mut reads = Vec::new(); // `from_row_at`'s field initializers

    for (position, (field, attrs)) in fields.iter().enumerate() {
        let ident = field.ident.as_ref().expect("named fields have identifiers");
        // A raw identifier names the column without its prefix: `r#type` reads `type`.
        let name = ident.unraw().to_string();
        let ty = &field.ty;
        let at = Index::from(position);

        // The type `resolve` checks the column against, and the expression that reads it.
        let (checked, read) = match attrs.read() {
            // … skip, flatten, json and try_from (Chapter 4)
            Read::Column => {
                bounds.add_if_generic(&[ty], parse_quote!(#ty: ::shelf::FromColumn));
                (
                    quote!(#ty),
                    quote!(::shelf::__private::get::<#ty>(row, columns.#at, #model,
#name)?),
                )
            }
        };
        // … `column` is the field's name, or what rename and rename_all make of it (Chapter
4)
        column_types.push(quote!(usize));
        resolves.push(quote! {
            ::shelf::__private::column::<#checked>(columns, #model, #name, #column)?
        });
        reads.push(quote!(#ident: #read));
    }

    // With nothing to read (every field skipped, or none at all), the parameters go unused.
    let reads_any = column_types.iter().any(|t| t.to_string() != "()");
    let (row, columns) = if reads_any {
        (quote!(row), quote!(columns))
    } else {
        (quote!(_row), quote!(_columns))
    };

    let name = &input.ident;
    let generics = bounds.apply(&input.generics);
    let (impl_generics, ty_generics, where_clause) = generics.split_for_impl();

    // Every path is absolute (`::shelf`, `::core`): the expansion lands in the user's
module,
    // where `Result` or even `shelf` may mean something else.
    Ok(quote! {
        #[automatically_derived]
        impl #impl_generics ::shelf::FromRow for #name #ty_generics #where_clause {
            type Columns = ( #(#column_types,)* );

            fn resolve(
                #columns: &[::shelf::__private::tokio_postgres::Column],
            ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
                ::core::result::Result::Ok(( #(#resolves,)* ))
            }

            fn from_row_at(
                #row: &::shelf::__private::tokio_postgres::Row,
                #columns: &Self::Columns,
            ) -> ::core::result::Result<Self, ::shelf::Error> {
                ::core::result::Result::Ok(Self { #(#reads,)* })
```

```
            }
        }
    })
}

/// The struct's named fields, or an error that says why the input cannot be mapped.
fn named_fields(input: &DeriveInput) -> Result<&Punctuated<Field, Token![,]>> {
    match &input.data {
        Data::Struct(data) => match &data.fields {
            Fields::Named(fields) => Ok(&fields.named),
            _ => Err(Error::new_spanned(
                &input.ident,
                "FromRow needs named fields: each one is read from the column of that name",
            )),
        },
        Data::Enum(data) => Err(Error::new_spanned(
            data.enum_token,
            "FromRow maps a row to a struct; for a Postgres enum, derive \
             postgres_types::FromSql instead",
        )),
        Data::Union(data) => Err(Error::new_spanned(
            data.union_token,
            "FromRow cannot be derived for a union",
        )),
    }
}
```

orm/shelf-codegen/src/from_row.rs

Three details are new since the stage program. The loop builds three lists instead of one, because each field contributes to `Columns`, to `resolve` and to `from_row_at`. `split_for_impl` splits the struct's generics into the three places an `impl` needs them (`impl<T>`, `Product<T>` and the `where` clause), with the bounds Chapter 4 adds. And a struct whose fields are all skipped would leave `row` and `columns` unused, so the generator names them `_row` and `_columns` then, as you would in hand-written code.

When not to write a derive

- **For a handful of types.** Chapter 1's impls are a few lines each, and need no proc-macro crate, and no syn, quote and proc-macro2 to compile before your own code.
- **When a blanket impl does the job.** If every type with some trait should get the behaviour, `impl<T: Trait> Other for T` needs no macro at all.
- **When the generated code would be hard to predict.** A derive's value is that users can guess its output. If they cannot, they will read the expansion every time, and a function they call explicitly would serve them better.

Summary

- A proc-macro crate (`proc-macro = true`) runs inside the compiler. A derive is a function from the item's tokens to new tokens, which are added after the item.
- A derive receives the whole item, attributes and doc comments included, but not the `#[derive]` line, and after `cfg` has been evaluated.
- A proc-macro crate can export only macros, and `proc_macro`'s types do not work outside the compiler. So shelf keeps code generation in `shelf-codegen`, on `proc_macro2`, with `shelf-macros` as a three-line shell per derive.
- syn parses tokens into `DeriveInput` → `Data` → `Fields` → `Field`; quote builds tokens with `#value` and `#(…)*`. A `String` pastes as a literal, a `usize` with a suffix (use `syn::Index` for tuple fields).
- syn 3's derive types are syn 2's, except two fields of `Field`; reading `attrs`, `ident` and `ty` is unchanged.

- A derive's tokens resolve at the call site: write every path absolute (`::core`, `::shelf`), reach tokio-postgres through `::shelf::__private`, strip `r#` with `unraw`, and mark the impl `#[automatically_derived]`.
- See an expansion with `pretty` (stable, assertable), `cargo expand` (the compiler's own view; runs on stable through `RUSTC_BOOTSTRAP`), or rust-analyzer.
- Errors are `syn::Error` values with spans, turned into `compile_error!`; tokens copied from the input keep their spans, so errors in generated code point at the user's code.

Exercises

1. **The column list, in Rust.** Extend §3.4's stage generator so that it also writes `impl Product { pub const COLUMNS: &'static str = "id, sku, tags"; }`. Compare with Chapter 2's Exercise 2, which built the same string with `macro_rules!`. *Hint:* the generator is ordinary Rust: collect the names into a `Vec<String>` and `join` them, then paste the result as one literal.
2. **Make it hygienic.** Rewrite §3.4's generator so that its output compiles in any module: absolute paths, and `r#type` reading `type`. Assume the trait is `::shelf_book::FromRow` and the row type is `::tokio_postgres::Row`. *Hint:* every bare name in the `quote!` is a name the user's module could define.
3. **An expansion test.** Write a test, as a program with an `assert_eq!`, that pins down the whole output of §3.4's generator for a two-field struct. *Hint:* compare `pretty`'s output with the expected text in a raw string literal, `r#"…"#`.

Solutions

1. In a proc macro the string is computed by ordinary Rust while the macro runs, and pasted as one literal; `macro_rules!` had to assemble it from `stringify!` and `concat!`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let idents: Vec<_> = fields
        .named
        .iter()
        .filter_map(|f| f.ident.as_ref())
        .collect();
    let columns: Vec<String> = idents.iter().map(|ident| ident.to_string()).collect();
    let list = columns.join(", ");

    Ok(quote! {
        impl #name {
            pub const COLUMNS: &'static str = #list;
        }

        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#idents: row.try_get(#columns)?,)* })
            }
```

```
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { id: i64, sku: String, tags: Option<Vec<String>> }
    };
    let code = shelf_playground::pretty(expand(&input).unwrap());
    println!("{code}");
    assert!(code.contains(r#"pub const COLUMNS: &'static str = "id, sku, tags";"#));
}
```

The repetition `#(#idents: row.try_get(#columns)?,)*` walks two lists in step, as Chapter 2's `$field` and `$ty` did.

2. Every path starts at a crate, and the column name comes from `unraw`:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::ext::IdentExt;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };

    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.unraw().to_string();
        quote! { #ident: row.try_get(#column)? }
    });

    Ok(quote! {
        #[automatically_derived]
        impl ::shelf_book::FromRow for #name {
            fn from_row(
                row: &::tokio_postgres::Row,
            ) -> ::core::result::Result<Self, ::tokio_postgres::Error> {
                ::core::result::Result::Ok(Self { #(#reads,)* })
            }
        }
    })
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product { sku: String, r#type: String }
    };
    let code = shelf_playground::pretty(expand(&input).unwrap());
    println!("{code}");
    assert!(code.contains("impl ::shelf_book::FromRow for Product"));
    assert!(code.contains(r#"r#type: row.try_get("type")?"#));
    assert!(code.contains("::core::result::Result::Ok(Self"));
}
```

`row` is still a bare name, and that is fine: it is a parameter the generated function declares itself, so the user's module cannot change what it means. This version still requires the user's crate to depend on tokio-postgres; shelf's `__private` re-export removes that last dependency.

3. The test fails, and prints both versions, as soon as the generator's output changes in any way, which is what shelf's expansion tests are for:

```
use proc_macro2::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Fields, parse_quote};

fn expand(input: &DeriveInput) -> syn::Result<TokenStream> {
    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs a struct",
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            &input.ident,
            "FromRow needs named fields",
        ));
    };
    let name = &input.ident;
    let reads = fields.named.iter().map(|field| {
        let ident = field.ident.as_ref().expect("named fields have names");
        let column = ident.to_string();
        quote! { #ident: row.try_get(#column)? }
    });
    Ok(quote! {
        impl FromRow for #name {
            fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
                Ok(Self { #(#reads,)* })
            }
        }
    })
}

const EXPECTED: &str = r#"impl FromRow for Price {
    fn from_row(row: &Row) -> Result<Self, tokio_postgres::Error> {
        Ok(Self {
            cents: row.try_get("cents")?,
            currency: row.try_get("currency")?,
        })
    }
}
"#;

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Price { cents: i64, currency: String }
    };
    let actual = shelf_playground::pretty(expand(&input).unwrap());
    assert_eq!(actual, EXPECTED, "the expansion changed:\n{actual}");
}
```

shelf's version, `orm/shelf-codegen/tests/expand.rs`, keeps the structs and their expected expansions in files, and rewrites the expected files when run with `SHELF_BLESS=1`, so that a deliberate change is accepted by reading a diff rather than by editing strings.

CHAPTER 4

Attributes, errors and generics

A real schema does not match your structs one to one. A column is called `sku_code` but the field is `sku`; a legacy table uses `camelCase`; a field holds a value computed in Rust and is not in the table at all; a `jsonb` column holds a document you want as a struct; an `int` should become a `Grams` that refuses negative values. Each of those is one attribute on a field, and serde has taught every Rust developer to expect them.

This chapter teaches the derive to read `#[shelf(…)]`. Parsing is the easy part. The work is in what happens when the attributes are wrong: a misspelt key, a value where none belongs, two attributes that contradict each other. A good derive reports all of them in one build, each at the token that caused it, in words the user understands. The same care goes into generic structs, which need exactly the bounds their fields require, and no others.

In this chapter you will learn:

- how a derive declares a helper attribute, and what happens if it forgets
- how to parse `#[shelf(…)]` with `parse_nested_meta` into a typed struct
- what `rename_all`, `rename`, `skip`, `flatten`, `json` and `try_from` generate
- how to report every mistake in one build, each at the right token
- how to test a derive's errors with trybuild, and what reading them changed in shelf
- how to give a generic struct the bounds its fields need, and why shelf refuses lifetimes
- what darling would do for you, and why shelf does without it

---

4.1 Helper attributes

The compiler resolves every attribute it meets. `#[allow(…)]` and `#[doc = "…"]` are built in; `#[tokio::main]` is an attribute macro, found by its path; `#[rustfmt::skip]` and `#[diagnostic::…]` belong to tools. `#[shelf(…)]` is none of those. It is a *helper attribute*: a name that a derive declares, which the compiler then accepts on any item that uses the derive, and leaves in place for the derive to read. It does nothing by itself. shelf declares it in the shell of Chapter 3:

```
// orm/shelf-macros/src/lib.rs (excerpt)
#[proc_macro_derive(FromRow, attributes(shelf))]
```

What happens without the declaration? This probe derive, written for this chapter like §3.1's `ShowInput` and not part of shelf, generates nothing and declares no helper:

```
#[proc_macro_derive(NoHelper)]
pub fn no_helper(_input: TokenStream) -> TokenStream {
    TokenStream::new()
}
```

Put it on a struct with a `#[shelf(skip)]` field, in a crate that depends on shelf, and the build fails:

```
error: cannot find attribute `shelf` in this scope
 --> user/src/bin/no_helper.rs:3:7
  |
3 |     #[shelf(skip)]
  |       ^^^^^
  |
  = note: `shelf` is in scope, but it is a crate, not an attribute
```

The derive never ran. The compiler refused the attribute while resolving names, before any macro saw the struct, and the note says where it looked: the only `shelf` in scope is the crate, and a crate is not an attribute. shelf met exactly this error when its hygiene test first used `#[shelf(skip)]` (PLAN.md §12 #9). Forgetting the `#[derive(FromRow)]` itself gives the same error, with the same note, for the same reason: a helper attribute exists only on an item whose derive declares it.

All of shelf's derives will declare the same helper, so `#[shelf(…)]` is one namespace, shared by `FromRow` and, from Chapter 6 on, `Table`, `Insert` and `Patch`. That decides how shelf parses it.

4.2 Parsing with `parse_nested_meta`

An attribute arrives as a `syn::Attribute`: its path, `shelf`, and the tokens between the parentheses, not yet parsed. syn does not know what `rename = "sku_code"` means to you, but it knows the shape that most attributes share, which its documentation calls the *conventional syntax for structured attributes*: a comma-separated list of items, each a path, optionally followed by `= value` or by a list in parentheses of the same shape. serde's attributes follow it, and so do shelf's.

`Attribute::parse_nested_meta` walks that list. For each item it parses the path and calls your closure with it, as `meta.path`. The closure reads whatever follows the path: `meta.value()?` consumes the `=` and returns the stream to parse the value from. When the closure returns, syn expects a comma or the end of the list. This stage program is a parser for three of shelf's keys, run on a struct built with `parse_quote!`:

```
use quote::ToTokens;
use syn::{Attribute, Data, DeriveInput, Fields, LitStr, Type, parse_quote};

/// The stage version of shelf's `FieldAttrs`: three keys, as typed values.
#[derive(Default)]
struct FieldAttrs {
    rename: Option<String>,
    skip: bool,
    try_from: Option<Type>,
}

fn parse(attrs: &[Attribute]) -> syn::Result<FieldAttrs> {
    let mut out = FieldAttrs::default();
    for attr in attrs.iter().filter(|attr| attr.path().is_ident("shelf")) {
        attr.parse_nested_meta(|meta| {
            if meta.path.is_ident("rename") {
                // `rename = "…"`: `value()` reads the `=`, then comes a string literal.
                let lit: LitStr = meta.value()?.parse()?;
                out.rename = Some(lit.value());
            } else if meta.path.is_ident("skip") {
                // A flag: the key is all there is.
                out.skip = true;
            } else if meta.path.is_ident("try_from") {
                // `try_from = i64`: the value is a type.
                out.try_from = Some(meta.value()?.parse()?);
            } else {
                return Err(meta.error("unknown shelf attribute"));
            }
            Ok(())
        })?;
    }
    Ok(out)
```

```
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            #[shelf(rename = "sku_code")]
            sku: String,
            #[shelf(try_from = i64, rename = "price_cents")]
            price: Cents,
            /// Set by the user interface; not stored.
            #[shelf(skip)]
            selected: bool,
        }
    };
    let Data::Struct(data) = &input.data else {
        panic!("a struct")
    };
    let Fields::Named(fields) = &data.fields else {
        panic!("named fields")
    };
    let mut parsed = Vec::new();
    for field in &fields.named {
        let attrs = parse(&field.attrs).expect("valid attributes");
        let try_from = attrs
            .try_from
            .as_ref()
            .map(|ty| ty.to_token_stream().to_string());
        println!(
            "{}: rename {:?}, skip {}, try_from {:?}",
            field.ident.as_ref().expect("a named field"),
            attrs.rename,
            attrs.skip,
            try_from
        );
        parsed.push(attrs);
    }
    assert_eq!(parsed[0].rename.as_deref(), Some("sku_code"));
    assert_eq!(parsed[1].rename.as_deref(), Some("price_cents"));
    assert!(parsed[1].try_from.is_some());
    assert!(parsed[2].skip);

    // Three inputs the stage parser handles badly.
    let unknown: Attribute = parse_quote!(#[shelf(renamed = "sku")]);
    let Err(error) = parse(&[unknown]) else {
        panic!("an unknown key is refused")
    };
    println!("renamed = \"sku\": {error}");

    let flag_with_value: Attribute = parse_quote!(#[shelf(skip = true)]);
    let Err(error) = parse(&[flag_with_value]) else {
        panic!("a value after a flag is refused")
    };
    println!("skip = true: {error}");
    assert_eq!(error.to_string(), "expected `,`");

    let twice: Attribute = parse_quote!(#[shelf(rename = "a", rename = "b")]);
    let attrs = parse(&[twice]).expect("accepted, wrongly");
    println!("rename twice: {:?}", attrs.rename);
    assert_eq!(attrs.rename.as_deref(), Some("b"));
}
```

```
sku: rename Some("sku_code"), skip false, try_from None
price: rename Some("price_cents"), skip false, try_from Some("i64")
selected: rename None, skip true, try_from None
renamed = "sku": unknown shelf attribute
skip = true: expected `,`
rename twice: Some("b")
```

Each kind of key reads its value its own way. `rename` parses a string literal, a `LitStr`, and keeps its contents without the quotes. `skip` is a *flag*: there is nothing after the key to read. `try_from` parses a `syn::Type`, because every node of syn's syntax tree can be parsed from a stream, a type as much as a literal. The doc comment on `selected` is an attribute too (`#[doc = "…"]`), and the filter on `shelf` passes over it. And `price` shows that one attribute can hold several keys; two `#[shelf(…)]` attributes would parse the same.

The last three lines are what the stage parser gets wrong. An unknown key is refused, but the message does not say which keys exist. A flag given a value, `skip = true`, fails with syn's "expected `,`": the closure read nothing after `skip`, so syn looked for the next item and found `=`. That is true, and no help to whoever wrote it. And a key written twice is accepted, with the second value silently winning. shelf's parser keeps the stage's shape and fixes all three:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// A parsed value, with the span of the key that set it, for error messages.
pub struct Keyed<T> {
    pub span: Span,
    pub value: T,
}

// … `FieldAttrs` has an `Option<Keyed<…>>` per key: `Keyed<LitStr>` for `rename`, and so on

impl FieldAttrs {
    pub fn parse(attrs: &[Attribute], errors: &mut Errors) -> Self {
        let mut out = FieldAttrs::default();
        for attr in shelf_attrs(attrs) {
            let parsed = attr.parse_nested_meta(|meta| {
                if meta.path.is_ident("rename") {
                    let lit: LitStr = meta.value()?.parse()?;
                    if lit.value().is_empty() {
                        return Err(Error::new_spanned(&lit, "a column name cannot be
empty"));
                    }
                    once(&mut out.rename, lit, &meta)
                } else if meta.path.is_ident("skip") {
                    flag(&mut out.skip, &meta)
                    // … `flatten` and `json` the same way
                } else if meta.path.is_ident("try_from") {
                    let ty = type_value(&meta)?;
                    once(&mut out.try_from, ty, &meta)
                } else {
                    Err(unknown(
                        &meta,
                        "a field",
                        &["rename", "skip", "flatten", "json", "try_from"],
                    ))
                }
            });
            if let Err(error) = parsed {
                errors.push(error);
            }
        }
        out.check_combinations(errors);
        out
    }
    // …
}
```

orm/shelf-codegen/src/attrs.rs

Every value is kept as a `Keyed<T>`: the value, and the span of the key that set it, so that a later check can point at `json` or `rename` in the user's source (§4.4). A flag is a `Keyed<()>`: all that matters is that it was there, and where. Errors are not returned but pushed onto `errors`, which collects them (§4.4). The rest of the work is in four small functions:

```rust
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Stores a key's value, refusing a second one.
fn once<T>(slot: &mut Option<Keyed<T>>, value: T, meta: &ParseNestedMeta) -> Result<()> {
    if slot.is_some() {
        return Err(meta.error(format!("duplicate `{}`", name_of(meta))));
    }
    *slot = Some(Keyed {
        span: meta.path.span(),
        value,
    });
    Ok(())
}

/// A key without a value, such as `skip`.
fn flag(slot: &mut Option<Keyed<()>>, meta: &ParseNestedMeta) -> Result<()> {
    if meta.input.peek(Token![=]) || meta.input.peek(syn::token::Paren) {
        return Err(meta.error(format!("`{}` takes no value", name_of(meta))));
    }
    once(slot, (), meta)
}

/// A type after `=`: `try_from = i64`, or quoted as serde writes it, `try_from = "i64"`.
fn type_value(meta: &ParseNestedMeta) -> Result<Type> {
    let value = meta.value()?;
    if value.peek(LitStr) {
        let lit: LitStr = value.parse()?;
        lit.parse()
            .map_err(|_| Error::new_spanned(&lit, "expected a type, as in `try_from =
i64`"))
    } else {
        value.parse()
    }
}

fn unknown(meta: &ParseNestedMeta, on: &str, known: &[&str]) -> Error {
    let known: Vec<String> = known.iter().map(|k| format!("`{k}`")).collect();
    meta.error(format!(
        "unknown shelf attribute `{}` on {on}; expected {}",
        name_of(meta),
        known.join(", ")
    ))
}
```

orm/shelf-codegen/src/attrs.rs

- `once` stores a value and refuses a second one.
- `flag` looks ahead before anything else (`peek` reads a token without consuming it): if an `=` or a parenthesis follows the key, it says that the key takes no value, instead of leaving syn to complain about a comma.
- `type_value` accepts a type as it is, `try_from = i64`, or quoted, `try_from = "i64"`, as serde writes its own `try_from`. A quoted type is parsed from the literal's contents by `LitStr::parse`.
- `unknown` lists the keys that do exist.

`meta.error(…)` builds a `syn::Error` that covers everything from the key to the last token the closure has read. So a duplicate `rename = "e2"` is underlined whole, because `once` refused it after its value was parsed, while `skip = true` underlines only `skip`, because `flag` refused it before reading on. One of shelf's UI tests (§4.5) holds six such mistakes, and this is what the compiler prints for them:

```
error: expected `=`
 --> tests/ui/bad_values.rs:6:19
  |
6 |     #[shelf(rename)]
  |                   ^

error: expected string literal
 --> tests/ui/bad_values.rs:8:22
```

```
  |
8 |     #[shelf(rename = 1)]
  |                      ^

error: a column name cannot be empty
  --> tests/ui/bad_values.rs:10:22
   |
10 |     #[shelf(rename = "")]
   |                      ^^

error: `skip` takes no value
  --> tests/ui/bad_values.rs:12:13
   |
12 |     #[shelf(skip = true)]
   |             ^^^^

error: duplicate `rename`
  --> tests/ui/bad_values.rs:14:28
   |
14 |     #[shelf(rename = "e1", rename = "e2")]
   |                            ^^^^^^^^^^^^

error: expected a type, as in `try_from = i64`
  --> tests/ui/bad_values.rs:16:24
   |
16 |     #[shelf(try_from = "not a type")]
   |                        ^^^^^^^^^^^^
```

The first two errors come from syn itself, at the token where parsing failed: after `rename`, syn wanted an `=` and found the end of the list (so it points at the closing parenthesis), and `LitStr` wanted a string where there was a number. The other four are shelf's.

> **Macro lens:** a proc macro on stable Rust cannot merge two spans into one (`Span::join` is unstable), so how does syn underline `rename = "e2"` from its first token to its last? It keeps both ends. `to_compile_error` writes `::core::compile_error!` with the first span on every token of the path, and the braces around the message with the last span, and the compiler underlines the whole invocation, which now runs from one to the other (syn 3.0.9, `error.rs`).

A key whose value is itself a list, `key(…)`, is parsed by calling `meta.parse_nested_meta` inside the closure: it walks the inner list with the same rules. `FromRow` has no such key; the relations of Chapter 11 will.

One namespace, every key. Because all of shelf's derives share `#[shelf(…)]`, each of them will parse every key and then use the ones that concern it. An unknown key is still refused, so that a typo cannot hide behind another derive. Today the only key on the struct is `rename_all`, and `table`, which belongs to Chapter 6's `Table` derive, does not exist yet, so `FromRow` refuses it (`tests/ui/unknown_attributes.rs`). Chapter 6 adds it to the shared parser, and `FromRow` then ignores it.

4.3 The six attributes

What each attribute changes is easiest to see in the generated code. This program runs shelf's generator on a struct that uses all six:

```rust
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(rename_all = "camelCase")]
        struct Listing {
```

```
            #[shelf(rename = "sku_code")]
            sku: String,
            unit_price: i64,
            #[shelf(flatten)]
            stock: Stock,
            #[shelf(json)]
            attributes: Attributes,
            #[shelf(try_from = i32)]
            weight: Grams,
            #[shelf(skip)]
            selected: bool,
        }
    };
    let expanded = shelf_codegen::from_row::expand(&input).expect("valid attributes");
    let code = shelf_playground::pretty(expanded);
    println!("{code}");

    assert!(code.contains(r#""sku", "sku_code")"#)); // rename
    assert!(code.contains(r#""unit_price", "unitPrice")"#)); // rename_all
    assert!(code.contains("<Stock as ::shelf::FromRow>::resolve(columns)?")); // flatten
    assert!(code.contains("::shelf::__private::Json<Attributes>")); // json
    assert!(code.contains("get_try_from")); // try_from
    assert!(code.contains("selected: ::core::default::Default::default()")); // skip
}
```

```
#[automatically_derived]
impl ::shelf::FromRow for Listing {
    type Columns = (
        usize,
        usize,
        <Stock as ::shelf::FromRow>::Columns,
        usize,
        usize,
        (),
    );
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<String>(columns, "Listing", "sku", "sku_code")?,
            ::shelf::__private::column::<
                i64,
            >(columns, "Listing", "unit_price", "unitPrice")?,
            <Stock as ::shelf::FromRow>::resolve(columns)?,
            ::shelf::__private::column::<
                ::shelf::__private::Json<Attributes>,
            >(columns, "Listing", "attributes", "attributes")?,
            ::shelf::__private::column::<i32>(columns, "Listing", "weight", "weight")?,
            (),
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            sku: ::shelf::__private::get::<String>(row, columns.0, "Listing", "sku")?,
            unit_price: ::shelf::__private::get::<
                i64,
            >(row, columns.1, "Listing", "unit_price")?,
            stock: <Stock as ::shelf::FromRow>::from_row_at(row, &columns.2)?,
            attributes: ::shelf::__private::get::<
                ::shelf::__private::Json<Attributes>,
            >(row, columns.3, "Listing", "attributes")?
                .0,
            weight: ::shelf::__private::get_try_from::<
                i32,
                Grams,
```

```
            >(row, columns.4, "Listing", "weight")?,
            selected: ::core::default::Default::default(),
        })
    }
}
```

Against the plain field of Chapter 3, which `resolve` finds with `column::<T>` and `from_row_at` reads with `get::<T>` (Chapter 5 explains the pair), each attribute changes one thing:

Attribute	`Columns` holds	`resolve` calls	`from_row_at` reads with
none	`usize`	`column::<T>`	`get::<T>`
`rename`, `rename_all`	`usize`	`column::<T>`, with another name	`get::<T>`
`skip`	`()`	nothing	`Default::default()`
`flatten`	`<T as FromRow>::Columns`	`T::resolve`	`T::from_row_at`
`json`	`usize`	`column::<Json<T>>`	`get::<Json<T>>`, then `.0`
`try_from = S`	`usize`	`column::<S>`	`get_try_from::<S, T>`

In the generator, this is the `match` that Chapter 3's excerpt elided. `attrs.read()` says how a field is read, once its attributes are known to be consistent (§4.4):

```
// orm/shelf-codegen/src/from_row.rs (excerpt)
// The type `resolve` checks the column against, and the expression that reads it.
let (checked, read) = match attrs.read() {
    Read::Skip => {
        bounds.add_if_generic(&[ty], parse_quote!(#ty: ::core::default::Default));
        column_types.push(quote!(()));
        resolves.push(quote!(()));
        reads.push(quote!(#ident: ::core::default::Default::default()));
        continue;
    }
    Read::Flatten => {
        bounds.add_if_generic(&[ty], parse_quote!(#ty: ::shelf::FromRow));
        column_types.push(quote!(<#ty as ::shelf::FromRow>::Columns));
        resolves.push(quote!(<#ty as ::shelf::FromRow>::resolve(columns)?));
        reads.push(quote! {
            #ident: <#ty as ::shelf::FromRow>::from_row_at(row, &columns.#at)?
        });
        continue;
    }
    // … Read::Column, as in Chapter 3
    Read::Json => {
        let json = quote!(::shelf::__private::Json<#ty>);
        bounds.add_if_generic(&[ty], parse_quote!(#json: ::shelf::FromColumn));
        (
            json.clone(),
            quote!(::shelf::__private::get::<#json>(row, columns.#at, #model, #name)?.0),
        )
    }
    Read::TryFrom(source) => {
        // … the bounds a generic struct needs (§4.6)
        (
            quote!(#source),
            quote! {
                ::shelf::__private::get_try_from::<#source, #ty>(
                    row, columns.#at, #model, #name,
                )?
            },
        )
    }
};
let column = match &attrs.rename {
    Some(rename) => rename.value.value(),
```

```
    None => container.column_for(&name),
};
```

orm/shelf-codegen/src/from_row.rs

`rename` and `rename_all`

`rename` replaces the column name; `rename_all` on the struct sets a convention for every field without one. Both work on the field's name after `unraw` (§3.5), and `rename` wins. shelf knows four conventions, written by hand in `case.rs` rather than taken from the `heck` crate, because four is all a PostgreSQL schema needs. `snake_case` is the default, because PostgreSQL folds unquoted names to lower case, so most schemas are written that way. The other three turn up in tables created with quoted names, often by tools in other languages:

```
// orm/shelf-codegen/src/case.rs (excerpt)
pub fn apply(self, field: &str) -> String {
    match self {
        Case::Snake => field.to_owned(),
        Case::ScreamingSnake => field.to_ascii_uppercase(),
        Case::Camel | Case::Pascal => {
            let mut out = String::with_capacity(field.len());
            let mut upper = self == Case::Pascal;
            for c in field.chars() {
                if c == '_' {
                    upper = !out.is_empty() || self == Case::Pascal;
                } else if upper {
                    out.extend(c.to_uppercase());
                    upper = false;
                } else {
                    out.push(c);
                }
            }
            out
        }
    }
}
```

orm/shelf-codegen/src/case.rs

The conversion runs inside the derive, so the generated code holds the finished name as a literal (`"unitPrice"` above) and pays nothing for it at run time. Names are then matched exactly, as PostgreSQL reports them: in SQL a camelCase column must be quoted, `"unitPrice"`, or the server folds it to `unitprice`.

`skip`

A skipped field is not read: it gets `::core::default::Default::default()`, with the absolute path that kept §3.5's hostile module, with its own trait called `Default`, from capturing it. Its element of `Columns` is `()` rather than nothing, so that every other field's element keeps the index of its field: `columns.4` is still the fifth field's.

`flatten`

A flattened field is a struct with its own `FromRow`, read from the same row. Its `Columns` is nested whole in the outer tuple, its `resolve` searches the same column list, and its `from_row_at` gets the same row with its own part of `columns`. The inner struct's derive does its own naming: the outer struct's `rename_all` does not reach into it. Chapter 1's limitation remains (§1.4): a struct flattened twice reads the same columns twice, and the way out is still to alias the columns in SQL.

`json`

`json` reads the column through `Json<T>`, postgres-types' wrapper (main book §3.7), whose `FromSql` accepts `json` and `jsonb` and deserializes with serde, so `T` must implement `Deserialize`. The generated code unwraps it with `.0`. Without the attribute, the field would have to be a `Json<T>`.

> **Pitfall:** `#[shelf(json)]` on an `Option<T>` field reads `Json<Option<T>>`, and that is not the same as `Option<Json<T>>`. A JSON `null` in the column becomes `None`, but an SQL `NULL` fails the row with a `Null` error: in postgres-types 0.2.14 only `Option<T>` decodes NULL (its `from_sql_null` returns `None`), and here the `Option` is inside the wrapper, whose `from_sql_null` is the default `WasNull` error. This program reads all three cases:

```
use serde::Deserialize;
use shelf::{Error, FromRow, Mapping};

#[derive(Debug, Deserialize)]
struct Attributes {
    colour: String,
}

#[derive(Debug, FromRow)]
struct Listing {
    #[shelf(json)]
    attributes: Option<Attributes>,
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = shelf_playground::connect().await?;

    let document: Listing = shelf::fetch_one(
        &client,
        r#"SELECT '{"colour": "blue"}'::jsonb AS attributes"#,
        &[],
    )
    .await?;
    assert_eq!(
        document.attributes.map(|a| a.colour).as_deref(),
        Some("blue")
    );

    let json_null: Listing =
        shelf::fetch_one(&client, "SELECT 'null'::jsonb AS attributes", &[]).await?;
    assert!(json_null.attributes.is_none());

    let sql_null = shelf::fetch_one::<Listing>(&client, "SELECT NULL::jsonb AS attributes",
&[])
        .await
        .unwrap_err();
    println!("{sql_null}");
    assert!(matches!(
        sql_null,
        Error::Mapping {
            problem: Mapping::Null { .. },
            ..
        }
    ));

    // Without the attribute, Option<Json<T>> reads an SQL NULL as None.
    let row = client.query_one("SELECT NULL::jsonb", &[]).await?;
    let attributes: Option<tokio_postgres::types::Json<Attributes>> = row.try_get(0)?;
    assert!(attributes.is_none());
    Ok(())
}
```

```
Listing.attributes: column "attributes" is NULL; read it into an Option
```

The message's advice does not fit this case: the field already is an `Option`. For a nullable `json` or `jsonb` column, declare the field as `Option<tokio_postgres::types::Json<T>>` without the attribute. sqlx has a separate key for the case, `#[sqlx(json(nullable))]`.

`try_from`

`try_from = S` reads the column as `S`, then converts with `TryFrom<S>`. The type to read is written as a type, `try_from = i32`, or quoted as serde writes it. A conversion that fails, a negative weight for a `Grams`, fails its row with a `Mapping::Decode` error that names the struct, the field and the column, and carries the conversion's own error as its source. For that, the conversion's error type must implement `std::error::Error + Send + Sync + 'static`, and the generated code says so in its bounds.

The side by side at the end of the chapter reads all six attributes from a table, by hand and with shelf, and shows that error next to the one hand-written code gives.

4.4 Every error at once

A derive that stops at its first complaint makes you rebuild once per mistake. A `syn::Error` can hold several messages instead: `combine` appends one error to another, `into_compile_error` turns each into its own `compile_error!`, and iterating an error yields them one by one. shelf collects them in a small wrapper:

```rust
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Errors gathered while parsing, so that one build reports all of them, not only the
first.
#[derive(Default)]
pub struct Errors(Option<Error>);

impl Errors {
    pub fn push(&mut self, error: Error) {
        match &mut self.0 {
            Some(all) => all.combine(error),
            None => self.0 = Some(error),
        }
    }

    pub fn finish(self) -> Result<()> {
        self.0.map_or(Ok(()), Err)
    }
}
```

orm/shelf-codegen/src/attrs.rs

and the generator parses everything before it gives up. This is the beginning of `from_row::expand`, which Chapter 3 elided:

```rust
// orm/shelf-codegen/src/from_row.rs (excerpt)
pub fn expand(input: &DeriveInput) -> Result<TokenStream> {
    let fields = named_fields(input)?;

    let mut errors = Errors::default();
    for lifetime in input.generics.lifetimes() {
        errors.push(Error::new_spanned(
            lifetime,
            "FromRow reads owned values, so the struct cannot borrow from the row: use
`String` \
             instead of `&str`, and `Vec<u8>` instead of `&[u8]`",
        ));
    }
    let container = ContainerAttrs::parse(&input.attrs, &mut errors);
    let fields: Vec<(&Field, FieldAttrs)> = fields
        .iter()
        .map(|field| (field, FieldAttrs::parse(&field.attrs, &mut errors)))
        .collect();
```

```
    errors.finish()?;
    // …
}
```

orm/shelf-codegen/src/from_row.rs

Only a struct of the wrong shape stops it at once (`named_fields(input)?`), because then there are no fields to look at. Everything else is pushed: the lifetimes (§4.6), the struct's attributes, and every field's. This program gives the generator a struct with five mistakes:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        #[shelf(rename_all = "kebab-case")]
        struct Product {
            #[shelf(renamed = "sku_code", skip = 1)]
            sku: String,
            #[shelf(skip, rename = "price_cents")]
            price: i64,
            #[shelf(json, try_from = i32)]
            weight: Grams,
        }
    };
    let error = shelf_codegen::from_row::expand(&input).expect_err("mistakes to report");
    let messages: Vec<String> = error.into_iter().map(|e| e.to_string()).collect();
    for message in &messages {
        println!("{message}");
    }
    assert_eq!(messages.len(), 4);
    assert!(messages[1].starts_with("unknown shelf attribute `renamed`"));
}
```

```
unknown case: expected "snake_case", "camelCase", "PascalCase" or "SCREAMING_SNAKE_CASE"
unknown shelf attribute `renamed` on a field; expected `rename`, `skip`, `flatten`, `json`,
`try_from`
`rename` cannot apply to a field marked `skip`, which is not read
`try_from` and `json` cannot be combined: read the JSON into a type that converts itself
```

Five mistakes, four errors. `skip = 1` is not reported, because it follows `renamed` in the same attribute: `parse_nested_meta` returns as soon as the closure returns an error, and the rest of that attribute is never parsed (syn 3.0.9, `meta.rs`). So the unit of collection is the attribute: each `#[shelf(…)]` reports at most one parse error, and every attribute of every field is parsed. Fix the first error in an attribute, and the next one shows up on the next build.

The last two errors are of another kind. Each key was valid alone, and the combination is not. Those checks run after a field's attributes are parsed, and point at the key that does not fit:

```
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// Refuses keys that cannot go together, pointing at the one that does not fit.
fn check_combinations(&self, errors: &mut Errors) {
    let others = [
        ("rename", self.rename.as_ref().map(|a| a.span)),
        ("flatten", self.flatten.as_ref().map(|a| a.span)),
        ("json", self.json.as_ref().map(|a| a.span)),
        ("try_from", self.try_from.as_ref().map(|a| a.span)),
    ];
    if self.skip.is_some() {
        for (name, span) in others {
            if let Some(span) = span {
                errors.push(Error::new(
                    span,
                    format!(
                        "`{name}` cannot apply to a field marked `skip`, which is not read"
```

```
                    ),
                ));
            }
        }
    } else if self.flatten.is_some() {
        // … the same, for every key but `flatten`
    } else if let (Some(_), Some(try_from)) = (&self.json, &self.try_from) {
        errors.push(Error::new(
            try_from.span,
            "`try_from` and `json` cannot be combined: read the JSON into a type that \
             converts itself",
        ));
    }
}
```

orm/shelf-codegen/src/attrs.rs

Each refusal has a reason the message gives. A skipped field is not read, so a `rename`, `json` or `try_from` on it would be dead. A flattened struct reads its own columns, so a name or a wrapper on the outer field would mean nothing. And `json` with `try_from` has two plausible meanings (convert the JSON value, or convert after decoding), so shelf refuses it and points at the answer serde already has: a type that converts itself while it is deserialized. The UI test for these three, with the span of each error on the key that does not fit:

```
error: `rename` cannot apply to a field marked `skip`, which is not read
  --> tests/ui/conflicts.rs:11:19
   |
11 |     #[shelf(skip, rename = "a")]
   |                   ^^^^^^

error: `json` cannot apply to a `flatten` field, whose struct reads its own columns
  --> tests/ui/conflicts.rs:13:22
   |
13 |     #[shelf(flatten, json)]
   |                      ^^^^

error: `try_from` and `json` cannot be combined: read the JSON into a type that converts
itself
  --> tests/ui/conflicts.rs:15:19
   |
15 |     #[shelf(json, try_from = i64)]
   |                   ^^^^^^^^
```

So an error lands at one of three places (PLAN.md §12 #12): at a key, when the key is the problem (`Keyed::span`, or `meta.error` before the value is read); at a value, when the value is (`Error::new_spanned(&lit, …)`, as for an empty name or an unknown case); or at a field's type, when the compiler finds that the generated code does not type-check. That last kind is the subject of the next section.

4.5 UI tests

An error message is part of a derive's interface, so shelf tests it like one. trybuild compiles each file in `tests/ui/`, expects it to fail, and compares the compiler's output with the `.stderr` file beside it. One test function covers the whole directory:

```
// orm/shelf/tests/ui.rs (excerpt)
#[test]
fn derive_errors() {
    trybuild::TestCases::new().compile_fail("tests/ui/*.rs");
}
```

orm/shelf/tests/ui.rs

Each file is a small program that should not compile, with its mistakes on separate lines so that each error has its own line number, and the `.stderr` beside it holds the compiler's output: `conflicts.rs` holds the three fields of §4.4, and `conflicts.stderr` is what §4.4 printed. There is a pair for each kind of mistake: the shapes of §3.7, unknown keys, the bad values of §4.2, the conflicts of §4.4, an unreadable type and a lifetime (below).

trybuild builds the files in a project of its own, which depends on shelf, and normalizes what the compiler prints before it compares, so that the files do not depend on the machine (trybuild 1.0.122, `normalize.rs`). Paths become relative to the crate (`tests/ui/…`). Line numbers are kept for the test file and erased for every other file, so that editing shelf's sources does not break every test. And a list that rustc cuts short with "and 20 others" says "and $N others".

The first run of a new test has no `.stderr` to compare with. trybuild then fails the test and writes what the compiler said into `wip/`, for you to read and move into place. Run with `TRYBUILD=overwrite` and it writes the file in place, and also replaces any `.stderr` that no longer matches:

```
TRYBUILD=overwrite cargo test -p shelf --test ui
git diff orm/shelf/tests/ui/
```

The expected files are rustc's words, so a new toolchain may change them without any change to shelf; shelf's are rustc 1.99.1's. Regenerate them deliberately, and read the diff as you would read a change to an API.

What reading them changed

Reading those files for the first time changed shelf. For a field whose type tokio-postgres cannot read, such as `std::time::Instant`, the error named the bound the generated code had used: `shelf::__private::tokio_postgres::types::FromSqlOwned`, a path that no user should ever have to see. shelf now bounds field types by a trait of its own, implemented for every type the driver can read, and carrying a message:

```
// orm/shelf/src/row.rs (excerpt)
#[diagnostic::on_unimplemented(
    message = "`{Self}` cannot be read from a PostgreSQL column",
    label = "tokio-postgres has no `FromSql` for this type",
    note = "use a type tokio-postgres reads, add `#[shelf(json)]` or `#[shelf(try_from =
…)]`, \
            or implement `FromSql` for it"
)]
pub trait FromColumn: FromSqlOwned {}

impl<T: FromSqlOwned> FromColumn for T {}
```

orm/shelf/src/row.rs

`#[diagnostic::on_unimplemented]` was stabilized in Rust 1.78.0 (the release notes: "Stabilize the `#[diagnostic]` namespace and `#[diagnostic::on_unimplemented]` attribute"). When a type does not implement the trait, the compiler uses `message` as the error's headline, `label` under the offending code and `note` as a note, and then adds its own explanation. It works through the blanket impl: `Instant` fails `FromColumn` because it fails `FromSqlOwned`, and the headline is still shelf's (PLAN.md §12 #11). For this struct

```
// A field type that tokio-postgres cannot read: the compiler's error names the field's
type.
use shelf::FromRow;

#[derive(FromRow)]
struct Timing {
    started: std::time::Instant,
}

fn main() {}
```

the compiler now says:

```
error[E0277]: `Instant` cannot be read from a PostgreSQL column
 --> tests/ui/not_readable.rs:6:14
  |
6 |     started: std::time::Instant,
  |              ^^^^^^^^^^^^^^^^^^ tokio-postgres has no `FromSql` for this type
  |
  = help: the trait `for<'a> shelf::__private::tokio_postgres::types::FromSql<'a>` is not
implemented for `Instant`
  = note: use a type tokio-postgres reads, add `#[shelf(json)]` or `#[shelf(try_from = …)]`,
or implement `FromSql` for it
  = help: the following other types implement trait
`shelf::__private::tokio_postgres::types::FromSql<'a>`:
            &'a [u8]
            &'a str
            Box<T>
            Box<[T]>
            Box<str>
            Cow<'a, str>
            HashMap<String, Option<String>, S>
            IpAddr
          and $N others
  = note: required for `Instant` to implement
`shelf::__private::tokio_postgres::types::FromSqlOwned`
  = note: required for `Instant` to implement `FromColumn`
note: required by a bound in `shelf::__private::column`
 --> src/row.rs
  |
  | pub fn column<T: FromColumn>(
  |                  ^^^^^^^^^^ required by this bound in `column`
```

The error points at the field's type in your struct, not at the derive, because quote keeps the spans of the tokens it copies from the input (§3.7): `column::<#ty>` carries the span of the `std::time::Instant` you wrote.

That is the first of two errors (`tests/ui/not_readable.stderr` has both). The second is the same, except for its last note, `required by a bound in shelf::__private::get`: the type is used twice in the generated code, by `column` in `resolve` and by `get` in `from_row_at`, and each use is checked. shelf accepts the repetition, because the one way to report it once loses both the message and the position (§4.6).

4.6 Generics

A generic struct needs a generic impl: `impl<T, M> FromRow for Page<T, M>`, with whatever bounds make its body compile. syn's `Generics::split_for_impl` gives the three pieces such an impl needs, `<T, M>` after `impl`, `<T, M>` after the type's name and the `where` clause, and the generator adds its own predicates to the `where` clause first. Which predicates? It depends on how each field is read, so shelf adds one per field, and only for a field whose type mentions a type parameter:

```
use syn::{DeriveInput, parse_quote};

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Page<T, M> {
            total: i64,
            item: T,
            #[shelf(flatten)]
            meta: M,
            #[shelf(json)]
            extra: Vec<T>,
        }
    };
    let expanded = shelf_codegen::from_row::expand(&input).expect("valid attributes");
    let code = shelf_playground::pretty(expanded);
    // The impl's header: everything before its first item.
    let header: Vec<&str> = code
```

```
        .lines()
        .take_while(|line| !line.contains("type Columns"))
        .collect();
    println!("{}", header.join("\n"));

    assert!(code.contains("T: ::shelf::FromColumn,"));
    assert!(code.contains("M: ::shelf::FromRow,"));
    assert!(code.contains("::shelf::__private::Json<Vec<T>>: ::shelf::FromColumn,"));
    assert!(!code.contains("i64: ")); // no bound for a concrete type
}
```

```
#[automatically_derived]
impl<T, M> ::shelf::FromRow for Page<T, M>
where
    T: ::shelf::FromColumn,
    M: ::shelf::FromRow,
    ::shelf::__private::Json<Vec<T>>: ::shelf::FromColumn,
{
```

`item: T` is read from a column, so `T: FromColumn`. `meta: M` is flattened, so `M: FromRow`. `extra: Vec<T>` is read through `Json`, so its bound is on `Json<Vec<T>>`, not on `T`: for that field alone, `T` would only have to be deserializable. And `total: i64` gets no bound.

The standard library's derives work differently: `#[derive(Clone)]` on `Page<T, M>` requires `T: Clone` and `M: Clone`, one bound per parameter, whatever the fields are. For `FromRow` that rule would be wrong, because the right bound depends on the attribute: a parameter bounded by `FromColumn` could not be flattened. This program uses a generic wrapper that adds a total to any row type, a derived struct and a tuple included:

```
use shelf::FromRow;

/// A row of a page, with the number of rows the whole query has.
#[derive(Debug, FromRow)]
struct Counted<T> {
    total: i64,
    #[shelf(flatten)]
    item: T,
}

#[derive(Debug, PartialEq, FromRow)]
struct Product {
    sku: String,
    price_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO products VALUES ('MUG-01', 1299), ('TEE-02', 2500), ('CAP-03',
1800);",
        )
        .await?;

    // count(*) OVER () is computed before LIMIT: every row carries the full count.
    let page: Vec<Counted<Product>> = shelf::fetch_all(
        &client,
        "SELECT sku, price_cents, count(*) OVER () AS total FROM products ORDER BY sku LIMIT
2",
        &[],
    )
    .await?;
    assert_eq!(page.len(), 2);
    assert_eq!(page[0].total, 3);
```

```
    assert_eq!(
        page[0].item,
        Product {
            sku: "CAP-03".into(),
            price_cents: 1800
        }
    );

    // A tuple is a FromRow too; it reads by position, the first column here.
    let skus: Vec<Counted<(String,)>> = shelf::fetch_all(
        &client,
        "SELECT sku, count(*) OVER () AS total FROM products ORDER BY sku",
        &[],
    )
    .await?;
    assert_eq!(skus[2].item.0, "TEE-02");
    Ok(())
}
```

`Counted<Product>` needs `Product: FromRow`, which holds; `T: FromColumn`, the `Clone`-style bound, would have refused it.

The code that decides is short. `mentions_param` walks a type's tokens looking for the name of a type parameter, inside groups too, so `Vec<T>` and `Option<(T, i64)>` count:

```
// orm/shelf-codegen/src/generics.rs (excerpt)
/// Adds `predicate` if any of `types` mentions a type parameter. A bound on a concrete
/// type would be checked where the derive is, far from the field, so it is left out: the
/// generated code reports a concrete type that does not fit at the field itself.
pub fn add_if_generic(&mut self, types: &[&Type], predicate: WherePredicate) {
    if types.iter().any(|ty| self.mentions_param(ty)) {
        self.predicates.push(predicate);
    }
}

// … `apply` adds the predicates to a clone of the struct's `where` clause
fn mentions(tokens: TokenStream, params: &[Ident]) -> bool {
    tokens.into_iter().any(|tree| match tree {
        TokenTree::Ident(ident) => params.contains(&ident),
        TokenTree::Group(group) => mentions(group.stream(), params),
        _ => false,
    })
}
```

orm/shelf-codegen/src/generics.rs

The check works on tokens, not on meaning, so it can be fooled: a concrete type that happens to share a parameter's name, say a path ending in `::T`, would get a bound too. That costs nothing when the bound holds, and darling's `usage` module (§4.7) is the precise alternative.

Why no bounds on concrete types

Why not bound every field's type, `std::time::Instant: FromColumn` included, so that §4.5's error appears once instead of twice? This probe derive, written for this chapter, does that for a struct's first field, in an empty impl of a marker trait that is there only to carry the `where` clause:

```
#[proc_macro_derive(BoundFirst)]
pub fn bound_first(input: TokenStream) -> TokenStream {
    // … `ty`, the first field's type, and `name`, the struct's
    quote! {
        impl ::core::marker::Unpin for #name where #ty: ::shelf::FromColumn {}
    }
    .into()
}
```

On §4.5's `Timing`, rustc 1.99.1 reports one error, as hoped, and loses both things §4.5 gained:

```
error[E0277]: the trait bound `Instant:
shelf::__private::tokio_postgres::types::FromSqlOwned` is not satisfied
 --> user/src/bin/bound_first.rs:2:10
  |
2 | #[derive(show::BoundFirst)]
  |          ^^^^^^^^^^^^^^^^ the trait `for<'a>
shelf::__private::tokio_postgres::types::FromSql<'a>` is not implemented for `Instant`
  |
  = help: the following other types implement trait
`shelf::__private::tokio_postgres::types::FromSql<'a>`:
            &'a [u8]
            &'a str
            Box<T>
            Box<[T]>
            Box<str>
            Cow<'a, str>
            HashMap<String, Option<String>, S>
            IpAddr
          and 20 others
  = note: required for `Instant` to implement
`shelf::__private::tokio_postgres::types::FromSqlOwned`
  = note: this error originates in the derive macro `show::BoundFirst` (in Nightly builds,
run with -Z macro-backtrace for more info)
```

The error points at the derive, not at the field, although `#ty` carries the field's span. And it is in the driver's words: to accept the `where` clause, the compiler first needs `Instant` to meet `FromColumn`'s supertrait, `FromSqlOwned`, and it reports that failure, without `FromColumn`'s message. Two errors at the field that say what to do are better than one at the derive that does not, so shelf bounds only what is generic, and leaves concrete types to be checked where the generated code uses them.

Lifetimes are refused

A struct that borrows from the row, `struct Product<'a> { sku: &'a str }`, is refused, with an error at the lifetime that says what to use instead:

```
error: FromRow reads owned values, so the struct cannot borrow from the row: use `String`
instead of `&str`, and `Vec<u8>` instead of `&[u8]`
 --> tests/ui/lifetime.rs:5:16
  |
5 | struct Product<'a> {
  |                ^^
```

Borrowing would save copying each string out of the row's buffer (main book §3.4). But `FromRow` returns `Self` with no lifetime tied to the row, and making it `FromRow<'r>` would spread that lifetime to every function that maps rows: `fetch_all` could no longer return the values, because the rows they borrow from would be dropped as it returns (Exercise 3). shelf keeps the trait simple and maps owned values. The generator does not try to tell a harmless lifetime from a borrow: every lifetime parameter is refused, each with its own error.

4.7 What darling would save

darling is the crate most derives use for this chapter's work. Version 0.24.1 is built on syn 3. You declare the attributes as a struct and derive a parser for it, much as serde derives a deserializer. This is shelf's parser in darling, written for this chapter and compiled against darling 0.24.1 in a probe:

```
// darling 0.24.1 (illustration): shelf's attributes as darling receivers
use darling::util::{Flag, SpannedValue};
use darling::{FromDeriveInput, FromField, FromMeta, ast};

#[derive(FromDeriveInput)]
```

```
#[darling(attributes(shelf), supports(struct_named))]
struct Input {
    data: ast::Data<(), FieldAttrs>,
    rename_all: Option<Case>,
}

#[derive(FromMeta)]
enum Case {
    #[darling(rename = "snake_case")]
    Snake,
    #[darling(rename = "camelCase")]
    Camel,
    // … PascalCase and SCREAMING_SNAKE_CASE the same way
}

#[derive(FromField)]
#[darling(attributes(shelf), and_then = FieldAttrs::check)]
struct FieldAttrs {
    rename: Option<SpannedValue<String>>,
    skip: Flag,
    flatten: Flag,
    json: Flag,
    try_from: Option<SpannedValue<syn::Type>>,
}

impl FieldAttrs {
    /// The combinations shelf refuses: darling cannot know them.
    fn check(self) -> darling::Result<Self> {
        let mut errors = darling::Error::accumulator();
        if self.json.is_present()
            && let Some(try_from) = &self.try_from
        {
            errors.push(
                darling::Error::custom("`try_from` and `json` cannot be combined")
                    .with_span(&try_from.span()),
            );
        }
        // … the other combinations, the same way
        errors.finish_with(self)
    }
}

// In the derive: Input::from_derive_input(&input), and on failure, errors.write_errors().
```

On a struct with the mistakes of shelf's UI tests, plus one attribute with two, this is what the probe's build reports:

```
error: Unknown value: `kebab-case`. Available values: `PascalCase`, `SCREAMING_SNAKE_CASE`,
`camelCase`, `snake_case`
 --> userd/src/bin/darling.rs:4:22
  |
4 | #[shelf(rename_all = "kebab-case")]
  |                      ^^^^^^^^^^^^

error: Unknown field: `renamed`. Did you mean `rename`?
 --> userd/src/bin/darling.rs:6:13
  |
6 |     #[shelf(renamed = "sku")]
  |             ^^^^^^^

error: `rename` cannot apply to a field marked `skip`
 --> userd/src/bin/darling.rs:8:28
  |
8 |     #[shelf(skip, rename = "a")]
  |                            ^^^

error: `try_from` and `json` cannot be combined
  --> userd/src/bin/darling.rs:10:30
```

```
   |
10 |     #[shelf(json, try_from = i64)]
   |                              ^^^

error: Duplicate field `rename`
  --> userd/src/bin/darling.rs:12:28
   |
12 |     #[shelf(rename = "e1", rename = "e2")]
   |                            ^^^^^^

error: Unexpected type `bool`
  --> userd/src/bin/darling.rs:14:20
   |
14 |     #[shelf(skip = true)]
   |                    ^^^^

error: Unexpected type `int`
  --> userd/src/bin/darling.rs:16:22
   |
16 |     #[shelf(rename = 1)]
   |                      ^

error: Unknown field: `renamed`. Did you mean `rename`?
  --> userd/src/bin/darling.rs:24:13
   |
24 |     #[shelf(renamed = "x", skip = 1, json)]
   |             ^^^^^^^

error: Unexpected type `int`
  --> userd/src/bin/darling.rs:24:35
   |
24 |     #[shelf(renamed = "x", skip = 1, json)]
   |                                   ^
```

What darling does for you:

- **Unknown keys, with a suggestion.** "Did you mean `rename`?" comes from comparing the unknown name with the known ones (the default `suggestions` feature).
- **Duplicates, wrong kinds of value, and enum values**, each with its message and the list of what would have been accepted.
- **Every error in an attribute.** darling goes on after an error inside one attribute, so the attribute on line 24 gets two errors (`renamed` and `skip = 1`) where shelf's parser would report one.
- **Spans.** `SpannedValue<T>` and `Flag` keep where each value came from, as shelf's `Keyed` does; the span is the value's rather than the key's.
- **`darling::usage`**, which finds where a struct's type parameters are used, the precise version of §4.6's token check.

What it does not do: shelf's combinations are shelf's rules, so they are written by hand in `and_then` either way. The wording of its own messages is darling's ("Unexpected type `bool`" where shelf says "`skip` takes no value"). And it is five more crates to compile before your derive: darling, darling_core, darling_macro, ident_case and, for the suggestions, strsim.

shelf parses by hand because this book is about the mechanism: `attrs.rs` is about 250 lines, and every one of them is something darling would otherwise be doing for you. In a derive of your own with more than a few keys, darling is the better choice. Either way, once you have written the parser yourself, darling no longer looks like magic.

Side by side — *attribute-driven mapping*

The task: read a legacy table whose names and types do not match the struct. Its columns were created with quoted camelCase names, except `sku_code`, added later; two of them belong in a nested struct; a `jsonb` column

holds a document; an `int` must become a `Grams` that refuses negative values; and one field is not in the table at all. sqlx's derive takes nearly the same attributes:

```
// sqlx (illustration): the same struct with sqlx 0.9's #[sqlx(…)] attributes
#[derive(sqlx::FromRow)]
#[sqlx(rename_all = "camelCase")]
struct Listing {
    #[sqlx(rename = "sku_code")]
    sku: String,
    unit_price: i64,
    #[sqlx(flatten)]
    stock: Stock,
    #[sqlx(json)]
    attributes: Attributes,
    #[sqlx(try_from = "i32")]
    weight: Grams,
    #[sqlx(skip)]
    selected: bool,
}
```

By hand and with shelf, on the same table, in one program:

```
use std::error::Error;

use serde::Deserialize;
use shelf::FromRow;
use tokio_postgres::Row;
use tokio_postgres::types::Json;

/// A document in a `jsonb` column.
#[derive(Debug, PartialEq, Deserialize)]
struct Attributes {
    colour: String,
    ml: Option<u32>,
}

/// A weight that is never negative, although the column would allow it.
#[derive(Debug, PartialEq)]
struct Grams(u32);

impl TryFrom<i32> for Grams {
    type Error = std::num::TryFromIntError;

    fn try_from(value: i32) -> Result<Self, Self::Error> {
        u32::try_from(value).map(Grams)
    }
}

/// Two columns of the same row, in a struct of their own.
#[derive(Debug, PartialEq, FromRow)]
#[shelf(rename_all = "camelCase")]
struct Stock {
    on_hand: i32,
    reserved: i32,
}

// By hand: every name, wrapper and conversion, written out in the body.
struct HandListing {
    sku: String,
    unit_price: i64,
    stock: Stock,
    attributes: Attributes,
    weight: Grams,
    selected: bool,
}

impl HandListing {
    fn from_row(row: &Row) -> Result<Self, Box<dyn Error + Send + Sync>> {
```

```
        Ok(HandListing {
            sku: row.try_get("sku_code")?,
            unit_price: row.try_get("unitPrice")?,
            stock: Stock {
                on_hand: row.try_get("onHand")?,
                reserved: row.try_get("reserved")?,
            },
            attributes: row.try_get::<_, Json<Attributes>>("attributes")?.0,
            weight: Grams::try_from(row.try_get::<_, i32>("weight")?)?,
            selected: false,
        })
    }
}

// shelf: the same mapping, as attributes.
#[derive(Debug, FromRow)]
#[shelf(rename_all = "camelCase")]
struct Listing {
    #[shelf(rename = "sku_code")]
    sku: String,
    unit_price: i64,
    #[shelf(flatten)]
    stock: Stock,
    #[shelf(json)]
    attributes: Attributes,
    #[shelf(try_from = i32)]
    weight: Grams,
    #[shelf(skip)]
    selected: bool,
}

const SELECT: &str = r#"SELECT sku_code, "unitPrice", "onHand", reserved, attributes, weight
                        FROM listings ORDER BY sku_code"#;

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error + Send + Sync>> {
    let client = shelf_playground::connect().await?;
    client
        .batch_execute(
            r#"CREATE TEMP TABLE listings (
                   sku_code    text PRIMARY KEY,
                   "unitPrice" bigint NOT NULL,
                   "onHand"    int NOT NULL,
                   reserved    int NOT NULL,
                   attributes  jsonb NOT NULL,
                   weight      int NOT NULL
               );
               INSERT INTO listings VALUES
                   ('MUG-01', 1299, 40, 3, '{"colour": "blue", "ml": 350}', 410),
                   ('TEE-02', 2500, 12, 0, '{"colour": "black"}', 180);"#,
        )
        .await?;

    let rows = client.query(SELECT, &[]).await?;
    let by_hand = rows
        .iter()
        .map(HandListing::from_row)
        .collect::<Result<Vec<_>, _>>()?;
    let with_shelf: Vec<Listing> = shelf::fetch_all(&client, SELECT, &[]).await?;
    for (hand, shelf) in by_hand.iter().zip(&with_shelf) {
        assert_eq!(hand.sku, shelf.sku);
        assert_eq!(hand.unit_price, shelf.unit_price);
        assert_eq!(hand.stock, shelf.stock);
        assert_eq!(hand.attributes, shelf.attributes);
        assert_eq!(hand.weight, shelf.weight);
        assert_eq!(hand.selected, shelf.selected);
    }
    assert_eq!(with_shelf[0].stock.on_hand, 40);
    assert_eq!(with_shelf[1].attributes.ml, None);
```

```
    // A weight the conversion refuses: the same failure, reported both ways.
    client
        .execute(
            "UPDATE listings SET weight = -5 WHERE sku_code = 'TEE-02'",
            &[],
        )
        .await?;
    let rows = client.query(SELECT, &[]).await?;
    let Err(hand_error) = rows
        .iter()
        .map(HandListing::from_row)
        .collect::<Result<Vec<_>, _>>()
    else {
        panic!("a negative weight is refused")
    };
    let Err(shelf_error) = shelf::fetch_all::<Listing>(&client, SELECT, &[]).await else {
        panic!("a negative weight is refused")
    };
    println!("by hand: {hand_error}");
    println!("shelf:   {shelf_error}");
    assert!(matches!(
        shelf_error,
        shelf::Error::Mapping {
            field: "weight",
            ..
        }
    ));
    Ok(())
}
```

```
by hand: number too small to fit in target type
shelf:   Listing.weight: column "weight": number too small to fit in target type
```

The message after shelf's colon is the conversion's own, from the standard library's `TryFromIntError`, captured rather than remembered: shelf's tests once expected a different wording, which an older Rust had used (PLAN.md §12 #10).

	by hand	shelf	sqlx
Same	The column's name, the nested struct, the JSON wrapper and the conversion, each stated once per field	The same, as attributes on the fields	The same, as attributes with the same names (`rename_all`, `rename`, `flatten`, `json`, `try_from`, `skip`)
Different	Any logic at all, in the function body; an error is whatever the failing call returned, without the field or the column	Unknown keys, duplicates and contradictions are compile errors at the token, all in one build; a failed conversion names the struct, the field and the column; `try_from` takes a type or a quoted type	`try_from` takes only a quoted type; `json(nullable)` and a `default` key exist; it has no branch for an unknown field key, and `unwraps` the result of parsing field attributes, so a mistake there stops the derive with a panic (§3.7; sqlx-macros-core 0.9.0's source)

In shelf

Parsing and generating meet at one enum. Once a field's attributes are known to be consistent, `read` reduces them to the single way the field is read, and the generator matches on that (§4.3) instead of on the combination of keys:

```rust
// orm/shelf-codegen/src/attrs.rs (excerpt)
/// How a field is read, once its attributes are known to be consistent.
pub enum Read<'a> {
    /// From one column, by its own `FromSql`.
    Column,
    /// From one column, through `Json<T>`.
    Json,
    /// From one column as the given type, then `TryFrom`.
    TryFrom(&'a Type),
    /// As a nested `FromRow`, from the same row.
    Flatten,
    /// Not at all: `Default::default()`.
    Skip,
}
```

orm/shelf-codegen/src/attrs.rs

`FieldAttrs::read` returns the first that applies, in the order `skip`, `flatten`, `json`, `try_from`, and `Column` otherwise; after `check_combinations`, at most one of them can. `rename` is not part of `Read`, because it changes the column's name, not how its value is read.

Adding an attribute to shelf touches these places, which is also a map of the chapter:

1. **`attrs.rs`**: the key in `FieldAttrs` (or `ContainerAttrs`), a branch in `parse`, and any combination it cannot join in `check_combinations`.
2. **`from_row.rs`**: what the field generates, usually a new `Read` arm, with the bound `add_if_generic` should add for a generic field (`generics.rs`).
3. **An expansion test**, `orm/shelf-codegen/tests/expand/NAME.rs`, which pins down the code it generates (Chapter 13).
4. **An integration test** in `orm/shelf/tests/from_row.rs`, against PostgreSQL.
5. **A UI test** for each error it can report, in `orm/shelf/tests/ui/`.
6. **The attribute table** in the documentation of `shelf::FromRow` (orm/shelf/src/lib.rs).

When not to use an attribute

- **When the SQL can do it.** `SELECT sku_code AS sku` needs no `rename`, and keeps the mapping where the query is.
- **When only one query needs it.** An attribute applies to every query that reads the struct: a `rename` for one report's alias binds the struct to that name everywhere. Alias in that query instead.
- **When the conversion is business logic.** `try_from` is for conversions that only check and wrap, such as a non-negative amount or a validated code. If the conversion needs other fields, or a lookup, read the raw values and build the type in a constructor.

Summary

- A derive declares its helper attributes, `attributes(shelf)`. Without that, or without the `#[derive]` on the struct, the compiler refuses `#[shelf(…)]` before any macro runs, with a note that `shelf` is a crate.
- `Attribute::parse_nested_meta` walks the conventional `key`, `key = value`, `key(…)` syntax; the closure reads each key's value with `meta.value()?` and a parse of the right type (`LitStr`, `Type`), or nothing for a flag.
- shelf's parser keeps each value with the span of its key (`Keyed`), refuses duplicates, flags with values and unknown keys (listing the known ones), and accepts a type quoted or not.
- `rename_all`, `rename`, `skip`, `flatten`, `json` and `try_from` each change one element of `Columns`, one call in `resolve` and one read in `from_row_at`. `#[shelf(json)]` on an `Option<T>` decodes a JSON `null`, but not an SQL `NULL`.

- Errors are collected with `syn::Error::combine` and reported together. `parse_nested_meta` stops at the first error in an attribute, so each attribute reports at most one parse error; combinations that cannot go together are refused at the key that does not fit.
- trybuild's UI tests pin down the compiler's output, normalized; regenerate them with `TRYBUILD=overwrite` and read the diff. Reading them gave shelf `FromColumn`, whose `#[diagnostic::on_unimplemented]` (Rust 1.78+) puts shelf's words on a type error.
- A generic struct gets one bound per field whose type mentions a type parameter, chosen by how the field is read; concrete types get none, so their errors keep shelf's message. Lifetime parameters are refused.
- darling would parse the same attributes from a declared struct, with better errors for free; shelf parses by hand to show the mechanism.

Exercises

1. **A default for a missing column.** Design `#[shelf(default)]`: read the column if the result has it, and use `Default::default()` if it does not (sqlx has the same key). Write, by hand, the `impl FromRow` the derive should generate for a struct with one such field, and check it against a query with the column and one without. *Hint:* `resolve` cannot return an index for a column that is not there; its element of `Columns` can be an `Option<usize>`.
2. **Two fields, one column.** Write a check, as a stage function on a `DeriveInput`, that refuses two fields that read the same column, and reports the second at its field. Then decide whether shelf should have it. *Hint:* collect each column name in a `HashMap` with the field that claimed it first, and combine the errors.
3. **Borrowing from the row.** Suppose `FromRow` became `FromRow<'r>`, with `fn from_row(row: &'r Row) -> Result<Self, Error>`, so that a field could be a `&'r str`. What goes wrong in `fetch_all`? *Hint:* who owns the rows, and when are they dropped?

Solutions

1. The field's element of `Columns` becomes `Option<usize>`: `resolve` records whether the column is there, and `from_row_at` reads it or falls back. This is the impl the derive should generate, written by hand without the type checks that `column` would add:

```
use shelf::{FromRow, Mapping};
use tokio_postgres::{Column, Row};

/// A product whose `note` column only some queries select: `#[shelf(default)]` on `note`.
#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    note: String,
}

impl FromRow for Product {
    /// `None` when the result has no `note` column.
    type Columns = (usize, Option<usize>);

    fn resolve(columns: &[Column]) -> Result<Self::Columns, shelf::Error> {
        let find = |name: &str| columns.iter().position(|c| c.name() == name);
        let sku = find("sku").ok_or_else(|| shelf::Error::Mapping {
            model: "Product",
            field: "sku",
            problem: Mapping::MissingColumn {
                column: "sku".into(),
            },
        })?;
        Ok((sku, find("note")))
    }

    fn from_row_at(row: &Row, columns: &Self::Columns) -> Result<Self, shelf::Error> {
```

```
        Ok(Product {
            sku: row.try_get(columns.0)?,
            note: match columns.1 {
                Some(at) => row.try_get(at)?,
                None => Default::default(),
            },
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), shelf::Error> {
    let client = shelf_playground::connect().await?;

    let with_note: Product =
        shelf::fetch_one(&client, "SELECT 'MUG-01' AS sku, 'fragile' AS note", &[]).await?;
    assert_eq!(with_note.note, "fragile");

    let without: Product = shelf::fetch_one(&client, "SELECT 'MUG-01' AS sku", &[]).await?;
    assert_eq!(without.note, "");
    Ok(())
}
```

In the derive, `default` is a flag in `FieldAttrs`, orthogonal to `Read`, like `rename`: a defaulted field can still be read through `json` or `try_from`, so it should not become a `Read` arm of its own. It cannot go with `skip` or `flatten`. The generator then wraps the field's `resolve` call and its read, and `__private` needs a version of `column` that returns `Ok(None)` for a missing column but still checks the type of one that is present.

2. The check needs each field's column name, so it reads `rename` (only `rename`, for brevity: a full version would apply `rename_all` too):

```
use std::collections::HashMap;

use syn::{Data, DeriveInput, Error, Field, Fields, Ident, LitStr, parse_quote};

/// The column a field reads: its `rename`, or its own name.
fn column_of(field: &Field) -> syn::Result<String> {
    let mut column = field.ident.as_ref().expect("a named field").to_string();
    for attr in field.attrs.iter().filter(|a| a.path().is_ident("shelf")) {
        attr.parse_nested_meta(|meta| {
            if meta.path.is_ident("rename") {
                column = meta.value()?.parse::<LitStr>()?.value();
            }
            Ok(())
        })?;
    }
    Ok(column)
}

/// Refuses a second field that reads a column another field already reads.
fn check_unique(input: &DeriveInput) -> syn::Result<()> {
    let Data::Struct(data) = &input.data else {
        return Ok(());
    };
    let Fields::Named(fields) = &data.fields else {
        return Ok(());
    };
    let mut first_reader: HashMap<String, &Ident> = HashMap::new();
    let mut errors: Option<Error> = None;
    for field in &fields.named {
        let ident = field.ident.as_ref().expect("a named field");
        let column = column_of(field)?;
        if let Some(first) = first_reader.get(&column) {
            let error = Error::new_spanned(
                ident,
                format!("`{first}` already reads the column {column:?}"),
            );
```

```
            match &mut errors {
                Some(all) => all.combine(error),
                None => errors = Some(error),
            }
        } else {
            first_reader.insert(column, ident);
        }
    }
    errors.map_or(Ok(()), Err)
}

fn main() {
    let input: DeriveInput = parse_quote! {
        struct Product {
            sku: String,
            #[shelf(rename = "sku")]
            code: String,
            price_cents: i64,
        }
    };
    let error = check_unique(&input).expect_err("two fields read sku");
    println!("{error}");
    assert_eq!(error.to_string(), "`sku` already reads the column \"sku\"");

    let fine: DeriveInput = parse_quote! {
        struct Product { sku: String, #[shelf(rename = "sku_code")] code: String }
    };
    assert!(check_unique(&fine).is_ok());
}
```

shelf does not have this check, on purpose: reading one column twice is odd, but not wrong. A struct may want a `jsonb` column both as a typed value and as a raw `serde_json::Value`, and `resolve` finds the same index for both without complaint.

3. `fetch_all` runs the query, which gives it a `Vec<Row>` that it owns, and maps every row. Each value would borrow from its row, so the values could not outlive the vector, and the vector is dropped when `fetch_all` returns: the borrow checker refuses to return them. The caller would have to own the rows instead, calling the query and then the mapping, two steps every time, with the rows kept alive beside the values. Every helper built on `FromRow` (`fetch_one`, and later the model's `find` and `all`) would inherit the same lifetime and the same two steps. The main book shows where borrowing does pay: a loop over rows that only looks at each value briefly (main book §3.4). shelf maps into owned values, and leaves that loop to hand-written code.

CHAPTER 5

Checking a statement once, not every row

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part I, sixth chapter. Builds on Chapters 1 and 4, and on main book Chapter 4 (`prepare`, `Statement::columns`). shelf: `FromRow::{Columns, resolve, from_row_at}`, `from_rows` and `fetch_*` in row.rs and fetch.rs, and the benchmark bench_mapping.rs (all built, S1a). Crates: tokio-postgres 0.7.18. Findings: PLAN.md §12 #4, #8, #13. Plan: orm/PLAN.md §4.0, §7, Chapter 5.

Chapter 1 measured what a lookup by name costs, and Chapter 3's derive does one per field per row. Everything that lookup finds depends only on the statement, not on the row: the position of `sku` is the same in the first row and in the millionth. So shelf finds each field's column once, when it sees the statement, and then reads every row by position.

The same step is the right moment to check types. Instead of discovering on the first row that `price_cents` is `text` and the field is `i64`, shelf checks every field against the statement's columns before the query runs, and reports the problem with the struct's name, the field's and the column's. Because tokio-postgres prepares a statement anyway when you pass it a string, the check costs no extra round trip.

In this chapter you will learn:

- what a statement knows about its columns before any row exists
- how `resolve` finds each field's column and checks its type once per statement
- what the derive generates for this: a `Columns` tuple and `from_row_at`
- why a wrong mapping fails before the query runs, even an `INSERT … RETURNING`
- what it is worth, measured on 100,000 rows

5.1 What a lookup by name costs, again

Draft prose:

On 100,000 rows of twelve columns, reading a struct by name took 38.2–38.5 ms in a release build, and reading it by position 12.4–12.6 ms (PLAN.md §12 #13, Apple Silicon, PostgreSQL 18.6 on localhost). The decoding is the same in both; the difference is the searching. Checking each column's type on every row, which tokio-postgres does whether you ask by name or by position, made no measurable difference (§12 #8).

5.2 A statement knows its columns

Plan:

- `client.prepare(sql)` returns a `Statement` whose `columns()` give each column's name and type, before any row; a `Row` carries the same columns.
- `resolve(columns) -> Result<Columns, Error>`: each field's position, found by exact name (PostgreSQL folds unquoted names itself, so shelf does not retry ignoring case, unlike `Row::try_get`; §12 #4).

5.3 Types checked once

Plan:

- `FromSql::accepts(column.type_())`, called once per field per statement, under shelf's `FromColumn` bound.
- The errors: `Mapping::MissingColumn`, `Mapping::WrongType` (with the SQL type and the Rust type, shortened: `Option<String>`, not `core::option::Option<alloc::string::String>`), and, per row, `Mapping::Null` and `Mapping::Decode`.
- The proof that the check comes first: an `INSERT … RETURNING` whose result lacks a field's column fails, and the table still has the rows it had (from `tests/from_row.rs`).

5.4 What the derive generates

Plan:

- The expansion of a two-field struct, printed: `type Columns = (usize, usize)`, `resolve` calling `__private::column::<T>` per field, `from_row_at` calling `__private::get::<T>` with `columns.0`, `columns.1`.
- `flatten` puts the nested type's `Columns` inside the tuple; `skip` puts `()`.
- `from_row` stays, as a provided method that resolves and reads one row.

5.5 Measured

Draft prose:

shelf resolving once took 13.0–13.1 ms on the same 100,000 rows: about 4% over the hand-written code that reads by position, and a third of the time of reading by name. Resolving again for every row, which is what calling `from_row` in a loop does, took 38.2–38.4 ms, as much as the lookups by name it repeats (§12 #13). That is why `fetch_all` and `from_rows` resolve once, and why the chapter's advice is to use them.

Plan:

- The table printed by `cargo run -p shelf --release --example bench_mapping`, pasted as captured.

5.6 Where it is used

Plan:

- `fetch_all`, `fetch_one`, `fetch_opt`: prepare, resolve against the statement, query.
- `from_rows` for rows you already have; streams resolve once too (Chapter 12).

The programs

- `run`: `fetch_all` into a struct, and the same rows through `from_row` in a loop, with both times printed and a relative assertion (debug build; to measure).
- `run`: the `INSERT … RETURNING` that never runs, and the row count unchanged.
- `run`: a wrong type reported for a query that returns no rows.
- The benchmark, as a `bash` command and its table (release; not run by `book.py check`).

Side by side

Reading 100,000 rows: by hand, by position · shelf (`from_rows`) · sqlx, whose rows look up names in a map built per statement (check in sqlx-postgres's source).

In shelf

The trait from `row.rs`, `column` and `get` from `row.rs`, and `fetch_all` from `fetch.rs`.

Claims

- Measured: §12 #8 and #13 (release, Apple Silicon, PG 18.6); the debug ratio (to measure).
- Docs: `Statement::columns`, `Column::type_` (docs.rs, 0.7.18).
- Opinion: an error before the first row is worth more than the speed.

When not to resolve once

- A single row: `from_row` is the same work, and simpler to call.
- Rows from different statements: their `Columns` are not interchangeable.

Exercises

Plan:

1. Prepare a statement once, resolve once, and run it twice with different parameters. *Hint:* `Columns` belongs to the statement, not to the execution.
2. Measure with three columns instead of twelve. *Hint:* the search grows with both.
3. Make `resolve` refuse a result with two columns of the same name. *Hint:* a join's `id`.

> **Open questions.** (1) Whether a relative timing assertion is reliable in a debug build (to measure). (2) How sqlx finds columns by name (check). (3) Whether S1b's executor should cache `Columns` with each prepared statement, so that `resolve` runs once per connection rather than once per call; a design question for Chapter 7.

CHAPTER 6

Tables: the model derive

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, first chapter. Builds on Chapters 3–5, and on main book Chapter 2 (`quote_ident`), Chapter 4 (the "cached plan must not change result type" error) and Chapter 14 (the repository layer). shelf: `#[derive(Table)]` in `shelf-codegen/src/table.rs`, the `Table` trait and `Error::NotFound` (proposed, S1b step 3). Crates: syn 3.0.9, tokio-postgres 0.7.18. Decisions: PLAN.md §10 (SQL as literals generated at compile time). Plan: orm/PLAN.md §4.2, §4.3, §7, Chapter 6; §13, S1b step 3.

`FromRow` reads any query's rows, but you still write every query. Most of them are the same few: fetch one row by its key, fetch them all, count them. The struct already knows its columns; what it lacks is the table they come from and which of them is the key. Two attributes supply that, and with them the derive can write the everyday `SELECT`s itself.

It writes them at compile time. The macro knows every name, so it can assemble each statement as a string literal, with every identifier quoted, and leave nothing to build at run time. Values never go into those strings: they stay `$1` parameters, as the main book insisted from its second chapter. What the macro cannot put in a literal, because it lives in another type, is where `LazyLock` comes in.

In this chapter you will learn:

- what a table adds to a row type: a name, a key and a column list
- how `#[derive(Table)]` reads `table`, `schema`, `primary_key` and `generated`
- how a derive writes SQL at compile time, quoting identifiers so that a table called `order` works
- `find`, `find_opt`, `all` and `count`, and the SQL each one sends
- why shelf never writes `SELECT *`
- where const evaluation stops, and what `LazyLock` is for

6.1 What a table adds

Plan:

- A model is a `FromRow` struct plus a table: `TABLE`, `COLUMNS`, the key's type and a way to get it from a value (proposed names, settled in S1b).
- `Table` requires `FromRow` (a supertrait): a model is always readable.

6.2 `#[derive(Table)]`

Plan:

- `#[shelf(table = "products")]` on the struct (and `schema` when needed); `#[shelf(primary_key)]` on one field, or on several for a composite key read as a tuple; `#[shelf(generated)]` for columns the database fills (`uuidv7()`, `now()`, identity), which writes will leave out (Chapter 8).
- The new keys added to `attrs.rs`; `FromRow` ignores them, so the UI test that refuses `table` today changes (PLAN.md §13).

6.3 SQL at compile time

Plan:

- The derive builds `SELECT "id", "sku", … FROM "products" WHERE "id" = $1` as a literal and emits it as an associated `const` (proposed).
- Quoting at macro time: a column called `type` or a table called `order` works; the `quote_ident` of the main book's Chapter 2, run by the compiler.
- A **Pitfall** box: an identifier is quoted, a value is a parameter; nothing a user types at run time ever reaches the generated SQL.

6.4 Reading

Plan:

- `find(&db, &key)` (one row, else `Error::NotFound`), `find_opt`, `all`, `count`: each with its SQL printed and its round trips counted.
- An explicit column list, never `*`: a cached statement that selected `*` fails with 0A000 after `ALTER TABLE … ADD COLUMN` (root PLAN.md §12 #42), and `*` would hand `resolve` columns the struct does not have.

6.5 Metadata as consts, and where they stop

Plan:

- `TABLE` and `COLUMNS` as consts are free. A literal cannot include another type's const, though: `concat!` takes literals only, so SQL that needs a `Table`'s name from elsewhere (an `Insert` struct naming its model, Chapter 8) is built once in a `LazyLock<String>`.
- A **Rust refresher** on `LazyLock` (stable since Rust 1.80; check) and why it is not a `const`.

The programs

- `run` (proposed): a model's `TABLE` and `COLUMNS` printed; the generated SQL printed.
- `run`: `find`, `find_opt`, `all` and `count` against a temporary table.
- `run`: a table called `order` with a column called `type`.
- `run`: `find` on a missing key, matching `Error::NotFound`.
- `run`: the expansion of a small model, printed by running the generator.

Side by side

Fetching by primary key: by hand (`fetch_one` with the SQL written out) · shelf (`Product::find(&db, &id)`, proposed) · SeaORM (`Entity::find_by_id(id).one(&db)`, check).

The data

A temporary `products` table with an identity key; a second table with a composite key (`stock (store_id, product_id)`, as in stockroom).

Claims

- Docs: PostgreSQL's rules for identifiers and key words (the documentation's "Lexical Structure"); root PLAN.md §12 #42 for the cached-plan error.
- Opinion: an ORM should make the SQL it sends easy to predict; constants make it easy to print.

When not to use `Table`

- Reports, joins and aggregates: SQL and `fetch_all` with a `FromRow` struct.
- Views: a `FromRow` struct suffices, since there is no key to find by (or declare one; check whether that is useful).

Exercises

Plan:

1. Add `exists(&db, &key)`. *Hint:* `SELECT EXISTS (SELECT 1 FROM … WHERE …)`.
2. Support `schema` and print the qualified name. *Hint:* quote each part separately.
3. Find by a composite key. *Hint:* the key type is a tuple.

> **Open questions.** (1) The trait's exact items and names (`TABLE`, `COLUMNS`, `Key`, `key()`) are proposed; settle them in S1b. (2) How `find` takes a composite key: a tuple of references or of owned values. (3) `concat!` against `LazyLock` (S1b probe). (4) Whether `count` belongs here or in the query builder (Chapter 10).

CHAPTER 7

The executor and shelf's errors

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, second chapter. Builds on Chapter 6, and on main book Chapter 4 (round trips), Chapter 6 (`DbError`, `SqlState`), Chapter 8 (the sealed `GenericClient`, the retry helper) and Chapter 13 (deadpool's `prepare_cached`). shelf: the `Executor` trait in `shelf/src/executor.rs`, the `deadpool` feature, `Error::NotFound` and `Error::Stale` (proposed, S1b step 2); `fetch_*` move onto it. Crates: tokio-postgres 0.7.18, deadpool-postgres 0.14.2. Findings: root PLAN.md §12 #6, #33, #42. Plan: orm/PLAN.md §4.5, §7, Chapter 7; §13, S1b steps 1–2.

So far shelf's functions take tokio-postgres's `GenericClient`, which covers a `Client` and a `Transaction`. A service holds neither: it holds a deadpool connection, whose own `GenericClient` adds the one thing a model layer wants most, a statement cache. Both traits are sealed, so shelf cannot implement either for anything, and code generic over one does not accept the other.

The way out is a trait of shelf's own, with only the calls shelf needs, implemented for the four types a program actually has. Deciding what goes in it means deciding how each statement reaches the server, which is a question of round trips the main book taught you to measure. The chapter measures three strategies through lagproxy, picks one per executor, and gives shelf an error type that says what an application can do about each failure.

In this chapter you will learn:

- why neither `GenericClient` will do, and what a trait of shelf's own must contain
- how to write async methods in a trait whose futures are `Send`, and why that matters in axum
- three ways to send a statement (`prepare`, `prepare_cached`, `query_typed`) and their round trips, measured
- how shelf calls run inside a transaction, and with the main book's retry helper
- `shelf::Error`, and how an application maps it to HTTP statuses

7.1 Why not `GenericClient`

Plan:

- tokio-postgres's `GenericClient` is sealed; deadpool's is sealed too, and only it has `prepare_cached` (root CLAUDE.md, "Verified API facts").
- An attempt to implement one for a wrapper, and the captured error.

7.2 The `Executor` trait

Plan:

- The calls shelf needs: prepare (cached where possible), `query`, `query_one`, `query_opt`, `execute` (proposed list).
- Methods returning `impl Future<Output = …> + Send`, and a **Rust refresher** on why the `Send` must be written: a handler's future must be `Send`, and a trait method's future is not known to be unless the trait says so.
- Impls for `tokio_postgres::Client`, `tokio_postgres::Transaction<'_>` and, behind the default feature `deadpool`, `deadpool_postgres::Client` and its `Transaction<'_>`.
- `fetch_*` moved onto it; the tests unchanged.

7.3 Three ways to send a statement

Plan:

- `prepare` then execute: two round trips every time (Parse/Describe, then Bind/Execute).
- `prepare_cached` (deadpool): two the first time, one after; the cache is per connection (root PLAN.md §12 #42: about 1 ms the first time, about 3 µs after).
- `query_typed`: one round trip and no named statement, but the parameter types must be known, and `resolve` has no `Statement` to check against before the query.
- Measured through lagproxy at 5 ms each way (root PLAN.md §12 #33: a round trip cost 15.3 ms); the choice per executor recorded in PLAN.md §10.

7.4 Transactions

Plan:

- shelf calls on a `&Transaction` see the transaction's own writes.
- The main book's `in_transaction` retry helper with an `AsyncFnMut` closure that calls shelf (root PLAN.md §12 #6: the closure must own its data).

7.5 `shelf::Error`

Plan:

- The variants: `Mapping` and `Db` (built), `NotFound` and `Stale` (proposed), each with what the caller can do about it.
- Constraint names stay the application's: stockroom's `AppError` maps `SqlState` and constraint names to statuses; an `impl From<shelf::Error> for AppError` shows the pattern.

The programs

- A `~~~rust` fragment that tries to implement `GenericClient`, with its captured error.
- `run` (proposed): one repository function called with a client, a transaction and a pooled connection.
- `needs=lagproxy`: the three strategies timed; asserts that a cached statement beats prepare-every-time by at least a round trip (relative).
- `run`: the retry helper running shelf calls under `SERIALIZABLE`.
- `run`: a unique violation from shelf mapped to an application error.

Side by side

One repository function over a client, a transaction and a pool: by hand (generic over deadpool's `GenericClient`, as stockroom does) · shelf (`&impl Executor`, proposed) · sqlx (`impl Executor<'_, Database = Postgres>`, check).

Claims

- Measured: root PLAN.md §12 #33 (round trip through lagproxy), #42 (`prepare_cached` timings); shelf's three strategies (to measure).
- Docs: deadpool-postgres 0.14.2's `GenericClient` and `prepare_cached`.
- Opinion: an ORM's executor trait should be small enough to implement for a new client type in ten lines.

When not to use the executor

- Pipelining many independent queries on one connection (main book Chapter 7): use the `Client` directly, or `try_join!` shelf calls on one shared client (check that the executor's futures allow it).

Exercises

Plan:

1. Implement `Executor` for a wrapper that logs each statement's name and duration. *Hint:* delegate, and time around the await.
2. Add a statement cache for a plain `Client`. *Hint:* a wrapper that owns the client and a map.
3. Map `Error::Stale` to 412 and `NotFound` to 404 in an axum handler. *Hint:* main book Chapter 15.

> **Open questions.** (1) The `Send` futures inside axum handlers and the retry helper must be probed before the trait is fixed (S1b step 1). (2) What a plain `Client` does without a cache: prepare every time, or `query_typed`. (3) Where `resolve`'s result lives when statements are cached (Chapter 5's open question).

CHAPTER 8

Writing rows: insert, update, delete, upsert

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, third chapter. Builds on Chapters 6 and 7, and on main book Chapter 2 (`RETURNING`, `ON CONFLICT`), Chapter 6 (constraint errors) and Chapter 10 (`MERGE`). shelf: `#[derive(Insert)]` in `shelf-codegen/src/insert.rs`, `update`, `delete`, `delete_by_key` and upsert in `shelf/src/write.rs` (proposed, S1b step 4). Crates: tokio-postgres 0.7.18. PostgreSQL: `uuidv7()` (PG 18), `MERGE … RETURNING` (PG 17). Plan: orm/PLAN.md §4.2–§4.4, §7, Chapter 8; §13, S1b step 4.

Reading is half of a model layer. The other half is writing, and the first question is what type you write. A `Product` read from the database has an `id` and a `created_at`, but a product you are about to create has neither: the database will assign them. Making those fields `Option`s would spread `unwrap()`s through every reader of the model. shelf does what stockroom already does by hand, and gives creation its own struct, `NewProduct`, with only the columns you supply.

`#[derive(Insert)]` on that struct writes the `INSERT`, and `RETURNING` hands back the full model, defaults filled in, in the same round trip. Updates and deletes go through the model itself, by its key, and an upsert is one more clause. Each method's SQL is printed, and each one says what happens when a constraint refuses it.

In this chapter you will learn:

- why creation gets its own struct, and how `#[derive(Insert)]` names its model
- how `RETURNING` gives back the whole model, with the database's defaults, in one round trip
- `update`, `delete` and `delete_by_key`, and what zero rows affected means
- upserts with `ON CONFLICT`, and when `MERGE` is the better statement
- what constraint violations from shelf's writes look like, and where to map them

8.1 `#[derive(Insert)]`

Plan:

- `#[shelf(model = Product)] struct NewProduct { sku, name, price_cents, tags }`: the table and the `RETURNING` list come from the model.
- The SQL: `INSERT INTO "products" ("sku", …) VALUES ($1, …) RETURNING "id", …`, built once (`LazyLock`, Chapter 6) because the model's names are not literals here.
- `new_product.insert(&db) -> Result<Product, Error>` (proposed).

8.2 Database defaults

Plan:

- `uuidv7()` (PG 18; verify on the server), `now()`, identity columns: the model marks them `generated`, the `New…` struct leaves them out.
- What happens if a `New…` struct names a `generated` column: a compile error from the derive (proposed), or the database's.

8.3 Update and delete

Plan:

- `product.update(&db)`: every column that is neither key nor generated, `WHERE` the key. Zero rows affected means the row is gone: `Error::NotFound`.
- `product.delete(&db)`, `Product::delete_by_key(&db, &key)`.
- A **SQL lens** box on what `UPDATE` of every column costs (a new row version either way).

8.4 Upsert

Plan:

- `ON CONFLICT ("sku") DO UPDATE SET … = excluded.…`: the conflict target named by an attribute or an argument (open question).
- `DO NOTHING` returns no row on conflict, so the method returns an `Option`.
- `MERGE … RETURNING merge_action()` (PG 17) as the statement for a batch with several outcomes; main book Chapter 10's import does it by hand.

8.5 Constraint errors

Plan:

- A duplicate `sku`: `Error::Db` carrying 23505 and the constraint's name; the application's mapping (Chapter 7) turns it into 409.
- A `CHECK` violation: 23514, 422.

The programs

- `run` (proposed): insert and read back, with the defaults filled.
- `run`: `uuidv7()` keys ordered by creation time.
- `run`: update, then delete, then update again (`NotFound`).
- `run`: an upsert twice, the second updating.
- `run`: a duplicate key's error, its constraint name printed.

Side by side

Inserting a row and reading it back: by hand (stockroom's `stores::create`) · shelf (`NewStore { … }.insert(&db)`, proposed) · SeaORM (`ActiveModel { … }.insert(&db)`, check).

In stockroom

`create` in stockroom/src/repo/stores.rs: the `New…` struct and the `INSERT … RETURNING` this chapter generates.

The data

A temporary `products` table with an identity key and a unique `sku`; a `uuid` key with `DEFAULT uuidv7()` for 8.2.

Claims

- Docs: PostgreSQL's `INSERT … ON CONFLICT`, `MERGE` (`RETURNING` since PG 17) and `uuidv7()` (PG 18); verify each on the 18.6 server before writing.
- Opinion: a separate struct for creation is clearer than `Option` fields on the model.

When not to use shelf's writes

- A guarded update such as stockroom's oversell check (`UPDATE … WHERE available >= $3`): the condition is the point, so write it.
- Bulk loads: Chapter 12.

Exercises

Plan:

1. Add `insert_returning_id` that returns only the key. *Hint:* a second `RETURNING` list.
2. Make `update` refuse a model whose key is a default value. *Hint:* there is no row to update.
3. Write the upsert with `MERGE` and compare. *Hint:* `merge_action()` tells you which branch ran.

> **Open questions.** (1) Naming the model (`model = Product`) and building the SQL with `LazyLock` (S1b probe). (2) How the upsert's conflict target is given: an attribute on the `New…` struct, or an argument. (3) Whether `update` writes every column or only changed ones (that is `Patch`, Chapter 9).

CHAPTER 9

Partial updates and optimistic locking

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part II, fourth chapter. Builds on Chapter 8, and on main book Chapter 4 (statements per shape), Chapter 8 (§8.8, optimistic concurrency) and Chapter 15 (ETag and `If-Match`). shelf: `#[derive(Patch)]` in `shelf-codegen/src/patch.rs`, the `version` and `updated_at` attributes, `Error::Stale` (proposed, S1b step 5). Crates: tokio-postgres 0.7.18, serde. Findings: root PLAN.md §12 #1 (domain-typed parameters). Plan: orm/PLAN.md §7, Chapter 9; §13, S1b steps 1 and 5.

An admin edits a product's price and nothing else. The request body is `{"price_cents": 990}`, and the update should touch that column only; another admin's change to the name, made a second earlier, must survive. Chapter 8's `update` writes every column, so it would undo that change. A partial update needs to know which fields were sent, and JSON has a third state that `Option` cannot express: a field that was sent as `null`, which means "set it to NULL", is not a field that was left out.

Then there is the edit that arrives late. Two tabs load version 7 of the product; both save. Without a check, the second save silently overwrites the first. With a version column, the second `UPDATE` finds no row with version 7, and the application can say so. This chapter builds both: `#[derive(Patch)]` in one of three designs, chosen by measurement, and a `version` attribute that turns a lost update into an error.

In this chapter you will learn:

- how to tell "absent" from "null" in a request body, with `Option<Option<T>>` and serde
- three ways to write a partial `UPDATE`, and what each costs in statements and round trips
- how `#[derive(Patch)]` generates the one shelf chose
- how a version column turns a lost update into `Error::Stale`, and how to tell it from `NotFound`
- what `updated_at = now()` records (the transaction's start, not the statement's)
- how ETags and `If-Match` fit on top, in axum

9.1 Absent or null

Plan:

- `Option<T>` for a non-nullable column (absent or a value), `Option<Option<T>>` for a nullable one (absent, null, or a value).
- serde: deserializing the double `Option` needs a helper (a `deserialize_with` or a small crate; check what is current).

9.2 Three designs

Draft prose:

There are three ways to send a partial update, and each gives something up. The first builds the `SET` list at run time from the fields that are present: the SQL is exactly what you would write by hand, but every combination of fields is a different statement, prepared and cached separately. The second sends every column, and writes `col = coalesce($n, col)`: one statement for every patch, but `coalesce` cannot tell a NULL you want from a value you did not send, so it can never set a column to NULL. The third sends a flag beside each value, `col = CASE WHEN $f THEN $v ELSE col END`: one statement, NULLs included, at the price of twice the parameters.

Plan:

- Each design run against the same table; prepared statements counted in `pg_prepared_statements`, round trips through lagproxy (to measure).
- The choice recorded in PLAN.md §10.
- The catch for design (c): an unset field still sends a typed value; check that a `None` of the field's type serializes for every column type, domains included (root PLAN.md §12 #1).

9.3 Version columns

Plan:

- `#[shelf(version)]` on an `i32` field: `WHERE "id" = $1 AND "version" = $2`, and `"version" = "version" + 1` in the `SET` list.
- Zero rows: either the row is gone (`NotFound`) or the version moved on (`Stale`). Telling them apart in one round trip with a CTE that also reports whether the row exists (S1b probe).
- The main book's §8.8 did this by hand; stockroom's `PATCH /products/{sku}` does it with `coalesce($2, version)`.

9.4 `updated_at`

Plan:

- `#[shelf(updated_at)]` writes `"updated_at" = now()`.
- A **SQL lens** box: `now()` is the transaction's start time; two updates in one transaction record the same instant (`clock_timestamp()` does not).

9.5 ETags in axum

Plan:

- The version as the ETag; `If-Match` into the patch's expected version; `Stale` ⟶ 412 with the current version, as stockroom does.
- A handler driven with `oneshot`.

The programs

- `run` (proposed): absent, null and a value deserialized into a patch struct.
- `run`: the three designs against one table, with the prepared statements counted.
- `run`: two tabs on one row, the second save getting `Error::Stale`.
- `run`: `updated_at` twice in one transaction, the same instant.
- `run`: an axum handler with `If-Match`, through `oneshot` (412 on a stale version).

Side by side

An edit from two admin tabs: by hand (stockroom's `PATCH /products/{sku}`) · shelf (`ProductPatch { … }.apply(&db, &id, version)`, proposed) · SeaORM (`ActiveModel` with `Set` and `NotSet`, check).

In stockroom

The `update` in stockroom/src/repo/products.rs: `coalesce` for the fields and for the version.

Claims

- Measured (to measure): statements prepared per design, round trips per patch.
- Docs: PostgreSQL's date/time functions (`now()` against `clock_timestamp()`).
- Opinion: design (c) is the right default for a model layer; (a) when the SQL must look hand-written.

When not to use `Patch`

- A change that is a business operation (an order's status transition, stockroom's `POST /orders/{id}/transition`): write a function with its own rules.
- Counters (`stock = stock - 1`): an expression, not a value from the client.

Exercises

Plan:

1. Make a patch with no fields a no-op that sends nothing. *Hint:* check before the round trip.
2. Return the changed columns' names along with the model. *Hint:* the flags already say which.
3. Add `#[shelf(version)]` on a `bigint`. *Hint:* the bound on the field's type.

> **Open questions.** (1) Which design, by measurement (S1b step 5). (2) Telling `Stale` from `NotFound` in one round trip (S1b probe). (3) The serde helper for `Option<Option<T>>` (check which crate or function is current).

CHAPTER 10

A typed query builder

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, first chapter. Builds on Chapters 6 and 7, and on main book Chapter 2 (parameters, injection), Chapter 3 (`= ANY($1)` with arrays) and Chapter 9 (keyset pagination). shelf: `Column<M, T>` constants generated by `Table`, and the query module `shelf/src/query/` (proposed, S1c). Crates: tokio-postgres 0.7.18. Plan: orm/PLAN.md §4.4, §7, Chapter 10.

`find` and `all` cover two queries. The third most common is "these rows, filtered, sorted, a page at a time", and its filter changes with the request: a tag if the client sent one, a price range if it sent that, both or neither. Writing that SQL by hand means assembling a string and a parameter list in step, numbering each `$n` correctly, which is exactly the code where injection bugs and off-by-one parameter errors live.

A query builder assembles it for you, from typed pieces. The derive already knows every column and its Rust type, so it can generate a constant per column, `Product::PRICE_CENTS`, that knows both. A filter built from it, `Product::PRICE_CENTS.gt(1_000)`, can only take an `i64`, and a mistake is a compile error at the call. The builder numbers the parameters, keeps the values out of the SQL, and stays small enough to read in one sitting.

In this chapter you will learn:

- what a query builder should type (columns and values) and what it should leave to SQL
- how `Table` generates a `Column<M, T>` constant per field, and what `PhantomData` does there
- how expressions become SQL with numbered parameters and an owned parameter list
- ordering, limits and keyset pages
- which mistakes fail to compile, shown with UI tests
- the escape hatch: raw SQL, inside a filter or instead of the builder

10.1 What to type

Plan:

- Typed: the column (it belongs to this model) and the value (it has the column's type).
- Left to SQL: functions, joins, subqueries; the builder accepts a raw fragment with its parameters instead of modelling them.

10.2 Column constants

Plan:

- `Product::PRICE_CENTS: Column<Product, i64>` (proposed names), generated from each field; the quoted column name inside.

- A **Rust refresher** on `PhantomData<(M, T)>`: a zero-sized marker that makes the type carry the model and the value type.

10.3 Expressions

Plan:

- `eq`, `ne`, `lt`, `le`, `gt`, `ge`, `between`, `like`, `is_null`, `any` (`= ANY($n)` with a `Vec`), `contains` (`@>` for arrays); `and`, `or`, `not`.
- An expression tree rendered once into SQL text and a parameter list, numbering `$1…$n` as it goes; the rendered SQL printed in the chapter.

10.4 Owned parameters

Plan:

- The query owns its values (`Box<dyn ToSql + Sync + Send>`, proposed), so it can be built in one function and run in another, across an await.
- The cost: one allocation per parameter; nothing compared with a round trip.

10.5 Order, limit and pages

Plan:

- `order_by(Product::SKU.asc())`, `limit`, `offset`; `count` and `exists` on the same filter.
- Keyset pagination with `after(Product::SKU, last)` (main book Chapter 9's recommendation), and why `offset` gets slower page by page.

10.6 What fails to compile

Plan:

- A wrong value type (`Product::PRICE_CENTS.eq("x")`), a column of another model in a `Product` query: UI tests and their captured errors.
- What still fails at run time: a raw fragment, and a column whose type changed in the database (Chapter 13).

10.7 The escape hatch

Plan:

- `fetch_all::<T>` with hand-written SQL, as always.
- A raw fragment inside a filter, with its own parameters renumbered by the builder.

The programs

- `run` (proposed): a filtered, sorted page against a temporary table.
- `run`: the rendered SQL and its parameters printed for three filters.
- `run`: keyset pages until the end.
- UI tests excerpted: the wrong value type, the wrong model.
- `run`: a raw fragment inside a filter.

Side by side

A filtered, sorted page: by hand (stockroom's keyset `page`) · shelf (`Product::query().filter(…).order_by(…).limit(20).all(&db)`, proposed) · diesel (`products.filter(price_cents.gt(1000)).order(sku.asc()).limit(20)`, check).

In stockroom

`page` in stockroom/src/repo/products.rs: the optional tag filter written as `($2::text IS NULL OR tags @> ARRAY[$2])`, which a builder replaces with a filter that is simply absent.

Claims

- Opinion: a query builder earns its place for optional filters; for fixed queries, plain SQL is clearer.
- Docs: PostgreSQL's array operators (`@>`, `= ANY`).

When not to use the builder

- Reports, aggregates, CTEs, window functions: SQL with a `FromRow` struct.
- A fixed query run often: a hand-written statement caches as one statement; each filter shape is a different statement (the open question below).

Exercises

Plan:

1. Add `ilike`. *Hint:* one more operator in the renderer.
2. Add `in_range` for `Option` bounds, skipping a missing bound. *Hint:* build the expression conditionally.
3. Render a query and run it through `fetch_all` yourself. *Hint:* the builder's output is SQL and parameters.

> **Open questions.** (1) Ergonomics: `&str` for a `String` column and an `i32` literal for an `i64` column; probe the trait design before writing (PLAN.md §11). (2) Statement caching with dynamic SQL: each filter shape becomes a statement. (3) Whether `count` lives here or on `Table` (Chapter 6).

CHAPTER 11

Relations without N+1

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, second chapter. Builds on Chapters 6, 7 and 10, and on main book Chapter 3 (arrays as parameters), Chapter 7 (pipelining) and lagproxy (apps/lagproxy). shelf: the `belongs_to` and `has_many` attributes, their generated methods, and `shelf/src/relations.rs` (proposed, S1c). Crates: tokio-postgres 0.7.18. Findings: root PLAN.md §12 #21, #33. Plan: orm/PLAN.md §4.3, §7, Chapter 11.

An order has lines. Showing a page of fifty orders with their lines is the textbook case where an ORM is either a great help or a disaster. The disaster is the obvious loop: fetch the orders, then, for each one, fetch its lines. That is fifty-one queries, and over a network each one costs a round trip, so the page that took a millisecond on localhost takes a second in production. It is called the N+1 problem, and the cure is older than ORMs: fetch all the lines for all fifty orders in one query, and sort them into their orders in Rust.

shelf makes the cure the easy path. A `has_many` attribute on `Order` generates a loader that takes a slice of orders and returns their lines grouped by order, in one query with `order_id = ANY($1)`. The chapter measures the loop and the loader through lagproxy, then looks at the two other answers, a join and pipelining, and explains why shelf has no lazy loading at all.

In this chapter you will learn:

- how `belongs_to` and `has_many` declare a relation, and the methods they generate
- what the N+1 problem costs, measured through lagproxy
- how batched loading with `= ANY($1)` fetches every child in one round trip
- how to pipeline independent loads on one connection
- when a join is the better query, and what it costs in duplicated rows
- why shelf has no lazy loading

11.1 Declaring relations

Plan:

- `#[shelf(has_many(OrderLine, key = order_id, as = lines))]` on `Order`; `#[shelf(belongs_to(Order, key = order_id))]` on `OrderLine` (proposed syntax; PLAN.md §4.3). Names are explicit: no pluralizing.
- Generated (proposed): `order.lines(&db)`, `Order::load_lines(&db, &orders)`, `line.order(&db)`.

11.2 N+1, measured

Draft prose:

Through lagproxy at 5 ms each way, one round trip cost 15.3 ms on this machine (root PLAN.md §12 #33). Fifty orders and their lines fetched one order at a time is fifty-one of them, three quarters of a second before

the server has done any real work, and the time grows with the page size. The loop is not slow because PostgreSQL is slow; it is slow because it waits for the network fifty-one times.

Plan:

- The loop and its time through lagproxy (to measure); the server's own time from `EXPLAIN ANALYZE` beside it, to show where the time goes.

11.3 Batched loading

Plan:

- One query, `WHERE order_id = ANY($1)` with the orders' keys as an array, then a `HashMap` from key to children; the loader returns the children in the parents' order.
- Measured through lagproxy: two round trips for the page, whatever its size (to measure).
- The foreign key type needs `Hash + Eq`; a composite key needs a tuple (open question).

11.4 Pipelining independent loads

Plan:

- Lines and the store for a page of orders: two loaders, independent, joined with `try_join!` on one connection, costing about one round trip (main book Chapter 7; §12 #21: one pipelined client beat a pool over latency).
- Which executors allow it: a shared `Client`, not a `Transaction` held mutably (check with the executor of Chapter 7).

11.5 Joins

Plan:

- A join returns each order once per line: duplicated parent columns, and a `FromRow` with `flatten` to read them.
- `json_agg` or `array_agg` of a composite to return one row per order; when that wins (a small number of children, one round trip).

11.6 Why no lazy loading

Plan:

- Lazy loading needs a hidden connection in the model and a hidden round trip in a field access; in async Rust it would also need `.await` on a getter. shelf makes every round trip visible instead (opinion, argued).

The programs

- `needs=lagproxy` (proposed): the N+1 loop timed.
- `needs=lagproxy`: the batched loader timed; asserts it is at least ten times faster than the loop for fifty orders (relative; to measure).
- `needs=lagproxy`: two loaders pipelined with `try_join!`.
- `run`: a join with `flatten`, and `json_agg` into `Json<Vec<…>>`.

Side by side

Orders with their lines: by hand (stockroom's `orders::get`, two pipelined queries) · shelf (`Order::load_lines`, proposed) · SeaORM (`find_with_related`, check).

The data

Temporary `orders` and `order_lines` tables with fifty orders and a few lines each; stockroom's schema in the capstone.

Claims

- Measured: root PLAN.md §12 #33 (a round trip through lagproxy), #21 (pipelining against a pool); N+1 against batched (to measure).
- Opinion: no lazy loading; every round trip should be visible at the call site.

When not to use relation loaders

- A report across several tables: one SQL query.
- A deep tree (categories of categories): a recursive CTE.

Exercises

Plan:

1. Load lines and products for a page of orders, two levels. *Hint:* the second loader takes the first one's children.
2. Return lines ordered by a column, inside each group. *Hint:* order in SQL, group in Rust.
3. Write the `json_agg` version as a `FromRow` struct. *Hint:* `#[shelf(json)]`.

> **Open questions.** (1) The names and signatures of the generated methods, and whether the loader returns a `HashMap` or a `Vec` of pairs. (2) Composite foreign keys. (3) Whether loaders can be pipelined through the `Executor` trait as designed in Chapter 7 (check).

CHAPTER 12

Bulk writes and streams

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, third chapter. Builds on Chapters 5, 7 and 8, and on main book Chapter 5 (domains and arrays), Chapter 9 (`RowStream`, backpressure, `pin!`) and Chapter 10 (COPY, binary COPY, pgload). shelf: `insert_many` (`unnest`), a binary COPY writer generated by the derive, and streams of models, in `shelf/src/bulk.rs` and `shelf/src/stream.rs` (proposed, S1c). Crates: tokio-postgres 0.7.18, futures-util. Findings: root PLAN.md §12 #1, #4, #22, #34–#37. Plan: orm/PLAN.md §7, Chapter 12.

Inserting ten thousand rows one `insert` at a time is the N+1 problem again, in the other direction: ten thousand round trips. The main book measured it with pgload: one `INSERT` per row was about a hundred times slower than binary COPY on localhost, and pipelining or `unnest` batches recovered most of the difference (root PLAN.md §12 #22). A model layer that offers only `insert` pushes every bulk load into that slow path.

shelf adds two bulk writes, both generated from the `New…` struct the reader already has: `insert_many`, one `INSERT … SELECT * FROM unnest(…)` with an array per column, and a binary COPY writer for the largest loads. Then it turns to the other large direction, reading, where a `Vec` of a million models is a million allocations held at once, and gives queries a stream of models that resolves its columns once and respects backpressure.

In this chapter you will learn:

- how `insert_many` sends a batch as one array per column through `unnest`
- why arrays of domain-typed columns need casts, and how shelf adds them
- how the derive writes a binary COPY row, and which types it must declare
- what each bulk strategy costs, measured against pgload's table
- how to stream models from a `RowStream`, resolving columns once
- how a COPY rejected at its start can kill a connection in 0.7.18, and how shelf avoids it

12.1 `insert_many`

Plan:

- One statement, `INSERT INTO "t" ("a", "b") SELECT * FROM unnest($1::text[], $2::bigint[])`, with one `Vec` per column built from the slice of `New…` values.
- In the select list, `unnest` keeps a composite whole; in `FROM` it expands it into fields (root PLAN.md §12 #4), which matters for composite-typed columns.
- Casts: an array of a domain needs the base type or a cast (root PLAN.md §12 #1); how the derive learns the SQL types (open question).

12.2 Binary COPY from the derive

Plan:

- The column types learnt from a prepared `SELECT "a", "b" FROM "t" LIMIT 0`, then a `BinaryCopyInWriter` fed one row per value through a generated method.
- The main book's warning: binary COPY trusts the declared types, and a wrong type of the same size loads wrong data silently (§12 #37).
- Pinned writer: scope it in a block; `drop` on a pinned reference drops nothing (§12 #34).

12.3 Measured

Plan:

- Ten thousand rows four ways through shelf: a loop of `insert`, pipelined `inserts`, `insert_many` in batches, binary COPY; times and ratios (to measure), set beside pgload's table (root PLAN.md §5.2; main book Chapter 10).
- Through lagproxy for the loop and the pipelined version, to show the round trips.

12.4 Streams of models

Plan:

- `query().stream(&db)` over `query_raw`: the columns resolved once from the first row (or the statement), each row decoded as it arrives.
- Backpressure: each request's response channel holds one message, so a slow consumer stalls the whole connection (main book Chapter 9); an unfinished stream stalls it too (§12 #35).
- Who owns the connection while the stream lives (root PLAN.md §12 #13), and whether shelf's stream should own a pooled one.

12.5 A COPY rejected at its start

Plan:

- In 0.7.18 a `COPY … FROM STDIN` that the server rejects before it starts (a missing table or column) leaves the connection unusable: rust-postgres issue #789 (root PLAN.md §12 #36).
- shelf's guard: the prepared `SELECT … LIMIT 0` of 12.2 fails first, cleanly, for a missing table or column.

The programs

- `run` (proposed): `insert_many` of a thousand rows, counted back.
- `run`: binary COPY of the same rows, with the types learnt from the table.
- `run`: four strategies timed on ten thousand rows; asserts COPY is faster than the loop by at least ten times (relative; to measure in debug).
- `run`: a stream of a hundred thousand models, counted without holding them.
- `run`: a missing column caught by the guard, and the connection still usable.

Side by side

Loading ten thousand rows: by hand (pgload's `unnest` batches) · shelf (`insert_many`, proposed) · sqlx (`QueryBuilder::push_values`, or `copy_in_raw`; check).

In stockroom

`stockroom seed` writes products and the ledger with binary COPY by hand (stockroom/src/cli/seed.rs).

Claims

- Measured: root PLAN.md §12 #22 (pgload); shelf's four strategies (to measure).
- Docs: rust-postgres issue #789; PostgreSQL's `COPY`.
- Opinion: offer `insert_many` by default, COPY when the batch is large and trusted.

When not to use shelf's bulk writes

- A client's CSV: COPY text through a staging table (main book Chapter 16), with errors reported by line.
- A load that needs `ON CONFLICT` per row: `insert_many` with a conflict clause, or `MERGE`.

Exercises

Plan:

1. Split `insert_many` into batches of a given size. *Hint:* chunks of the slice.
2. Make the stream report its row count at the end. *Hint:* `rows_affected()` once exhausted.
3. Load through COPY into a temporary table, then upsert. *Hint:* main book §10.6.

> **Open questions.** (1) How the derive learns the SQL types for `unnest` casts: from a prepared statement at run time, or from an attribute. (2) Whether shelf's stream owns its connection (root PLAN.md §12 #13). (3) The debug-build ratio for a relative timing assertion (to measure).

CHAPTER 13

Trusting the mapping: schema checks and tests

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fourth chapter. Builds on Chapters 4 and 5, and on main book Chapter 4 (`Statement::columns`) and Chapter 17 (testing against a real database). shelf: `shelf::verify` and `Error::Schema` in `shelf/src/verify.rs` (proposed, S1c); the test suites (built, S1a): tests/expand.rs, tests/ui.rs, tests/from_row.rs and the harness tests/common/mod.rs. Crates: tokio-postgres 0.7.18, trybuild 1.0.122. Findings: PLAN.md §12 #6, #11, #12. Plan: orm/PLAN.md §6, §7, Chapter 13.

A struct and a table are two descriptions of the same data, kept in different places by different people. A migration renames a column; a column becomes nullable; a `bigint` becomes `numeric`. Chapter 5's check catches each of those, but only when the query runs, which may be the first request after a deploy, or a nightly job that runs at three in the morning.

This chapter moves the check earlier. `shelf::verify` prepares each model's own `SELECT` at startup, compares its columns with the struct's fields, asks the catalog which columns may be NULL, and reports every mismatch at once. Then it turns to the other side of trust, the macros themselves, and shows the three kinds of test shelf has used since phase S1a: expansion tests, UI tests and tests against the database, with what each one catches and what it cannot.

In this chapter you will learn:

- what can drift between a struct and its table, and when each kind of drift is caught
- how `verify` uses `Statement::columns()` and the catalog to check a model before it is used
- where to run it: at startup, and in a test per model
- how to test a derive: expansion tests, UI tests, integration tests
- what compile-time checking (sqlx's `query!`, Cornucopia) adds, and what it costs

13.1 What can drift

Plan:

- A renamed or dropped column, a changed type, a column made nullable, a new `NOT NULL` column a `New…` struct lacks: each with when it is caught today (Chapter 5: at the query; a NULL: at the row).

13.2 `verify`

Plan:

- Prepare the model's `SELECT` (no rows read): `resolve` already checks names and types.
- Nullability is not in a row description. `Column::table_oid()` and `column_id()` (built into 0.7.18; §12 #6) identify the table column, and `pg_attribute.attnotnull` says whether it can be NULL; a nullable column read into a non-`Option` field is reported.
- Every mismatch collected into `Error::Schema(Vec<Mismatch>)` (proposed).
- Computed columns and views have no table OID: `verify` says it cannot know.

13.3 Where to run it

Plan:

- At startup, before the server accepts requests, so a bad deploy fails its readiness check (main book Chapter 16).
- In a test per model against a migrated database (main book Chapter 17's template).

13.4 Testing a derive

Draft prose:

shelf tests its derives three ways, because each catches what the others cannot. An expansion test runs the generator on a struct and compares the pretty-printed code with a file: it catches any change to what users get, and its diff is the review. A UI test compiles a broken struct and compares the compiler's errors with a file: it pins down the message and where it points. An integration test derives for real and runs queries against PostgreSQL: it is the only one that proves the generated code does the right thing.

Plan:

- Excerpts: `tests/expand.rs` and one `.expanded.rs` file; `tests/ui.rs` and one `.stderr` file; one integration test with the harness.
- Blessing: `SHELF_BLESS=1` and `TRYBUILD=overwrite`, and why the diff must be read.

13.5 Compile-time checking

Plan:

- sqlx's `query!` checks SQL against a live database (or saved metadata in offline mode) at compile time (docs; check the current details); Cornucopia generates Rust from SQL (main book Chapter 14).
- What they buy (no drift reaches a build) and cost (a database at build time, or metadata to keep in sync); where `verify` sits between.

The programs

- `run` (proposed): `verify` on a model that matches its table.
- `run`: a renamed column, the report printed.
- `run`: a nullable column read into `String`, reported from the catalog.
- Excerpts of the test files named above, checked by `book.py excerpts`.

Side by side

Catching a renamed column: by hand (at the first query) · shelf (`verify` at startup, proposed) · sqlx (`query_as!` at compile time, check).

Claims

- Docs: PostgreSQL's `pg_attribute` catalog; sqlx's offline mode (check).
- Opinion: check the mapping at startup; a deploy that cannot run should fail before it takes traffic.

When not to use `verify`

- Queries written by hand: `verify` checks models' own SQL; a `FromRow` struct read from a custom query is checked when that query is prepared (Chapter 5).

Exercises

Plan:

1. Run `verify` for every model in one call. *Hint:* a list of functions, one per model.
2. Report extra columns the struct does not read. *Hint:* the statement has more columns than `resolve` used.
3. Write an expansion test for a struct of your own. *Hint:* add a file to `tests/expand/`.

> **Open questions.** (1) Views and computed columns: report "unknown" or skip. (2) The report format of `Error::Schema`. (3) Whether `verify` should also check `New…` structs against `NOT NULL` columns without defaults.

CHAPTER 14

Capstone: stockroom on shelf

> **Sketch, 7 November 2026.** A plan of this chapter, not its text; it will be replaced by the chapter. Part III, fifth chapter. Builds on every chapter of this book, and on main book Chapter 18 (stockroom end to end) and stockroom's repository layer (stockroom/src/repo/). shelf: `orm/stockroom-shelf`, a library and binary reusing the `stockroom` crate's migrations and domain types (proposed, S2). Crates: axum 0.8.9, deadpool-postgres 0.14.2. Plan: orm/PLAN.md §4.1, §7, Chapter 14; §9, phase S2.

stockroom's data layer is written by hand: about a dozen repository functions, each with its SQL, its parameters and its `TryFrom<&Row>`. It is the code this book set out to generate. This chapter rewrites part of it with shelf, behind the same axum routes, against the same schema, with the same domain types, and compares the two line for line.

The comparison is honest in both directions. shelf removes the mapping code and most of the everyday SQL, and the routes keep their behaviour; the chapter counts lines, statements and round trips to show it. It also leaves alone the code shelf should not touch: the guarded `UPDATE` that keeps orders from overselling, the `MERGE` import, the COPY export, the `LISTEN` connection. Knowing where an ORM stops is the last thing this book teaches.

In this chapter you will learn:

- how shelf's models fit a real schema: a domain, a composite, an enum, a `jsonb` document, a virtual column
- what a repository function looks like rewritten with shelf, beside the original
- how much code, how many statements and how many round trips the rewrite changes, measured
- where shelf steps aside, and how raw SQL with `FromRow` fills the gap
- what a production ORM would add next

14.1 The shape

Plan:

- `orm/stockroom-shelf`: the `stockroom` crate as a dependency for its migrations, its domain types (`Sku`, `Address`, `OrderStatus`, `Cents`) and its error mapping; a repository layer on shelf; the routes for stores, products and orders, built with the same router shape.
- Its tests: stockroom's expectations for the ported routes, through `oneshot`, against a database cloned from stockroom's template (main book Chapter 17).

14.2 Models for a real schema

Plan:

- `Store` with an `address` composite (postgres-types' derive, passed through); `Product` with a `sku` domain (stockroom's hand-written `accepts`), `tags text[]`, `attributes jsonb` and a `version`; `StockLevel` with the virtual column `available`, read-only (never written by `Insert` or `Patch`: `generated`).
- `Order` and `OrderLine` with `has_many`, and `OrderStatus` as a Postgres enum.

14.3 Line for line

Plan:

- Three functions side by side: `stores::create`, `products::update` (versioned patch), and `orders::get` (an order with its lines).
- Counted (to measure): lines of Rust, statements prepared, round trips per request.

14.4 Where shelf steps aside

Plan:

- The oversell-proof reservation: `UPDATE stock … WHERE available >= $3`, pipelined in product-id order (main book Chapter 8): kept as SQL, its result read with a `FromRow` struct.
- The CSV import's `MERGE` and the CSV export's `COPY`, the `LISTEN` connection, the outbox worker's `SKIP LOCKED`: unchanged.
- The pattern: shelf for the everyday reads and writes, hand-written SQL for the statements whose wording is the point.

14.5 What a production ORM would add

Plan:

- Migrations generated from models, a query builder with joins, lazy relations, multi-database support, change tracking: what each would cost shelf in simplicity, and which existing crate already does it (diesel, SeaORM, toasty).

The programs

- `run` (proposed): the ported router on a pool of one, driven with `oneshot`: create a store, create a product, patch it with `If-Match`, place an order, read it back.
- `orm/stockroom-shelf/tests/`: stockroom's route tests for the ported routes.

Side by side

This chapter is one long side by side: stockroom's hand-written repository functions against the shelf versions, with the same routes on top.

The data

stockroom's schema, migrated by stockroom's own migrations, seeded with `stockroom seed`.

Claims

- Measured (to measure): lines, statements and round trips per ported route.
- Opinion: an ORM should cover the boring 80% and get out of the way of the rest.

When not to use shelf

- Every statement in 14.4, and any statement whose `WHERE` clause carries a business rule.

Exercises

Plan:

1. Port `GET /stores/{code}/stock`. *Hint:* the virtual column is read-only.
2. Add a route that lists a store's open orders with their lines. *Hint:* a filter, then a batched loader.
3. Count the round trips of `POST /orders` in both versions. *Hint:* `pg_stat_statements`, or lagproxy's timings.

> **Open questions.** (1) Which routes to port: stores, products and orders' reads are the proposal. (2) Whether stockroom-shelf reuses stockroom's test harness as a dependency or copies its pattern. (3) Whether `AppError` comes from stockroom unchanged, with `From<shelf::Error>`.

Appendix — attribute reference, generated code and phrasebook

> **Sketch, 7 November 2026.** A plan of this appendix, not its text; it will be replaced by the appendix. It collects reference material from every chapter: the `#[shelf(…)]` attributes (Chapters 4, 6, 8–11), what each derive generates (from the expansion tests in `orm/shelf-codegen/tests/expand/`), a phrasebook from shelf to diesel, SeaORM and sqlx (Chapter 0's comparison and each chapter's side by side), a glossary, and further reading. Entries already known are drafted; the rest is planned. Plan: orm/PLAN.md §4.3, §7, Appendix.

Look things up here once the chapters have taught them: which attribute does what and where it may go, what a derive writes for a given struct, and how to say a shelf call in the crate you use at work.

A Attribute reference

Drafted for the attributes that exist (phase S1a); the rest are added as their phases build them.

On the struct, read by `FromRow`:

Attribute	Meaning	Chapter
`rename_all = "…"`	Column names follow a convention: `"snake_case"` (the default), `"camelCase"`, `"PascalCase"` or `"SCREAMING_SNAKE_CASE"`	4

On a field, read by `FromRow`:

Attribute	Meaning	Chapter
`rename = "col"`	Read the column `col`; overrides `rename_all`	4
`skip`	Do not read; use `Default::default()`	4
`flatten`	A nested `FromRow` struct, read from the same row	4
`json`	Read a `json` or `jsonb` column through `Json<T>` (serde)	4
`try_from = Type`	Read a `Type`, then convert with `TryFrom`; the type may be quoted	4

Rules, as the derive enforces them: keys are refused if unknown or repeated; `skip` goes with nothing else; `flatten` goes with nothing but itself; `json` and `try_from` exclude each other; every error is reported at its token, all in one build.

Plan for the attributes still to come: `table`, `schema`, `primary_key`, `generated` (Chapter 6); `model` (Chapter 8); `version`, `updated_at` (Chapter 9); `belongs_to`, `has_many` (Chapter 11).

B What each derive generates

For this struct:

```
// orm/shelf-codegen/tests/expand/basic.rs (excerpt)
#[derive(FromRow)]
struct Product {
    id: i64,
    sku: String,
    r#type: String,
    tags: Option<Vec<String>>,
}
```

`#[derive(FromRow)]` generates, as the expansion test pins it down:

```
// orm/shelf-codegen/tests/expand/basic.expanded.rs (excerpt)
#[automatically_derived]
impl ::shelf::FromRow for Product {
    type Columns = (usize, usize, usize, usize);
    fn resolve(
        columns: &[::shelf::__private::tokio_postgres::Column],
    ) -> ::core::result::Result<Self::Columns, ::shelf::Error> {
        ::core::result::Result::Ok((
            ::shelf::__private::column::<i64>(columns, "Product", "id", "id")?,
            ::shelf::__private::column::<String>(columns, "Product", "sku", "sku")?,
            ::shelf::__private::column::<String>(columns, "Product", "type", "type")?,
            ::shelf::__private::column::<
                Option<Vec<String>>,
            >(columns, "Product", "tags", "tags")?,
        ))
    }
    fn from_row_at(
        row: &::shelf::__private::tokio_postgres::Row,
        columns: &Self::Columns,
    ) -> ::core::result::Result<Self, ::shelf::Error> {
        ::core::result::Result::Ok(Self {
            id: ::shelf::__private::get::<i64>(row, columns.0, "Product", "id")?,
            sku: ::shelf::__private::get::<String>(row, columns.1, "Product", "sku")?,
            r#type: ::shelf::__private::get::<
                String,
            >(row, columns.2, "Product", "type")?,
            tags: ::shelf::__private::get::<
                Option<Vec<String>>,
            >(row, columns.3, "Product", "tags")?,
        })
    }
}
```

Plan:

- The other expansion tests (`rename`, `skip_flatten`, `json_try_from`, `generics`) in the same form, each one line per attribute in the text and the full expansion as an excerpt.
- `Table`, `Insert` and `Patch` as their phases add them.

C Phrasebook

Plan: one table per task (map a row, find by key, insert, partial update, filter a page, load children, bulk insert), with the shelf call and the diesel, SeaORM and sqlx equivalents, each checked against the crate's version at the time of writing (check). A first row, to show the shape:

Task	shelf	sqlx	SeaORM
Read rows into a struct	`#[derive(FromRow)]`, `fetch_all`	`#[derive(FromRow)]`, `query_as`	`#[derive(DeriveEntityModel)]` (check)

D Glossary

Draft entries:

- **Derive macro**: a procedural macro attached with `#[derive(…)]`; it receives the item and adds code next to it (Chapter 3).
- **Expansion**: the code a macro produces in place of its invocation, or beside it for a derive; shelf prints it with `pretty` (Chapter 3).
- **Helper attribute**: an attribute a derive declares (`attributes(shelf)`) so the compiler passes it through to the derive (Chapter 4).
- **Hygiene**: the rules that keep a macro's names from clashing with the caller's; paths are not covered, hence the absolute paths in generated code (Chapters 2 and 3).
- **Resolve**: shelf's step that finds and checks each field's column once per statement (Chapter 5).
- **Span**: where a token came from; errors are reported at spans (Chapter 4).
- **TokenStream**: the sequence of tokens a procedural macro receives and returns; `proc_macro2::TokenStream` is the same thing usable outside the compiler (Chapter 3).
- **N+1**: one query for a list, then one per item (Chapter 11).

Plan: the terms of Part II and III as their chapters are written.

E Further reading

Plan:

- *The Little Book of Rust Macros*; the Rust Reference on macros by example and procedural macros.
- syn's and quote's documentation; dtolnay's proc-macro workshop.
- diesel's, SeaORM's and toasty's guides; the main book's Appendix F.

> **Open questions.** (1) Whether the phrasebook belongs here or in Chapter 0, with a short version in the other. (2) Whether B should include every expansion test or one per derive.